FINAL MANUSCRIPT PAST TENSE (Revised 4.0).docx

My Files

My Files

Tarlac State University

Document Details

Submission ID

trn:oid:::26063:130049454

Submission Date

Mar 6, 2026, 10:25 AM GMT+8

Download Date

Mar 6, 2026, 10:34 AM GMT+8

File Name

FINAL MANUSCRIPT PAST TENSE (Revised 4.0).docx

File Size

31.4 MB

152 Pages

22,952 Words

128,403 Characters

53% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

85 AI-generated only 52%
Likely AI-generated text from a large-language model.

5 AI-generated text that was AI-paraphrased 1%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Chapter 1

INTRODUCTION

A. Project Context

Object detection is a computerized technique used to identify and locate specific objects within an image. It involves forming a boundary around the target objects and associating them with their corresponding object category. [1] Object detection has numerous applications, including automating business processes like inventory management in retail and enabling autonomous vehicles to identify and perceive objects in their environment. [2] Object tracking involves extracting features, background subtraction, and identifying the object's position in each frame of the video. [3]

The camera is commonly used for object tracking because it provides stable detection and allows for precise temporal information. Video Surveillance aims to analyze video being captured by a fixed camera in an environment. Visual surveillance [4][5][6][7][8][9] is one of the most important safety monitoring and management systems widely used in many fields. In video surveillance, tracking objects of interest provides dynamic information of key objects for target monitoring and motion characteristics analysis. [10][11] The videos are captured by a camera, which makes sense that the background remains the same. The captured video can be used in manual, semi-autonomous, and fully autonomous systems. [12]

Camera tracking refers to the process of automatically following and capturing the movement of a camera or a target object without human intervention. It involves the use of various devices and systems to accurately track the camera's position, orientation, and lens

data, allowing for precise control and synchronization of the camera's movements and settings. [13] [14]

Webcams are commonly used as sensors for object detection and tracking, especially in surveillance systems. Additionally, utilizing webcams in object detection systems can lead to efficient real-time tracking by extracting features, performing background subtraction, and identifying objects from multiple angles with high resolutions. [15]

YOLOv8 (You Only Look Once Version 8) represents a new object detection model introduced by Redmon and Farhadi. This model stands out for its significant advancement in real-time object detection, offering enhanced speed and accuracy compared to its predecessors. YoloV8 helps in detection of real-world objects [16]

Yunet has 75856 parameters and a size of less than 1/5 of other small-size detectors, the algorithm is a millisecond-level, anchor-free, face detector used for edge devices. Its lightweight design includes a small, effective feature extraction backbone and a simplified pyramid feature fusion neck. The Algorithm reduces redundant parameters, focused on speed and accuracy. [17]

There are numerous algorithms used for detecting objects by a picture or a live feed video, or video, such as YOLOv8, Mean-Shift, Yunet, Lucas-Kanade, Farneback, and MOSSE (Minimum Output Sum of Squared Error) algorithms, OpenFace, and FaceNet. [17] [21] [22] [23] [24] [25] [26] [27] [35] [44] [45]

Object detection are is a relevant field in computer vision for applications such as surveillance, security, autonomous applications, and robotics. Object detection involves identifying and tracking of an object's positions, trajectory, and its movement in real time.

These tasks are useful for real-time monitoring and analysis of objects seen in videos and images. [18][19][20]

B. PURPOSE AND DESCRIPTION

This study was conducted to propose a system that can track and follow objects based on their physical appearance and Facial features seen in a Camera Feed. Using YoloV8, together with YuNet Facial Detection, the system was designed to be ran on a Video Camera using YOLOv8 and YuNet Algorithms, which can track and follow objects and faces.

The Development process, involved the integration of the YOLOv8 and YuNet Algorithm, to Track and follow Objects and faces in on three (3) cameras feeds in the same network using RSTP and USB Cameras connected in the Computer. The Study was limited only in detecting the same objects and faces in the camera with the same physical features, selected by a user, and followed by the YOLOv8 and YuNet Algorithm on Multiple Cameras and no other objects and faces except the one selected by the user to follow.

C. OBJECTIVES

The Objective of the study was to develop an Adaptive Object Tracking and Following using three (3) camera system using YOLOv8 and YuNet Algorithms that would track and follow an object and a face using a USB Camera. The Objective of the system was to track and follow objects and faces by supervised selection of bounding boxes that will follow and track objects on multiple cameras, ignoring other bounding boxes in the camera feed.

Specifically, the study aimed to do the following:

1. To design and develop an Adaptive Object Tracking and Following System incorporating the following features:
    1.1. Real-time object and facial detection and tracking
    1.2. Multi-Camera target tracking and following.
2. To test the performance of the Adaptive Object Tracking and Following System through:
    2.1. Black box Testing
    2.2. Load Testing
        2.2.1. JMeter Response Time Testing
        2.2.2. YoloV8 Object Detection Algorithm Benchmark
        2.2.3. YuNet Face Detection Algorithm Benchmark

D. SCOPE AND LIMITATION

This study focused on designing and developing a system for adaptive object and facial tracking and following using YOLOv8 and YuNeT algorithms. The study was limited to the development of a Webcam system's adaptive tracking and following capabilities. Specifically, object and facial tracking and following capabilities only and did not explore other applications of the camera technology. The system might not have fully worked perfectly when detecting and tracking an object, at the same time, the system could not detect all objects within the range of the cameras. The real-time adaptive object tracking and following was implemented within the webcam's visual range. Low-light conditions or high-contrast lighting scenarios might have resulted in noisy or ambiguous image data, making it challenging for the system to identify and track objects. Single tracking might not have worked when the person being followed was facing the back of the camera or wearing any face coverings, also when the person was in side view. The system's algorithms might not have labeled each person when detected in the camera because of limited datasets during the training of algorithms. For object detection, the only objects that the system could detect were bag, book, bottle, cup, keyboard, remote, mouse, trash bin, mug, and spoon. The compatibility with other Operating Systems was not considered, as the implementation of the software system was done using laptops running on the Windows Operating system. Performance evaluations were conducted in controlled environments. Specifically, within the CCS Building Center wing at Tarlac State University, San Isidro Campus.

Chapter 2

REVIEW OF RELATED LITERATURE AND STUDIES

A. Discussion of Models

Object detection is a subclass of computer vision and is commonly implemented using deep learning algorithms. Deep learning is considered the most advanced approach for object detection. Some older methods like Histogram of Gradients and Scale Invariant Feature Transform have been used for object detection too. These days, CNNs (Convolutional Neural Networks) and R-CNNs (Region Based Convolutional Neural Networks) are among the go-to deep learning algorithms for detecting objects. Researchers have also created other algorithms - Fast R-CNN, Faster R-CNN, YOLO (You Only Look Once), and SSD (Single-Shot Detector) - to make object detection faster and more accurate.[1] Object Detection can detect objects that occupy different proportions of an image's area, ranging from 2% to 60%, making it versatile for different scenarios and object sizes. Offers high-speed object localization, typically greater than 15 frames per second, enabling real-time processing and applications in dynamic environments. A newer version of YOLO uses an adaptive sampling strategy and includes something called a Pico-Fuse feature aggregation module. This combination helps it perform really well on various object detection tests while still running fast enough for real-time use on mobile devices. [2] Object tracking is basically about following and keeping tabs on how people or objects move around in what the camera sees. When it comes to security, tracking potential suspects as they move around gives law enforcement useful intel to catch them. The system

we're proposing uses computer vision methods to track people in CCTV footage and study how they behave. [3]

Video surveillance is all about analyzing footage from stationary cameras in a given area. Visual surveillance [4][5][6][7][8][9] has become one of the most critical safety and monitoring systems out there, used across a ton of different fields - traffic monitoring, aviation, navigation safety, port management, security work, and tracking or identifying objects, just to name a few. When you're doing video surveillance, being able to track specific objects gives you real-time info about important targets and helps you understand how they're moving. [10][11] Since the cameras are fixed in place, the background stays constant throughout the footage, which makes sense. The video you capture can work with manual setups, semi-autonomous systems, or even fully automated ones. [12]

Camera tracking involves keeping tabs on a camera's movement and following it around. Data gets collected about where the camera sits and what direction it's pointing. This data then gets analyzed, which allows the movements to be tracked as they happen in real-time. This technique is used in various fields. Film production is one area where it's applied. Virtual reality and augmented reality also rely on it, and robotics is another field where it shows up regularly. There's a paper titled "Electronic tracking device for camera and related system for controlling image output of the camera" that introduces a trackable camera beacon designed to attach to cameras. The beacon makes tracking simpler and allows for automatic control because it gathers position data and outputs packets that combine position and orientation information. That being said, the paper doesn't specifically go into camera tracking as a concept on its own. [13] [14]

Webcams get used a lot as frame-based cameras when detecting objects. They work pretty well for stable detection as long as the lighting is normal. When it comes to object detection, webcams grab frame images at a set rate. This lets them identify objects in the frames and figure out where those objects are located. The detection results from webcams usually come as bounding boxes. The top-left corner of each bounding box shows where the object sits in the frame. [15]

YOLOv8 is the newest version of the YOLO (You Only Look Once) algorithm. The algorithm has been changing and improving over time. YOLO versions work really well for industrial surface defect detection and object detection because they classify things accurately and run in real-time. [28] These days, YOLO has become an important technology for real-time object detection. It gets used a lot in robotics, self-driving cars, and video surveillance. [29]

The proposed YuNet achieves 81.1% mAP (single-scale) on the WIDER FACE validation hard track with high inference efficiency (1.6ms per frame at 320 × 320 resolution on Intel i7-12700K). YuNet has the best trade-off between accuracy and speed. It has only 75856 parameters and is less than 1/5 of other small-size detectors. [17] [30] YuNet's unique advantages have garnered significant popularity on GitHub, accumulating over 11K stars. [35]

Object tracking is the process of segmenting an object of interest (or multiple objects) from a video sequence. It keeps track of the object's movement, direction, and when it gets blocked from view. The system being proposed shows strong detection and tracking performance. It has low false positive rates and can accurately locate pedestrians. This holds up even in tough situations like when objects are blocked or lighting conditions

change. Object tracking involves finding the trajectory and identifying the different positions of an object in each frame of a video. In surveillance and security management systems, object detection is how objects get identified and tracked in real-time. What happens is multiple cameras view an object from various angles and at high resolutions. The main parts of object detection involve extracting features, subtracting the background, and identifying the object that was extracted. Object tracking, on the other hand, focuses on finding the trajectory and different positions of an object in each frame of a video. Going through this procedure makes it easier to understand the object's movement and behavior as time goes on. [18][19][20]

Object tracking and following algorithms help locate specific objects in videos or live footage and keep monitoring how they move. This makes it possible to do things like autonomous navigation, surveillance work, and human-computer interaction. There are lots of different algorithms out there for object tracking and following. Each algorithm has its strengths and weaknesses. YOLOv8 is known for being fast and efficient. This makes it work well when used in real-time applications. Detecting multiple objects in a single image or frame is something it can handle. When tested on different object detection benchmarks, the algorithm achieves high accuracy. People often pair it with trackers so it can monitor things continuously. The main drawbacks are that it has limited tracking capabilities on its own and can be computationally expensive to run. [21] [22]

The Mean-Shift algorithm performs well on low-powered devices because it's relatively simple to implement and doesn't require an explicit object model. This makes it suitable for tracking objects that have diverse appearances. There are some disadvantages to using this algorithm though. It has limited capabilities for tracking multiple objects. Fast

motion causes it to struggle. It can also get stuck in local minima of the similarity landscape, which leads to tracking failure. [23]

Yunet's lightweight design has numerous support in GitHub. It includes a streamlined pyramid neck and a small feature extraction backbone. YuNet, with only 75856 parameters and less than 1/5 the size of other small-scale detectors, offers the optimum balance between speed and accuracy. [17][35]

Lucas-Kanade is easy to understand and debug, it also provides accurate estimations of object movement based on pixel-level intensity changes and efficient implementation suitable for real-time applications on devices with limited resources. The drawback of this algorithm is that it is sensitive to noise and has a limited tracking range. [24] [25]

Farneback can handles larger object movements compared to Lucas-Kanade. It also has dense motion estimation offering rich data for object tracking and can handle deformations due to its global optimization approach. Unfortunately, this algorithm is computationally expensive and sensitive to noise. [26]

MOSSE (Minimum Output Sum of Squared Error) algorithm requires only minimal training data in which single image of the object to initialize the tracker, making it suitable for rapid deployment. This algorithm is fast and efficient, robust to appearance variations, continuously updates its correlation filter to adapt to object appearance changes, maintaining tracking accuracy. The only drawback of this algorithm is that the performance degrades when the object rotates significantly out of the image plane, also it

easily distracted by complex backgrounds, leading to tracking error and has limited scale adaptation. [27]

The OpenFace detection algorithm is a facial recognition system that aims to achieve higher accuracy and validation rates on low-resolution face images, The Extended Openface system utilizes facial landmark detection, head pose, and eye gaze features to enhance facial recognition accuracy on low-resolution images. While its weakness is OpenFace detection may experience a gradual decrease in accuracy and validation rate on low-resolution images over time, leading to a decrease in performance. [44]

The Facenet detection algorithm is a powerful tool in facial recognition technology. It is used for face feature extraction in image processing applications, particularly in the context of facial recognition. While its weakness is that the effectiveness of the FaceNet algorithm for masked face identification may be reduced by masks hiding facial information, which reduces the effectiveness of face recognition. [45]

Table 1: Systems' Functionality and Feature Matrix

SOFTWARE	DESCRIPTION	1	2	3	4	5	6
YawCam	Transforms your webcam into a motion-detecting, basic object-tracking tool for surveillance and monitoring, offering real-time alerts and region-based tracking.	✓	✓	X	X	X	X

TinyTrack	Is a lightweight, real-time, and accurate multi-object tracking system for research and development.	✓	✓	✓	✓	✓	X
Improved Vehicle Detection and Tracking Using YOLO and CSRT[3]	Combines the accuracy of YOLO object detection with the smoothness of CSRT3 tracking to reliably follow specific vehicles in real-time.	✓	✓	X	✓	✓	X
Ultralytics YOLO[5]	is a high-performance, open-source deep learning model for real-time object detection in images and videos.	✓	✓	X	✓	✓	X
CamTrack WebCam Software	enhances your webcam, enabling real-time face and object tracking with smooth pan and zoom control for improved videocalls, presentations, and streaming.	✓	✓	✓	X	X	X

1. Real-time Detection
2. Object Tracking

3. Adaptive Tracking and Following
4. High Accuracy
5. Robust to Noise
6. Selective Object Following

Based on the review of the listed object tracking and following software, it was evident that real-time detection and high accuracy were common features across all the solutions, highlighting their crucial importance. Object tracking was a minimal availability feature, with only two software lacking this function. After reviewing the object tracking and following software listed, it became clear that real-time detection and high accuracy showed up in all the solutions. This highlighted how important these features are. Object tracking wasn't available in much of the software - only two lacked this functionality, which showed its significance in terms of overall capabilities. The system needed to incorporate object tracking, adaptive tracking and following, robustness to noise, and selective object following. This would make it advanced and versatile. We got this approach by analyzing what existing software could actually do. The aim was boosting the system's performance and making it useful across various applications - things like surveillance, autonomous camera tracking, and object tracking.

Table 2: Algorithms' Functionality and Feature Matrix

Algorithms	Real-time	Robust	Lightweight	GPU Compatible
YuNet	✓	✓	✓	✓
FaceNet	✓	✓	X	✓
OpenFace	✓	X	✓	✓
Yolov8	✓	✓	✓	✓
Mean Shift	✓	X	X	✓
Lucas Kanade	✓	✓	X	✓
Farneback	✓	X	X	✓
MOSSE (Minimum Output Sum of Squared Error)	✓	✓	X	✓

1. **Real-time** – this feature allows the system to continuously identify and analyze objects and faces as they occur, with minimal delay.
2. **Robust** – This capability enables the system to recognize events or objects even in difficult situations with accuracy and dependability.

3. **Lightweight** – Within the field of detection algorithms, models created with efficiency in mind are referred to as lightweight. With the help of this functionality, devices with constrained resources can detect objects or events quickly and efficiently. They put efficiency and speed first, which makes them ideal for real-time applications on mobile devices and in settings with limited resources.
4. **GPU Compatible –** This feature makes it possible for detection algorithms to operate on any CPU and PC, enabling it to meet the demands of these algorithms. Better detection of objects, tracking, and facial recognition algorithms are made possible by the faster and smoother operation of algorithms on more recent GPUs with larger VRAM.

In Table 2, Algorithms for detection are being compared. Yunet, FaceNet, OpenFace, Yolov8, Mean-shift, Lukas Kanade, Farneback, and MOSSE can detect in real-time. Only OpenFace, Mean-shift, and Farneback doesn't recognize object in difficult situation. Yunet, OpenFace and Yolov8 are algorithms that can detect quickly. While all of them are GPU compatible.

Table 3: Literature Matrix

REFERENCE	DESCRIPTION	STRENGTH	WEAKNESS
"Object detection using YOLO: challenges, architectural successors, datasets and applications" by Diwan, T., Anirudh, G. & Tembhurne, J.V (2022)	a comprehensive review of single stage object detectors specially YOLOs, regression formulation, their architecture advancements, and performance statistics	High accuracy in object detection	Does not detect the same object for prolong period of time
Parking Time Violation Tracking Using YOLOv8 and Tracking Algorithms by Sharma, N et.al (2023)	An approach that can introduce a low-cost time violation tracking system using CCTV, Deep Learning models, and object tracking algorithms, with YOLOv8	High accuracy in multiple object tracking	Algorithm focus only in multiple object tracking
"Object tracking algorithm for unmanned surface vehicle based on improved mean-shift method" by Xiang Z, Tao T, Song L, et al (2020)	Tracking algorithm for unmanned surface vehicle based on an improved mean-shift optimization algorithm	Solves the pixel interference and occlusion failure in tracking	Issues in providing bounding box predictions
"Real-Time Multiple Object Tracking: A Study on YOLO Algorithm" by Wang et al. (2018)	Presents an extension of the Mean Shift algorithm for object tracking with multiple cues	Real-time detection of multiple objects	Limited in complex and cluttered environments
"Robust Truth Discovery Scheme Based on Mean Shift Clustering Algorithm" by Jingxue Chen et.al (2021)	Presents an extension of the Mean Shift algorithm for object tracking with multiple cues	Robust approach for multiple cue integration	Increased complexity with multiple cues

GitHub-BrunaStef/YOLOv8-Object-Tracking: YOLOv8 Object Tracking using PyTorch, OpenCV and DeepSORT by B. Stef (2021)	a GitHub repository that demonstrates how to use YOLOv8 for object tracking with DeepSORT, a deep learning-based tracking algorithm.	provides a practical and accessible way to implement YOLOv8 for object tracking. It uses a state-of-the-art tracking algorithm (DeepSORT) that can handle occlusions and identity switches.	It does not provide a theoretical or empirical analysis of the performance or accuracy of the tracking method. It does not provide a theoretical or empirical analysis of the performance or accuracy of the tracking method.
Real-time Object Detection and Tracking with YOLOv8 and Streamlit – GitHub by Coding Mantras (2021).	another GitHub repository that shows how to integrate YOLOv8 and Streamlit, a Python web application framework, for real-time object detection and tracking in video streams	It demonstrates how to use Streamlit to create a user-friendly and customizable interface for object detection and tracking.	It does not explain how the segmentation task is performed or how it affects the tracking results.
Object Identification and Tracking Using YOLO Model: A CNN Based Approach by Singh et. Al (2021)	a book chapter that presents a method for object identification and tracking using YOLO and a convolutional neural network (CNN). The method consists of three steps: object detection using YOLO, feature extraction using CNN, and object tracking using a Kalman filter. The chapter describes the implementation and	It shows the effectiveness and efficiency of the proposed method, achieving high accuracy and low latency in object identification and tracking.	It does not use YOLOv8 or Mean-Shift for the object detection and tracking, which may affect its performance and applicability to more challenging scenarios.

	evaluation of the method on various datasets, such as PASCAL VOC, COCO, and KITTI		
Real Time Object Detection System with YOLO and CNN Models: A Review by Viswanatha et al (2022).	a review paper that summarizes the state-of-the-art methods for real-time object detection using YOLO and CNN models. The paper covers the main concepts and techniques of object detection, such as bounding box regression, anchor boxes, non-maximum suppression, and feature pyramid networks.	It analyzes the advantages and disadvantages of each YOLO version, as well as the trade-offs between speed and accuracy	It does not provide any experimental results or comparisons of the object detection methods, which may limit its usefulness and validity.
A Comprehensive Review of YOLO: from YOLOv1 to YOLOv8 and Beyond by J. R. Terven et al (2022)	The paper reviews the different versions of YOLO, from YOLOv1 to YOLOv8, highlighting their main characteristics and improvements.	summarized the essential lessons from YOLO's development and provide a perspective on its future, highlighting potential research directions to enhance real-time object detection systems.	It does not focus on the object tracking task, which is also part of the topic of interest.

YuNet: A Tiny Millisecond-level Face Detector by Hanyang, Peng., Shiqi, Yu. (2023).	Research about the development of the smallest image detector that used CNN as its backbone.	Could capture the tiniest face on the spectrum depends on how clear its pixel is. Suitable for crowd source and real time video.	Only focuses on face detection.

Shown in Table 3, the analysis revealed common trends in strengths, such as robust performance and real-time capabilities, as well as weaknesses, including reduced accuracy for small objects and performance degradation in certain conditions. These findings served as the foundation for identifying the system's novelty and crafting solutions to address the identified limitations. Developing the proposed system relied heavily on the insights and lessons learned from these literature reviews. This highlighted how important evidence-based guidance is when creating an effective adaptive tracking and following system.

B. CONCEPTUAL FRAMEWORK

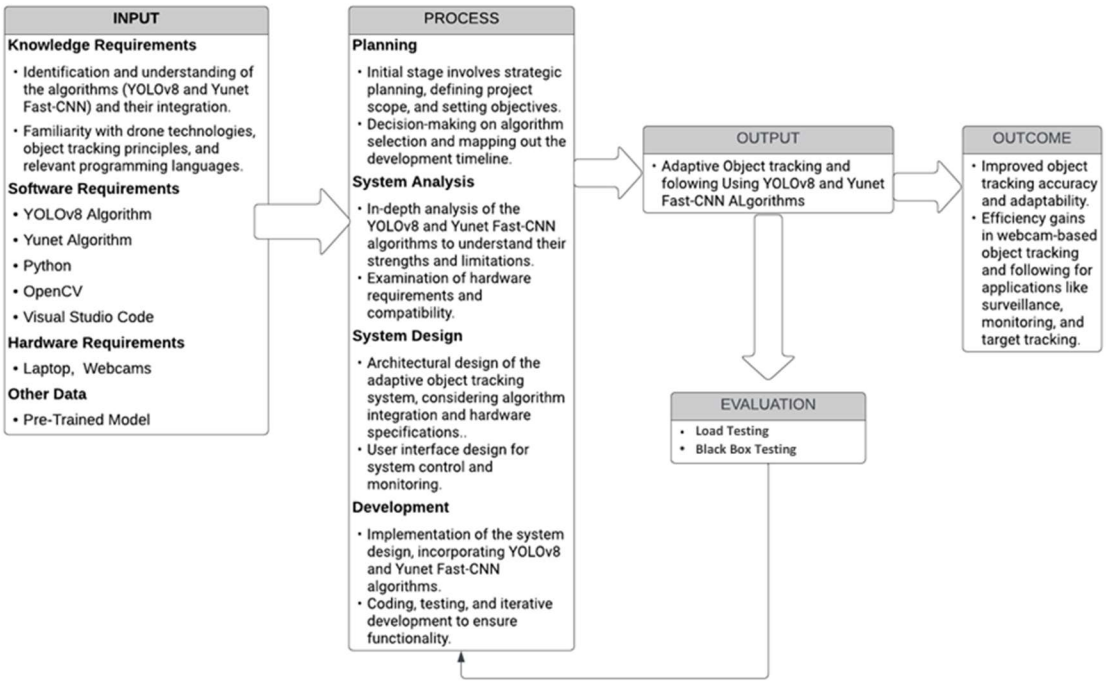


Figure 1: Conceptual Framework

Figure 1 shows the conceptual framework diagram. It illustrated how the 'Adaptive Object Tracking and Following Using YOLOv8 and Yunet System' was developed. The process started with inputs. Knowledge requirements were one input. Software requirements were another - these included the YOLOV8 and Yunet algorithms, Python, OpenCV, and Visual Studio Code. Hardware requirements were the third input, which consisted of a laptop, web camera, and router.

The process broke down into four different phases. These were planning, system analysis, system design, and development. Planning involved making strategic decisions on which algorithms to select and what the project scope should be. During the system analysis phase, YOLOV8 and Yunet algorithms were analyzed along with hardware compatibility. During the system design phase, the architecture for the adaptive tracking

and following system got created. This phase also involved designing user interfaces for control and monitoring. Development was the next phase. It included implementing the system design. YOLOV8 and Yunet algorithms were incorporated at this stage. Coding happened after that. Testing came next, along with iterative development that made sure everything worked right. All of this resulted in the final system being proposed.

The evaluation phase included Black Box Testing and Load testing. Furthermore, feedback and review improved the process of the development of the system. Lastly, the outcome involved improved object tracking accuracy and adaptability, and efficiency gains in webcam tracking for applications like surveillance and monitoring.

C. Definition of Terms

Adaptive Object Tracking and Following	Capacity to stay aware of the location of the monitored object, stopping motion temporarily but still having the ability to quickly resume tracking as soon as the object is visible once more.
High Accuracy	Refers to the system's ability to accurately detect and track objects. High accuracy is essential to ensure the reliability of the system.
Multiple cue integration	Refers to the process by which the brain combines information from multiple sensory cues or sources to form a coherent perception of the environment. The integration of these cues helps create a more accurate and reliable perception of the object's position.

Object Tracking	Refers to the system's ability to keep track of the detected objects over time. It's important in scenarios where the object of interest can move, and the system needs to follow its trajectory.
Occlusion	Refers to the situation where one object in a scene is partially or completely hidden by another object from the viewpoint of an observer or a camera.
Real-time Detection	This feature refers to the system's ability to detect objects in real-time. It's crucial for applications that require immediate response
Robustness	refers to the algorithm's ability to effectively locate and track modes or peaks in a probability density function, even in the presence of noise, clutter, or varying densities

Robust to Noise	refers to the system's ability to perform well even in the presence of noise in the input data. Noise can come from various sources, such as sensor errors, environmental factors, or interference, and a good system should be robust to these factors.
Selective Object Following	Empowers the user to actively choose a specific object of interest, enabling the system to initiate tracking and following maneuvers based on the user's selection.
Yunet	It is a CNN-based face detector developed by Chengrui Wang and Yuantao Feng. It is a very lightweight and fast model. With a model size of less than an MB, it can be loaded on almost any device.

Chapter 3

TECHNICAL BACKGROUND

A. Software Development Requirements

Table 4: Software Requirements

Sources	Description
Windows 	The research team used the Windows Operating System all throughout the proposed research and system development.
Visual Studio Code 	A lightweight and powerful source code editor used for the development and testing of the proposed system. The software was used to code and implement the system.
Nvidia CUDA Toolkit 	Provides a development environment for creating high-performance, GPU-accelerated applications. Used to enable Graphics Processing Unit Acceleration during the Runtime of the System.
Nvidia CUDA Deep Neural Network (CUDNN) 	A GPU-accelerated library of primitives for deep neural networks. Used to enable high performance GPU acceleration during the runtime and inference of the algorithms used in the System.

Python 	Primary programming language for the System algorithm implementation. Includes downloadable modules required by the system to develop and run the system and the algorithm.
OpenCV 	OpenCV is one of the most widely used computer vision library for deploying computer vision applications. This will be used for image and video processing.
Apache JMeter 	Apache JMeter is an open-source load testing software designed to load test functional behavior and measure performance of applications during development. This was used to test the Response time of the System running at a given time.

Shown in table 4, all the software requirements that the researchers used for designing and developing the Adaptive Object Tracking and Following System. The researchers used the Windows Operating System to develop and test the system. Windows gets used widely for development and is user-friendly. Visual Studio Code (VS Code) provided a Code Editor that was flexible and effective. What it mainly did was make the software development lifecycle stages easier. It did this by integrating a Code Editor, Command Line Interface, and extensions into one program. The focus was on developing

systems using Python, which was the main language for developing Machine Learning Algorithms. Nvidia CUDA included a toolkit that included GPU-accelerated libraries, debugging and optimization tools, and a runtime library. This enabled the GPU-Acceleration in Nvidia graphics cards using Compute Unified Device Architecture (CUDA) for better performance during System Runtime. The system runtime then used Nvidia's CUDA Deep Neural Network (CUDNN). This is a GPU-accelerated library that's been highly optimized. It provides a collection of routines designed specifically for deep neural network computations. The CUDA parallel computing platform was what cuDNN got built on. This let it leverage the parallel processing capabilities that NVIDIA GPUs have, which accelerated deep learning operations.. Used for developing the system, which was a library of programming functions primarily for real-time computer vision. Apache Jmeter was used to Load test a software or web application. It was used to test the Response time of the system at a given time.

B. Hardware Development Requirements

Table 5: Hardware Requirements

HARDWARE	SPECIFICATION
Operating System	Microsoft Windows 10 Professional (or Newer)
Motherboard	ASUS Prime H310M-E R2.0 or Newer (Minimum)
Processor (CPU)	Intel(R) Core (TM) i3 CPU @ 2.00GHz, 2 Cores, 4 Logical Processors or Higher (Minimum)
Architecture	x64-based Laptop
Graphics Processor	Intel UHD Graphics 620, with a base clock speed of 350 MHz and a maximum dynamic frequency of 1.05 GHz or Higher (Minimum)
Storage Device	Kingston A400 240GB SATA SSD or Higher (Minimum)
RAM	Crucial 4GB DDR4 2400MHz or Higher (Minimum)
Web Camera	Quantity**:** 3 Web Cameras Video Resolution**:** 720p Frame Rate**:** 30 FPS White Balance : Indoor Exposure : Locked ISO : Auto

Shown in table 5, A strong processor was essential for computers; ideally, it should have been a multi-core processor with a clock speed optimized for effective object tracking and real-time picture processing. To manage the computational load, adequate RAM was necessary, especially when working with big image collections or live video streams. Sufficient storage capacity was an additional requirement, as it held the operating system, development tools, datasets, and video recordings produced during trials. The three

webcams were directly connected to a set of USB ports to be used immediately when the program ran. Using three high-resolution cameras with a minimum resolution of 720 pixels or higher, these cameras had a frame rate of at least 30 frames per second or higher, making them suitable for real-time tracking.

C. Network Architecture

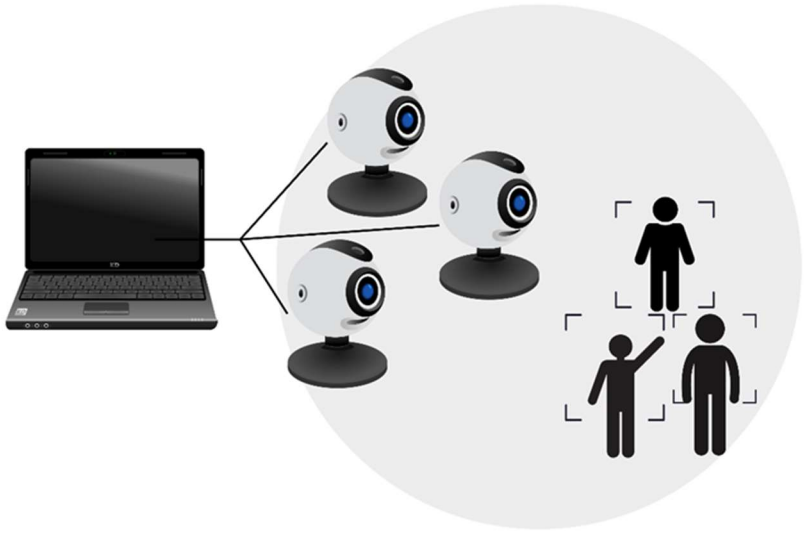


Figure 2: Network Architecture

Figure 2 shows how the system's network architecture was built. The goal was providing smooth coordination and communication between two essential parts - the Laptop and Webcams. The laptop was a symbol for the central control unit. This was the hub for system administration and information sharing. A set of USB Ports established the communication link between the Laptop and Webcams, which enabled control and data exchange in real-time. The Laptop got connected to three Webcams. As the cameras captured video feeds, the Laptop processed this data using sophisticated algorithms for object detection and tracking. Meanwhile, whatever entities the system's camera tracked got represented by the Objects component. These could be various types of targets. The system's task was detecting and following them using predefined algorithms. At the same time, the Laptop and camera worked together. Control commands got sent while real-time tracking information got received.

D. Sources of Data for Development and Testing

Table 6: Sources of Data

Sources	Description
YOLOv8 Algorithm Documentation	Official YOLOv8 documentation, research papers, and related publications; Information on algorithm architecture, parameters, and usage guidelines; 2023
Sample Object Datasets for Testing	Publicly available object detection datasets (e.g., COCO, Pascal VOC); Annotated images with labeled objects for testing algorithm accuracy
Github	GitHub is a code hosting platform with lots of repositories that the researchers can use for different programs. The proponents will get different datasets from GitHub to be used in training our object tracking and following system.
Google Scholar	A freely accessible web search engine that indexes articles, theses, books, and conference papers, providing a comprehensive platform for academic research.
Library	A place or institution that houses collections of books, documents, and other materials for people to borrow or access for research, education, and reading.

Chapter 4

METHODOLOGY

A. Research Design

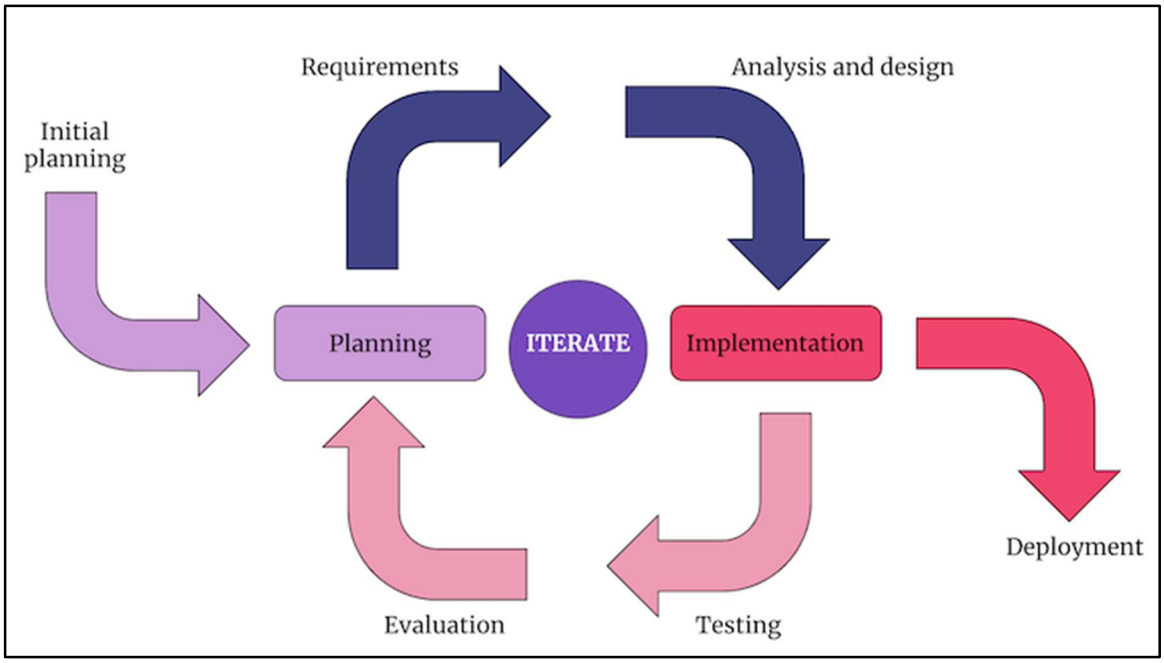


Figure 3: Iterative and Incremental Software Development Life Cycle (SDLC)

Shown in figure 3, the Software Development Life Cycle (SDLC) selected is shown in Figure 3. The approach chosen was Iterative and Incremental Development (IID). This method suited the study well since the study's nature was dynamic and ever-changing. It involved incorporating algorithms into an adaptive object tracking and following system. The system operated in real-time.

The development process began with thorough planning. This step defined the project's goals, objectives, and salient characteristics. YOLOv8 is a reliable object detection algorithm, The phase includes using YOLOv8 algorithm's for object detection

and YuNet algorithm's bounding box and face detection, both running in real time. Combining the Algorithms required user control by clicking and adding snapshot of the object in real time, and follow the object on another camera.

It includes supervised interaction such as, switch to "Single Object Tracking", to limit the bounding box to one selected object or face only, then "Select Target" to snap a picture of the object or face by clicking the bounding box, and "Follow Target" on multiple cameras, by matching the visual color of the snap shot and what is seen in the live camera. The system reached deployment stage and delivered incremental functionalities. Adaptive functionality was integrated into these. Constant monitoring ensured the system performed well. Problems that came up during deployment got addressed quickly. User feedback and data on adaptive technique efficacy got collected in the evaluation phase. This was to evaluate system performance in real-world scenarios

The IID approach brought flexibility and adaptability to the development process. Frequent iterations took place. Testing and refinement cycles happened regularly too. These allowed Yunet and YOLOv8 algorithms to be integrated into a coherent and adaptable tracking and following system. This aligned with the study's objectives.\

Table 7: SDLC

Tasks	Description	Durations
Initial Planning	Held meetings to review the manuscript	1 week
Planning	Design and development of the system comprising its initial features up to the final system.	3 weeks
Requirements	Hardware preparation	3 weeks
Coding	Code and integrate the Yolov8 and Yunet algorithms for object detection.	4 weeks
Testing	Operate the system through testing, including functionality, to identify defects and improve overall quality and reliability.	3 weeks

Table 7 shows the Software Development Life Cycle (SDLC) for this project. It began with initial planning, where project requirements got reviewed over one week. A week of detailed planning followed for designing and developing the system's initial features to its final form. Hardware preparations happened next - they took three weeks to make sure the necessary infrastructure was ready. A four-week coding phase followed. The YOLOv8 and YUNet algorithms got integrated for object detection in this phase. Testing came last with two weeks allocated to it. The system got operated and defects were identified, improving overall quality and reliability. This approach was structured, so it ensured progression happened systematically from planning all the way through development, deployment, and testing. The result was delivering a system that was robust and high-quality.

Table 8: Iterative

Iteration	Tasks	Workdays	Time Frame
1	Initialize Camera in the Graphical User Interface.	6 Working Days	1 week
2	Integrate the YoloV8 algorithm by Ultralytics into the System	9 Working Days	2 weeks
3	Integrate the Mean-shift Algorithm in the System	13 Working Days	2 weeks
4	Implementing Single Tracking with mouse Selection	16 Working Days	3 weeks
5	Replace the Mean-Shift Algorithm and Integrate the YuNet Algorithm to the Graphical User Interface	8 Working Days	2 weeks
6	Apply Optimization in the Camera inference performance of the System using Dedicated GPU	14 Working Days	2 weeks
7	Setup and Access IP CCTV Camera In the Network Router	5 Working Days	1 week
8	Find the IP CCTV Camera IP Address for Live Video Feed Access via the Network Router	9 Working Days	2 weeks
9	Access IP CCTV Cameras in the Graphical User Interface	6 Working Days	1 week

10	Adjust Code for Multiple Camera Inferences using the three IP CCTV Cameras.	7 Working Days	1 weeks
11	Integrate Buttons in the Graphical User Interface	5 Working Days	1 week
12	Improve YuNet Algorithm Inference Performance using OpenVino Execution Provider via ONNX Runtime	12 Working Days	2 weeks
13	Modify the system to work on three webcams	5 working days	1 week
14	Re-apply object detection implementation of combined YuNet and Yolo	11 working days	2 weeks
15	Implement Re-identification system	12 days	2 weeks
16	Integrate database and crude operations	6 working days	1 week
17	Camera view implementation and control panel creation	5 working days	1 week
18	Implement Multi-tracking mode	6 working days	1 week
19	Implement single tracking mode	28 working days	4 week
20	Add save and load target functionality	6 working days	1 week
21	Implement Upload Image tracking	6 working days	1 week

Detailed Iterative Plan:

Iteration 1: Initialize Camera in the Graphical User Interface (GUI)

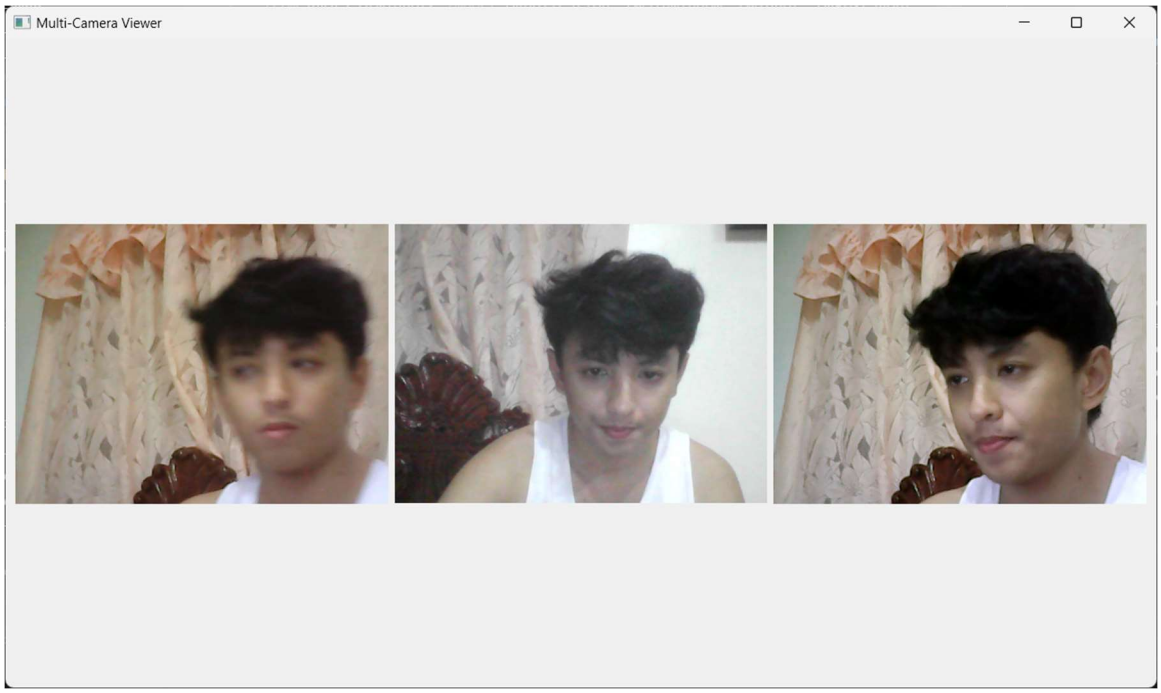


Figure 4: Iteration 1

- **Tasks**:
    - Develop the initial setup for camera integration within the GUI.
    - Ensure basic functionality for camera access and display.
- **Workdays**:
- **Time Frame**: 1 week
- **Review & Feedback**:

Figure 4 shows the initial iteration of the Multi-Camera Viewer application. It provided a solid foundation for displaying three camera feeds simultaneously in a PyQt5 GUI. The code used QMainWindow, QWidget, and QLabel components

effectively to create a user-friendly interface. QTimer got used to ensure smooth updating of camera frames at regular intervals. cv2.VideoCapture was implemented for each camera. Frames got converted to QImage and QPixmap objects. This demonstrated a good understanding of how to integrate OpenCV with PyQt5.

Review: Following the code was easy because it was well-structured. It achieved the main goal successfully - showing multiple camera feeds in one window. QVBoxLayout and QHBoxLayout layouts were used. Camera feeds got arranged properly because of this. They also responded well when the window got resized.

Feedback: To enhance this iteration, some suggestions came up. Adding error handling for cases where cameras might not be available or fail to initialize was one idea. Implementing a method to release camera resources when the application closed was another. This would have improved resource management. Future iterations could have benefited from adding certain controls. Starting and stopping individual camera feeds was one control. Adjusting camera properties was another. Overall, this served as an excellent starting point for building more complex multi-camera applications.

Iteration 2: Integrate the YoloV8 algorithm by Ultralytics into the System

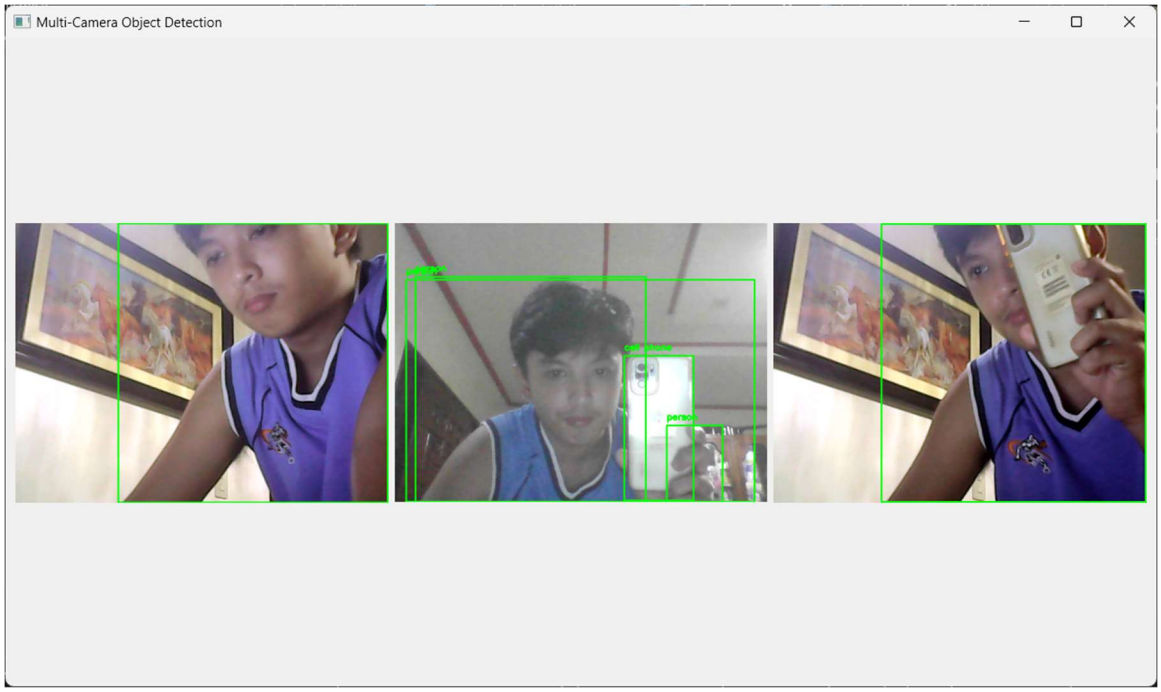


Figure 5: Iteration 2

Tasks:

- Add YoloV8 Algorithm to the system for object detection.
- Ensure basic compatibility and functionality.

- **Workdays**: 9
- **Time Frame**: 2 weeks
- **Review & Feedback**:

Figure 5 shows the second iteration. It successfully integrated the YOLOv8 algorithm by Ultralytics into the multi-camera system. The code built on what the previous iteration had by adding object detection capabilities to each camera feed. The YOLO model got initialized in the constructor. It got applied to each frame in the update_frame method. Detected objects were visualized by a bounding box indicators, following detected objects in the live camera feed.

Review: The YoloV8 was integrated effectively, used by three cameras where both the YOLOv8 and YuNet Algorithm's Models were used. Detected objects and faces were seen on the three live cameras. The code had clear structure and readability.

Feedback: This iteration marked a substantial improvement in the application's capabilities happened in this iteration. Further enhancement of the system could come from several additions. One suggestion was adding options for different YOLO model selection or detection threshold adjustments for users. Multiple camera feeds might make performance optimization necessary for smoother real-time processing. Adding a save or export feature for detection results could help as well. Overall, advanced computer vision capabilities got integrated into the multi-camera viewer successfully, which this iteration demonstrated.

Iteration 3: Integrate the Mean-shift Algorithm with Yolov8 in the System

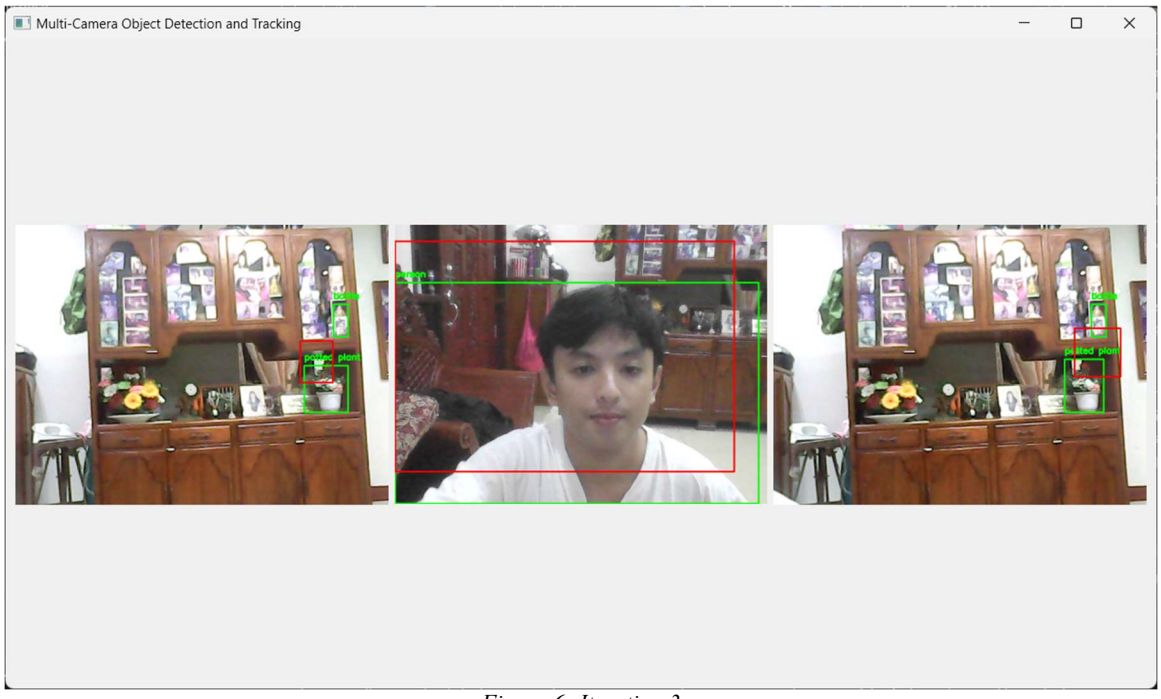


Figure 6: Iteration 3

Tasks:

- Integrate the Mean-Shift Algorithm to the system
- Ensure basic compatibility and functionality.

- **Workdays**: 13
- **Time Frame**: 2 weeks
- **Review & Feedback**:

Figure 6 shows the third iteration. It successfully integrated the Meanshift algorithm into the existing system. This enhanced it with object tracking capabilities. The code built on previous iterations by adding tracking functionality. This complemented the YOLO object detection. The Meanshift algorithm got implemented

to track detected objects across frames. It provided continuous tracking even when YOLO detection might have failed in some frames.

Review: The integration of Meanshift tracking was well-implemented. It worked together with the YOLO detection. The code kept its clear structure while this significant feature got added. HSV color space got used for tracking. Back projection was implemented too. This demonstrated a good understanding of computer vision techniques. The system now provided a more robust solution for detecting and tracking objects across multiple camera feeds.

Feedback: This iteration showed an improvement in the application's capabilities, combining detection and tracking for enhanced performance. Adding a feature to reinitialize tracking when it failed could have enhanced robustness. Performance optimization might have been necessary for smoother real-time processing with multiple cameras. Overall, this iteration showed successfully how both detection and tracking capabilities could be integrated into the multi-camera viewer. This provided a more comprehensive solution for monitoring objects.

Iteration 4: Implement Tracking with mouse Selection

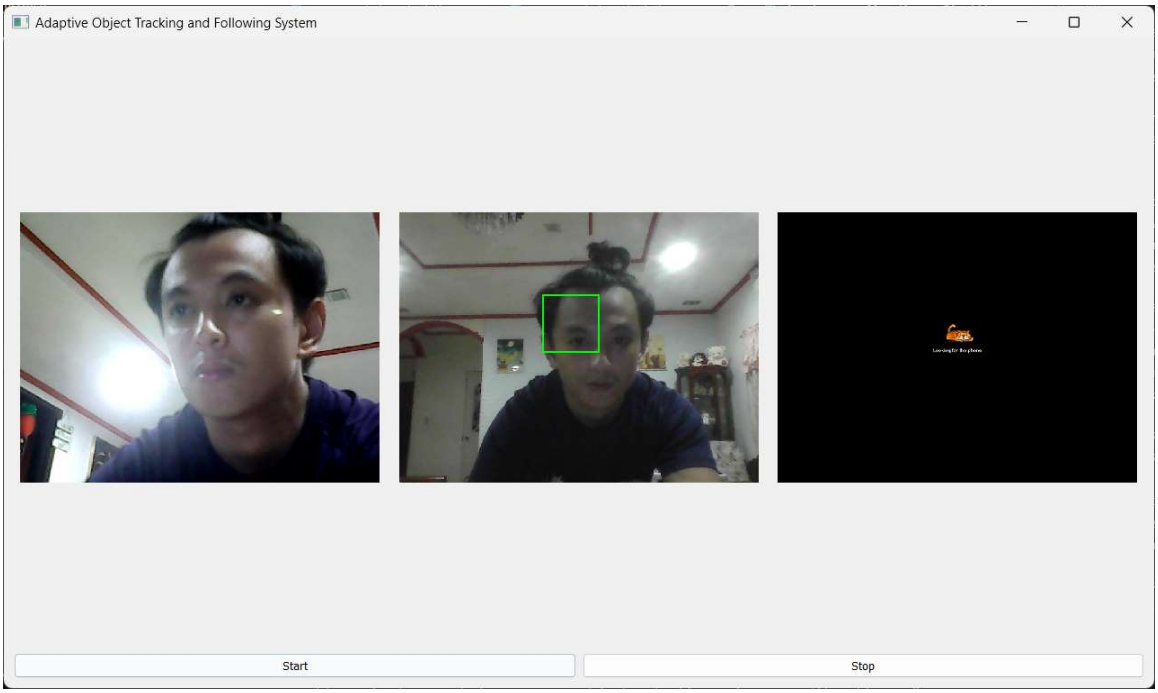


Figure 7: Iteration 4

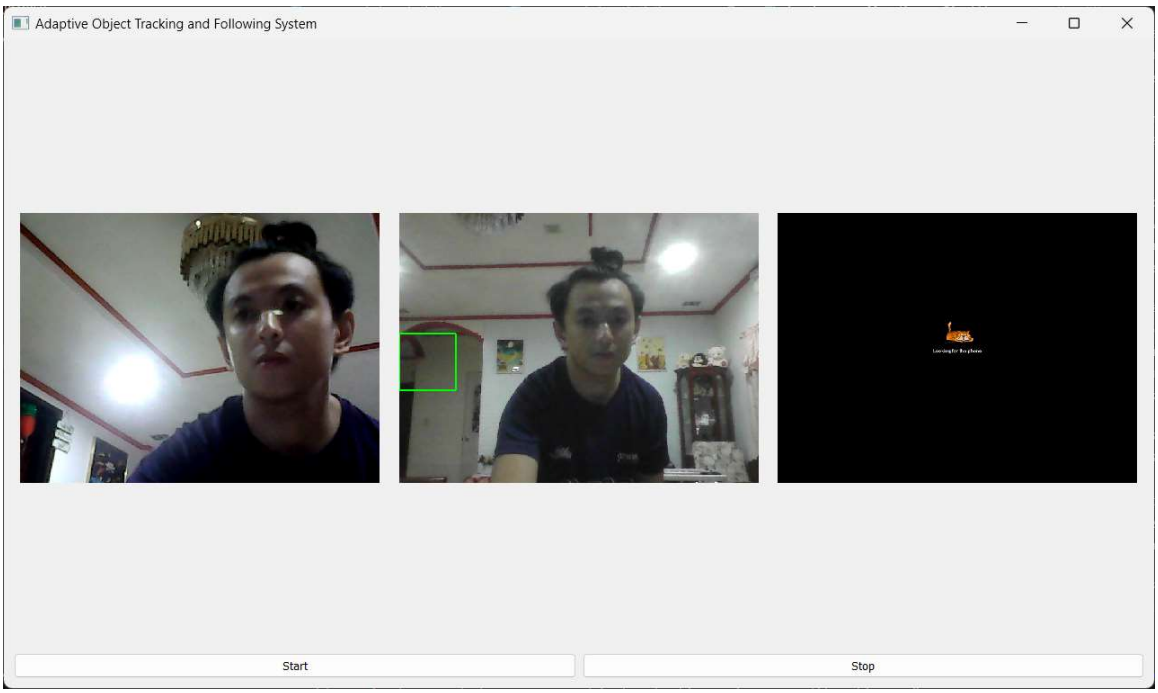


Figure 8: Iteration 4.2

- **Tasks**: Integrate the tracking Algorithm using mouse selection
- **Workdays**: 16
- **Time Frame**: 3 weeks
- **Review & Feedback**:

Figures 7 and 8 show the iteration that implemented single tracking with mouse selection. Users could click on an object in any camera feed to start tracking. The main changes that happened included several things. A select_roi method got added to handle mouse clicks and initialize tracking. The update_frame method was modified so it only tracked in the selected camera. A tracking_camera variable was added to keep track of which camera was currently doing the tracking. The system now let users select a target by clicking on it in any camera feed. Once they selected something, the system used the Meanshift algorithm to track the object in whichever camera feed was selected.

Review and feedback: This iteration implemented user-initiated tracking successfully. In terms of user interaction and control, this marked a significant improvement. Structure in the code was maintained while functionality for object selection and single-camera tracking got added.

However, as noted, The Meanshift algorithm had some limitations though when used in this multi-camera setup. Feature matching across different camera views wasn't something it could handle. This made it hard to track consistently across multiple cameras. Sometimes the algorithm would lose track of the object.

To address these issues, YuNet was one such algorithm. This was a lightweight face detection model that could have been adapted for general object detection and tracking. YuNet's feature extraction capabilities could potentially be used to create a more robust tracking system. This system could maintain object identity across multiple camera views. Next steps could have included several things. Implementing YuNet or a similar feature-based detection and tracking algorithm was one. Developing a method to match object features across different camera views was another. Implementing a re-identification system to maintain object identity when switching between cameras could help. Adding a feature to reinitialize tracking when the object was lost was also an option. Improving the user interface to provide feedback on tracking status and allow for manual corrections was the last suggestion.

Iteration 5: Replace the Mean-Shift Algorithm and Integrate the YuNet Algorithm to the System

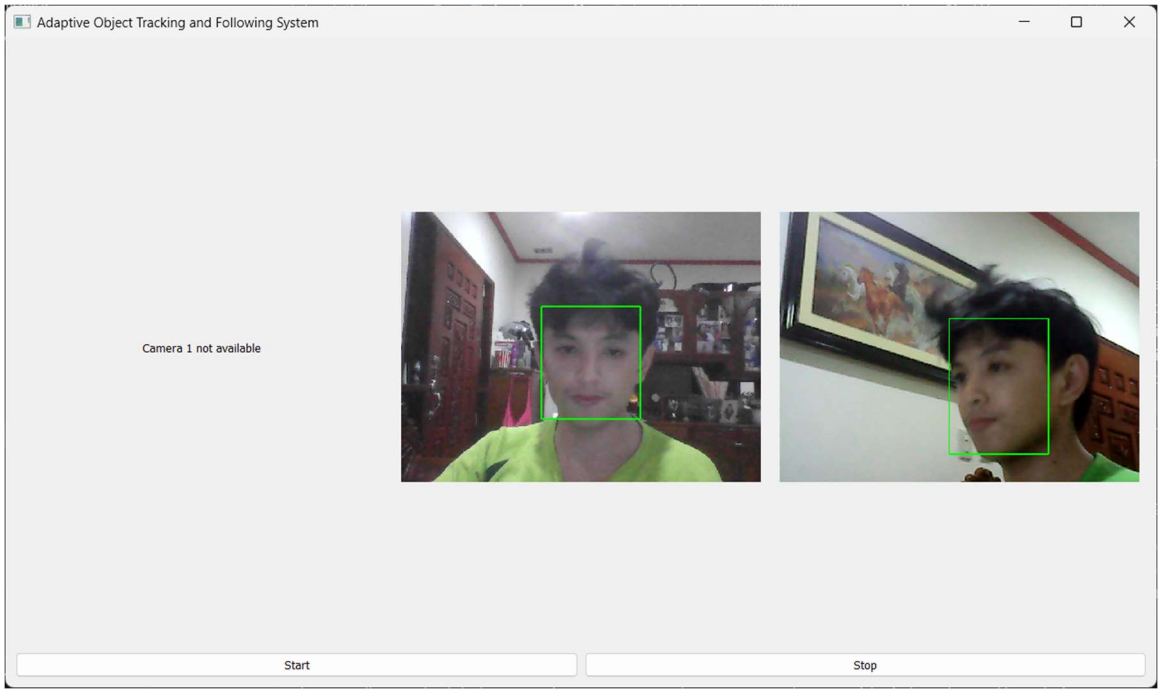


Figure 9: Iteration 5

Tasks:

- Combine YuNet and YoloV8 algorithms for enhanced inferencing capabilities.
- Test its capabilities to track better than the Mean-shift

- **Workdays**: 8
- **Time Frame**: 2 weeks
- **Review & Feedback**:

Mean-Shift algorithm replacement with YuNet algorithm happened in this iteration. Integrating it into the Graphical User Interface was part of this. Main changes included a

few things. The init method had YuNet face detector initialization added. Using YuNet for face detection and tracking meant modifying the update_frame method. Updating the tracking ROI was based on what face got detected in the selected camera. YuNet gets used by the system now to detect and track faces across all camera feeds. For updating the tracking ROI, focus stays on the selected camera. More robust tracking across different camera views and lighting conditions comes from this approach.

Review and feedback: This iteration significantly improves how well the system tracks things by incorporating the YuNet algorithm. The code keeps its overall structure while replacing Mean-Shift algorithm with a more advanced method for face detection and tracking. Using YuNet allows for better feature matching across different camera views. This addresses what the previous Mean-Shift implementation couldn't do well. More consistent tracking should result from this, especially when the subject moves between camera views

Areas for improvement and next steps could include several things. Extending the YuNet implementation to track general objects, not just faces, was one idea. Implementing a re-identification system to maintain object identity across camera views was another. User controls might be added for adjusting YuNet parameters or switching tracking modes. Multiple objects tracked at once would need the selection mechanism improved. Handling occlusions and temporary tracking loss could use a method for it

Iteration 6: Apply Optimization in the Camera inference performance of the System using Dedicated GPU

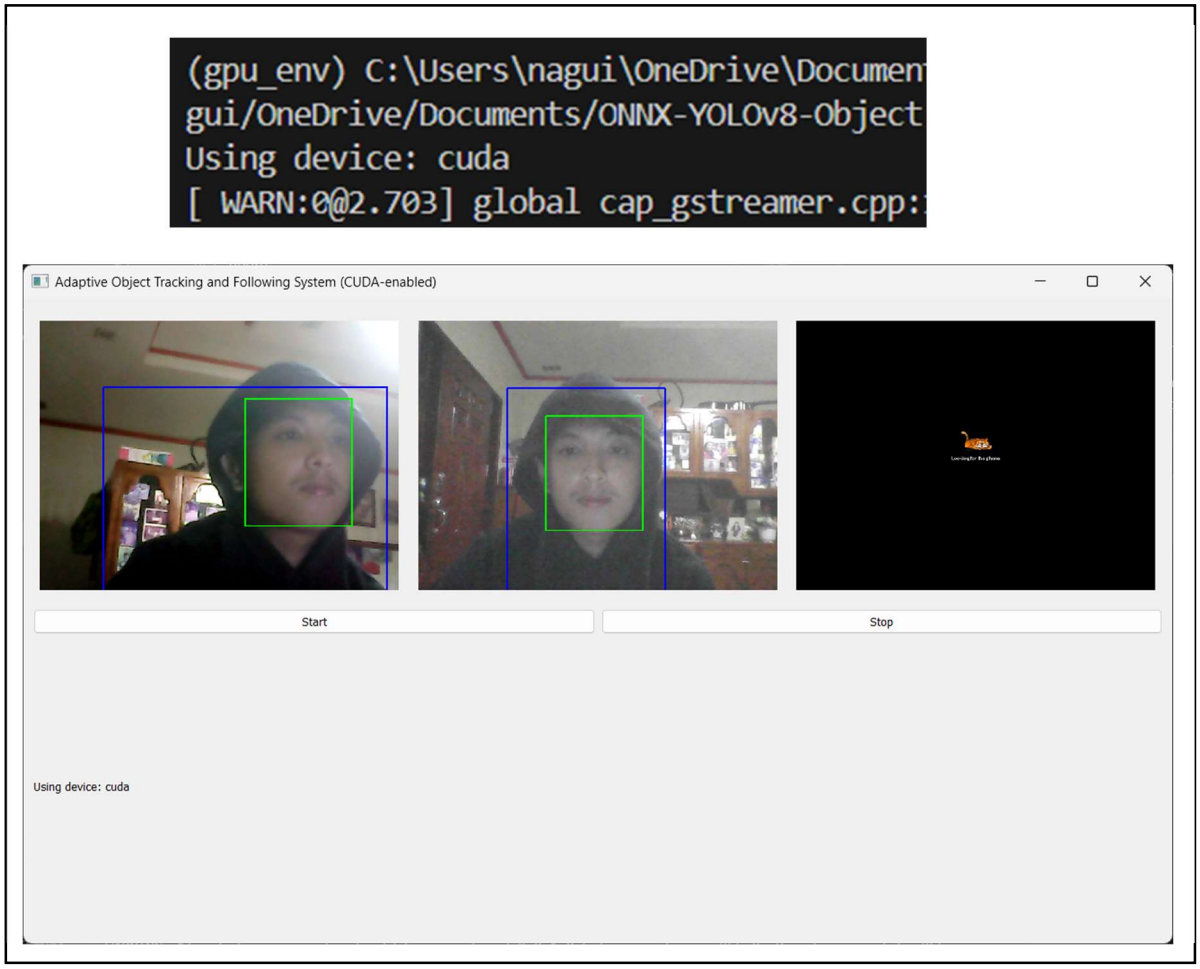


Figure 10: Iteration 6

Tasks:

- Apply Optimization in the Camera inference performance of the system

- **Workdays**: 5 days
- **Time Frame**: 1 week
- **Review & Feedback**:

This iteration integrates CUDA support for the YOLOv8 model. Yunet indicates whether the system is using CUDA. The main changes included several things. torch.device gets used to determine if CUDA is available and set the device accordingly. The YOLO and Yunet model move to the selected device, whether that's CUDA or CPU. A status label was added to display which device is being used. Frame processing with YOLO and Yunet happens on the GPU if CUDA is available. The system now leverages CUDA for faster processing when it's available on the system. This should significantly improve YOLO object detection performance, especially on systems with CUDA-capable GPUs

Review and feedback: CUDA support got successfully integrated in this iteration. The system's performance for object detection tasks might get a boost from this. While GPU acceleration capabilities were added, the code kept its structure. Adding the CUDA status label gives users clear feedback. They can see whether the system is using GPU acceleration or not.

Performance metrics for comparing CPU and GPU processing times could be added. Memory usage when working with GPU tensors might need optimization. Users can control the switching between CPU and GPU processing.

Iteration 7: Setup and Access IP CCTV Camera in the Network Router

- **Tasks**:
    - Configure the network router to access IP CCTV cameras.
    - Ensure basic connectivity and access.

- **Workdays**: 5
- **Time Frame**: 1 week
- **Review & Feedback**:

This Iteration was focused on verifying the setup and access of the IP CCTV Cameras connected via an Ethernet Cable to a network router. The team ensured that the camera were correctly integrated on the network, using RSTP Network Connection Protocol to get the camera feeds via the network.

The review process involved multiple tests to confirm that the CCTV Cameras were accessible via the network connection. The included checking if the cameras are on the correct feed from each video displayed in the system.

Iteration 8: Find the IP CCTV Camera IP Address for Live Video Feed Access via the Network Router

- **Tasks**:
    - Identify the IP addresses of CCTV cameras for live feed access.
    - Ensure correct and stable IP address configuration.
- **Workdays**: 9
- **Time Frame**: 2 weeks
- **Review & Feedback**:

In this iteration, the primary goal was to identify and configure the IP addresses of the CCTV cameras to ensure stable and reliable access to their live video feeds via the network

router. This iteration was done in two weeks, which the team dedicated nine workdays to accomplish the system changes.

The process involved testing phase where the identified IP addresses of the CCTV cameras were used to access their live video feeds. The stability and reliability of these feeds were evaluated to ensure they met the project's requirements for continuous and uninterrupted monitoring. Various scenarios, such as network congestion and potential interference, were simulated to test the robustness of the live video feeds.

Iteration 9: Access IP CCTV Cameras in the Graphical User Interface

- **Tasks**:
    - Integrate IP CCTV camera access within the GUI.
    - Ensure seamless viewing of live feeds within the GUI.
- **Workdays**: 6
- **Time Frame**: 1 week
- **Review & Feedback**:

The iteration focus shifted to integrating IP CCTV camera access within the graphical user interface (GUI). The team dedicated six workdays over the course of one week to ensure that live feeds from the cameras could be seamlessly viewed within the GUI.

The team tested various aspects of the integration, including the responsiveness of the video streams, the clarity and quality of the feeds, and the overall user experience. But

there is a bit latency or buffering issues, which is critical for effective object tracking and following.

Iteration 10: Adjust Code for Multiple Camera Inferences Using Three IP CCTV Cameras

- **Tasks**:
    - Modify the system to handle inferences from multiple cameras simultaneously.
    - Ensure system stability and performance with multiple feeds.
- **Workdays**: 7
- **Time Frame**: 1 week
- **Review & Feedback**:

The team focused on adjusting the code to support inferences from multiple IP CCTV cameras simultaneously. Over the course of one week, spanning seven workdays, the system was modified to handle inputs from three cameras at once, ensuring both stability and performance under increased load.

The team assessed various aspects, including the accuracy and speed of the inferences, the synchronization of the feeds, and the overall system performance. Emphasis was placed on ensuring that the system could handle the increased computational demands without compromising on the real-time processing essential for effective object tracking and following.

Iteration 11: Integrate Buttons in the Graphical User Interface

- **Tasks**:
    - Add functional buttons for user interactions in the GUI.
    - Ensure buttons are responsive and intuitive.
- **Workdays**: 5
- **Time Frame**: 1 week
- **Review & Feedback**:

The primary objective was to enhance the graphical user interface (GUI) by integrating functional buttons for improved user interaction. Over the course of one week, spanning five workdays, the team focused on adding these interactive elements to ensure they were both responsive and intuitive for users.

The team tested each button to ensure it performed the intended actions reliably and without delay. This included basic functionalities such as starting and stopping the camera feeds, switching between different camera views, and accessing settings or configurations relevant to the object tracking and following system. Special attention was given to the buttons' placement and design to ensure they were easily accessible and did not clutter the interface.

Iteration 12: Improve YuNet Algorithm Inference Performance Using OpenVino Execution Provider via ONNX Runtime

- **Tasks**:
    - Optimize YuNet algorithm performance using OpenVino and ONNX Runtime.
    - Enhance speed and accuracy of inferences.

- **Workdays**: 12
- **Time Frame**: 2 weeks
- **Review & Feedback**:

In Iteration 12 of the "Adaptive Object Tracking and Following using Yolov8 and Yunet algorithm" project, the focus was on enhancing the inference performance of the YuNet algorithm by utilizing the OpenVino Execution Provider through ONNX Runtime. Over a span of two weeks, encompassing twelve workdays, the team aimed to optimize the algorithm to improve both the speed and accuracy of its inferences.

The team conducted various tests to measure the enhanced inference speed and accuracy, comparing the results with the previous iteration. Key metrics such as processing time per frame, accuracy in object detection, and overall system latency were analyzed to determine the effectiveness of the optimizations.

Iteration 13: Modify the System to Work on Three Webcams

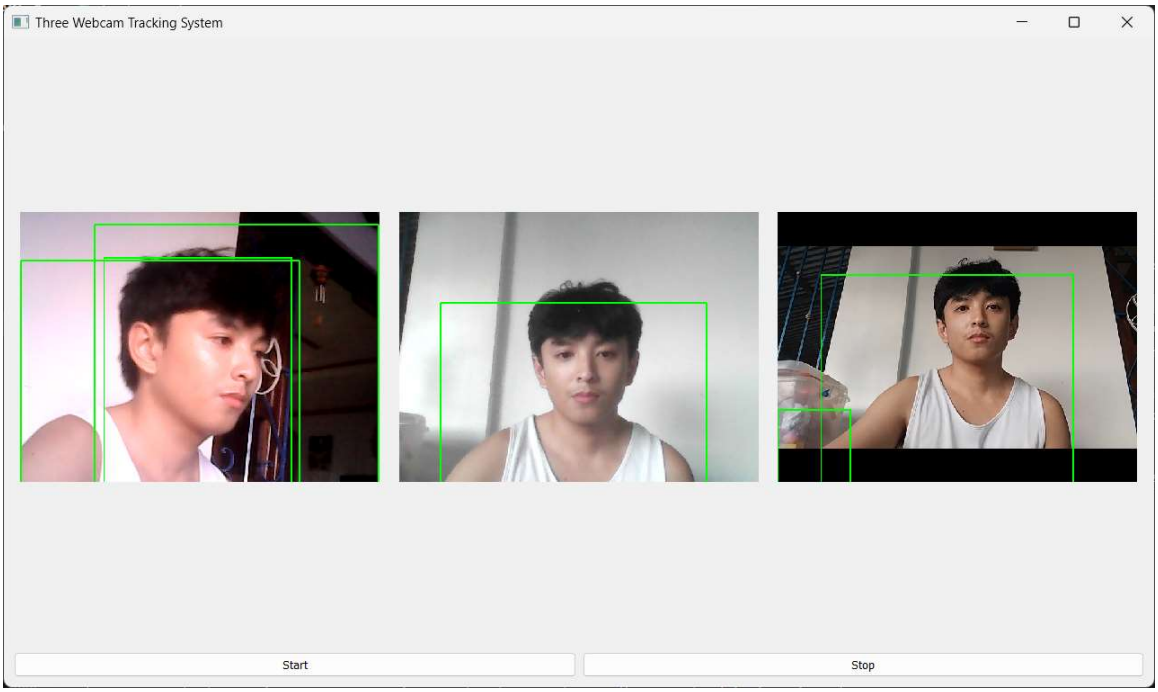


Figure 11: Iteration 13

- **Tasks**:
    - Adapt the system to support and work with three webcams.
    - Ensure smooth functionality and performance.
- **Workdays**: 5
- **Time Frame**: 1 week
- **Review & Feedback**:

This iteration modifies the system to work with three webcams. The main changes include, initializing three webcams using cv2.VideoCapture(i) for i in range(3). Updating the get_frame method to retrieve frames from the webcams. Modifying the update_frame method to process frames from all three webcams.

The system now supports three webcams simultaneously. It initializes the webcams at startup and processes frames from all three in real-time. The YOLO object detection is applied to each webcam feed, with bounding boxes drawn around detected objects.

Review and feedback: This iteration successfully adapts the system to work with three webcams, providing a multi-camera tracking solution using readily available hardware. The code maintains its structure while simplifying the camera setup process. The system now offers a consistent interface for all three webcams, making it easier to track objects across multiple views. This setup is particularly useful for applications requiring coverage of a wider area or multiple angles.

Areas for improvement and next steps could include: Implementing a method to handle webcam disconnections and reconnections. Adding camera selection options in case more than three webcams are available. Implementing frame synchronization between webcams for more accurate multi-view tracking. Adding a feature to calibrate multiple webcams for 3D position estimation. Optimizing performance to handle high-resolution webcam feeds efficiently.

Iteration 14: Re-apply Object Detection Implementation of Combined YuNet and Yolo

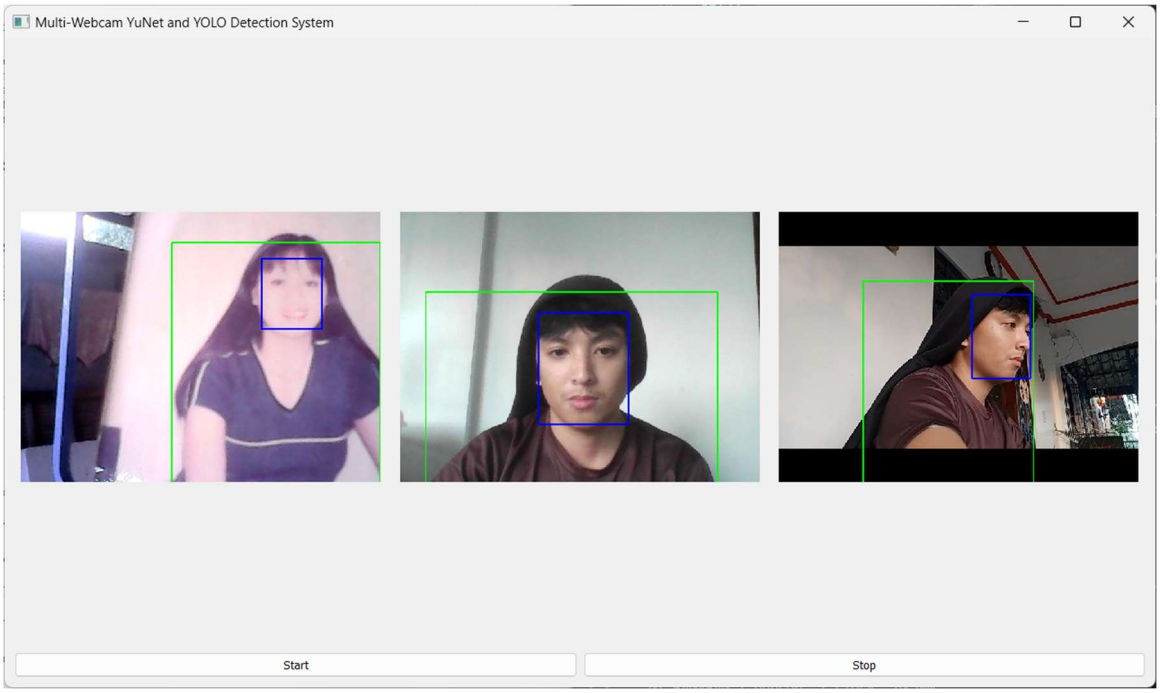


Figure 12: Iteration 14

- **Tasks**:
    - Re-implement object detection using the combined YuNet and Yolo algorithms.
    - Ensure accuracy and efficiency.
- **Workdays**: 11
- **Time Frame**: 2 weeks
- **Review & Feedback**:

This iteration re-applies the object detection implementation, combining YuNet for face detection and YOLO for general object detection. The main changes include initializing both YuNet and YOLO detectors in the __init__ method. Updating the

update_frame method to perform both face detection using YuNet and object detection using YOLO on each frame. Drawing bounding boxes for both face and object detections on the frame.

The system now processes frames from all three webcams in real-time, applying both face detection (YuNet) and general object detection (YOLO) to each frame. Face detections are drawn with blue bounding boxes, while YOLO object detections are drawn with green bounding boxes.

Review and feedback: This iteration successfully combines two powerful detection algorithms, providing a comprehensive detection system for both faces and general objects. The code maintains its structure while integrating these two detection methods seamlessly.

The combination of YuNet and YOLO offers a more versatile detection system, capable of identifying both specific facial features and a wide range of objects. This makes the system suitable for various applications, from security and surveillance to human-computer interaction and object tracking.

Areas for improvement and next steps could include: Implementing object tracking to maintain consistent identities across frames. Adding classification labels for detected objects. Implementing a method to switch between detection modes or adjust detection parameters on-the-fly. Optimizing performance to handle high-resolution feeds and multiple detectors efficiently. Adding a feature to save detection results or generate reports.

Iteration 15: Implement Re-identification System

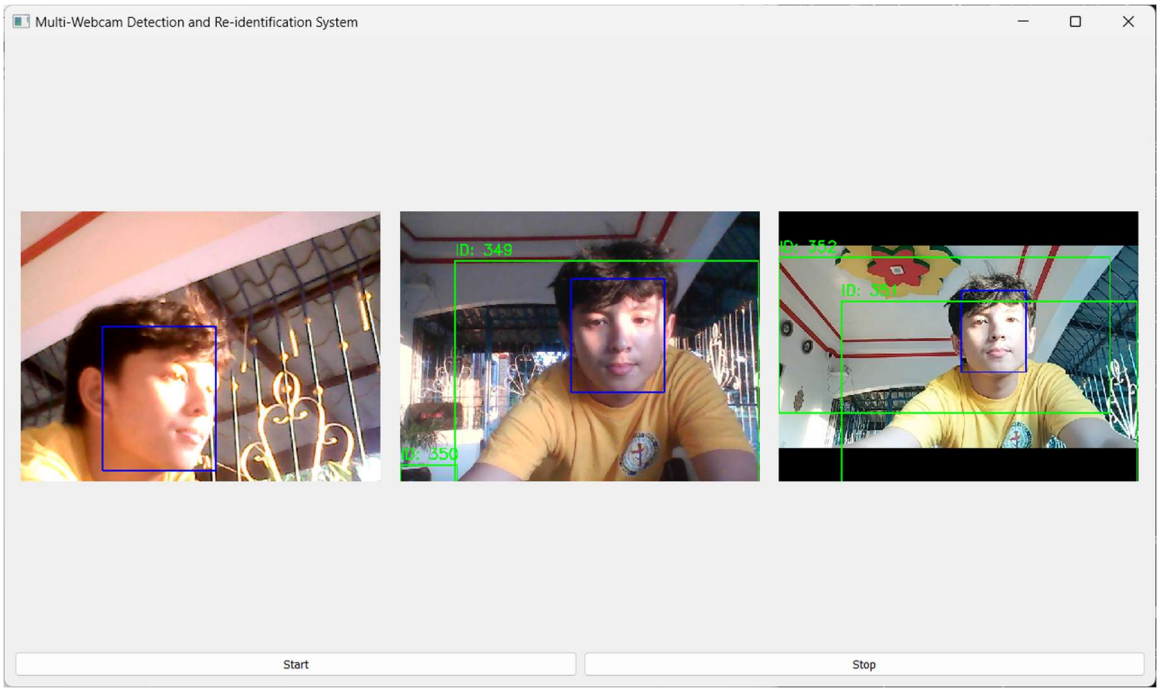


Figure 13: Iteration 15

- **Tasks**:
    - Develop a re-identification system for tracking objects across frames.
    - Ensure accurate and reliable re-identification.
- **Workdays**: 12
- **Time Frame**: 2 weeks
- **Review & Feedback**:

This iteration implements a re-identification system on top of the existing YuNet and YOLO detection framework. The main changes include adding a ReIDModel class that uses a pre-trained ResNet50 model for feature extraction. Implementing a match_person

method to compare extracted features with a database of known persons. Updating the update_frame method to extract features from detected persons and assign unique IDs.

The system now not only detects faces and objects but also attempts to re-identify persons across frames and cameras. It extracts features from detected persons, compares them with a database of known persons, and assigns a unique ID to each person. This ID is then displayed alongside the bounding box for each detected person.

Review and feedback: This iteration significantly enhances the system's capabilities by adding person re-identification. The code maintains its structure while integrating this complex feature seamlessly. The use of a feature extraction provides a robust foundation for person re-identification.

The re-identification system allows for tracking individuals across different camera views and frames, which is crucial for many surveillance and monitoring applications. The simple matching algorithm based on feature distance provides a good starting point for person re-identification. Areas for improvement and next steps could include: Implementing a more sophisticated matching algorithm, possibly using a metric learning approach. Adding a temporal component to the re-identification system to improve consistency over time. Implementing a method to handle occlusions and partial detections. Adding a feature to manually correct or update person identities. Optimizing the re-identification process for real-time performance, possibly by running it on a separate thread.

Iteration 16: Integrate Database and CRUD Operations

```python
class Database:
    def __init__(self):
        self.conn = sqlite3.connect('person_database.db')
        self.cursor = self.conn.cursor()
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS persons
                               (id INTEGER PRIMARY KEY, features BLOB)''')
        self.conn.commit()

    def add_person(self, features):
        self.cursor.execute("INSERT INTO persons (features) VALUES (?)", (features.tobytes(),))
        self.conn.commit()
        return self.cursor.lastrowid

    def get_all_persons(self):
        self.cursor.execute("SELECT * FROM persons")
        return [(row[0], np.frombuffer(row[1], dtype=np.float32)) for row in self.cursor.fetchall()]

    def update_person(self, person_id, features):
        self.cursor.execute("UPDATE persons SET features = ? WHERE id = ?", (features.tobytes(), person_id))
        self.conn.commit()

    def delete_person(self, person_id):
        self.cursor.execute("DELETE FROM persons WHERE id = ?", (person_id,))
        self.conn.commit()
```

Figure 14: Iteration 16

- **Tasks**:
    - Set up a database and implement Create, Read, Update, Delete (CRUD) operations.
    - Ensure data persistence and management.
- **Workdays**: 6
- **Time Frame**: 1 week
- **Review & Feedback**:

This iteration integrates a SQLite database with CRUD (Create, Read, Update, Delete) operations for person re-identification. The main changes include adding a Database class that handles all database operations using SQLite. Modifying the match_person method to use the database for storing and retrieving person features.Updating the person's features

in the database when a match is found, allowing the system to adapt to slight changes in appearance over time.

The system now stores person features in a persistent SQLite database, allowing for long-term tracking and re-identification across multiple sessions. The CRUD operations (Create new person, Read all persons, Update person features, Delete person) are implemented in the Database class.

Review and feedback: This iteration significantly enhances the system's capabilities by adding persistent storage for person re-identification. The code maintains its structure while integrating database operations seamlessly. The use of SQLite provides a lightweight yet powerful solution for storing and retrieving person features. The database integration allows the system to maintain person identities across multiple sessions, which is crucial for long-term surveillance and monitoring applications. The update operation allows the system to adapt to gradual changes in a person's appearance over time.

Areas for improvement and next steps could include implementing a user interface for manual database management (e.g., adding, editing, or deleting persons). Adding more metadata to the database, such as timestamps, camera IDs, or additional person information. Implementing a more sophisticated matching algorithm that considers multiple recent feature vectors for each person. Adding support for exporting and importing the database for backup or system migration. Implementing a cleanup mechanism to remove old or infrequently seen persons from the database.

Iteration 17: Camera View Implementation and Control Panel Creation

- **Tasks**:
    - Develop a control panel and implement multiple camera views in the GUI.
    - Ensure user-friendly interface and controls.
- **Workdays**: 5
- **Time Frame**: 1 week
- **Review & Feedback**:

This iteration implements separate camera views and creates a control panel. The main changes include creating a CameraView class to encapsulate the display and information for each camera. Implementing a ControlPanel class with start/stop buttons and a detection threshold slider. Reorganizing the main window layout to include multiple camera views and the control panel. Updating the frame processing to work with individual camera views and adjustable detection threshold.

The system now displays three separate camera views, each with its own frame and detection information. The control panel allows users to start/stop processing and adjust the detection threshold in real-time.

Review and feedback: This iteration significantly improves the user interface and control capabilities of the system. The code is well-structured, with clear separation of concerns between the main window, camera views, and control panel. The addition of individual camera views provides better visibility and organization of the multi-camera

setup. The control panel enhances user interaction, allowing for real-time adjustments to the system's behavior.

Areas for improvement and next steps could include adding more advanced controls, such as camera selection or detection model switching. Implementing a more detailed information display for each camera view, including detected object classes. Adding the ability to save and load configuration settings. Implementing a feature to capture and save snapshots or video clips from individual cameras. Adding a log or history panel to track events and detections over time.

Iteration 18: Implement Multi-Tracking Mode

- **Tasks**:
    - Develop a mode for tracking multiple objects simultaneously.
    - Ensure accurate and efficient multi-tracking.
- **Workdays**: 6
- **Time Frame**: 1 week
- **Review & Feedback**:

This iteration implements a Multi-Tracking Mode, enhancing the system's ability to track multiple objects simultaneously across all camera feeds. The main changes include adding a Tracker class that maintains the state of multiple tracks. Modifying the update_frame method to use the tracker for each camera. Updating the visualization to show tracked objects with their IDs.

The system now tracks multiple objects across frames, maintaining their identities and positions. Each camera has its own tracker, allowing for independent tracking across different views. The tracker uses a simple age-based mechanism to remove stale tracks.

Review and feedback: This iteration significantly improves the system's tracking capabilities by implementing multi-object tracking. The code maintains its structure while adding this complex feature seamlessly. The Tracker class provides a flexible foundation for more advanced tracking algorithms in the future. The multi-tracking mode allows the system to follow multiple subjects simultaneously, which is crucial for comprehensive surveillance and monitoring applications. The age-based track management helps in handling occlusions and temporary disappearances.

Areas for improvement and next steps could include add a more sophisticated tracking algorithm. Adding track prediction to handle short-term occlusions better. Implementing cross-camera tracking to maintain identities across different views.

Iteration 19: Implement Single Tracking Mode

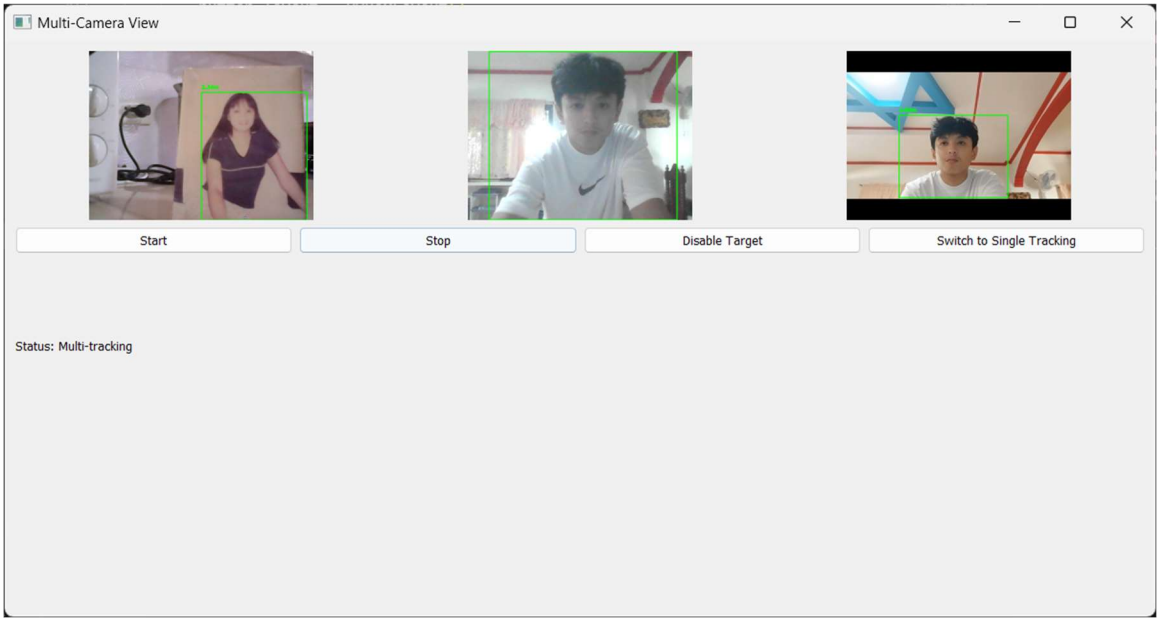


Figure 15: Iteration 19

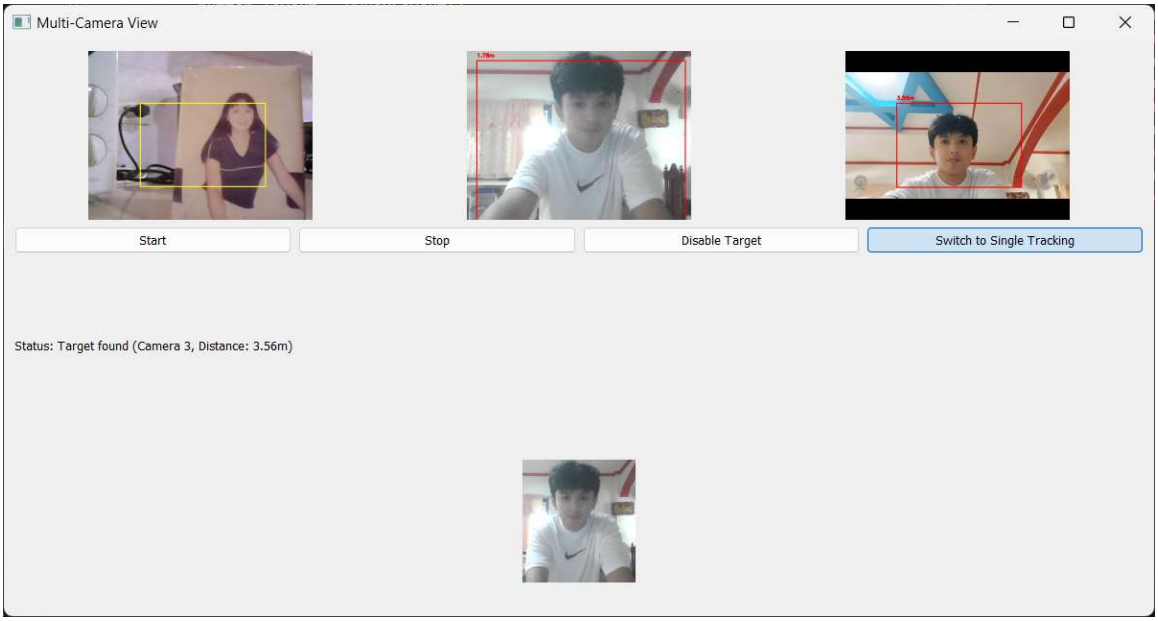


Figure 16: Iteration 19.2

- **Tasks**:
    - Develop a mode for tracking a single object.
    - Apply Target Selection Mechanism
    - Implement Feature Selection and Matching to the Selected Target
- **Workdays**: 28 days
- **Time Frame**: 4 weeks
- **Review & Feedback**:

This iteration implements Single Tracking Mode with Target Selection and Feature Matching. The main changes include adding a select_target method that allows users to click on a camera feed to select a target. Implementing feature extraction for the selected target using the ReID model.

Modifying the update_frame method to match the selected target across all camera feeds using feature similarity. The system now allows users to select a single target by clicking on one of the camera feeds. It then extracts features from the selected target and uses these features to track the target across all camera feeds. The tracking is based on feature similarity, allowing the system to maintain the target's identity even if it moves between different camera views.

Review and feedback: This iteration successfully implements a Single Tracking Mode with intuitive target selection and robust feature matching. The code maintains its structure while adding these new capabilities seamlessly. The use of feature similarity for tracking provides a more reliable method for maintaining the target's identity across different views

and poses. The target selection mechanism is user-friendly, automatically selecting the object closest to the clicked point. The feature matching approach allows for tracking that is more robust to changes in appearance and lighting compared to simple bounding box overlap methods.

Areas for improvement and next steps could include implementing a visual indicator for the currently selected target. Adding a mechanism to clear or change the current target. Implementing a confidence threshold for target matching to avoid false positives. Adding trajectory prediction to handle short-term occlusions. Implementing a way to save and load target features for long-term tracking across sessions.

Iteration 20: Add Save and Load Target Functionality

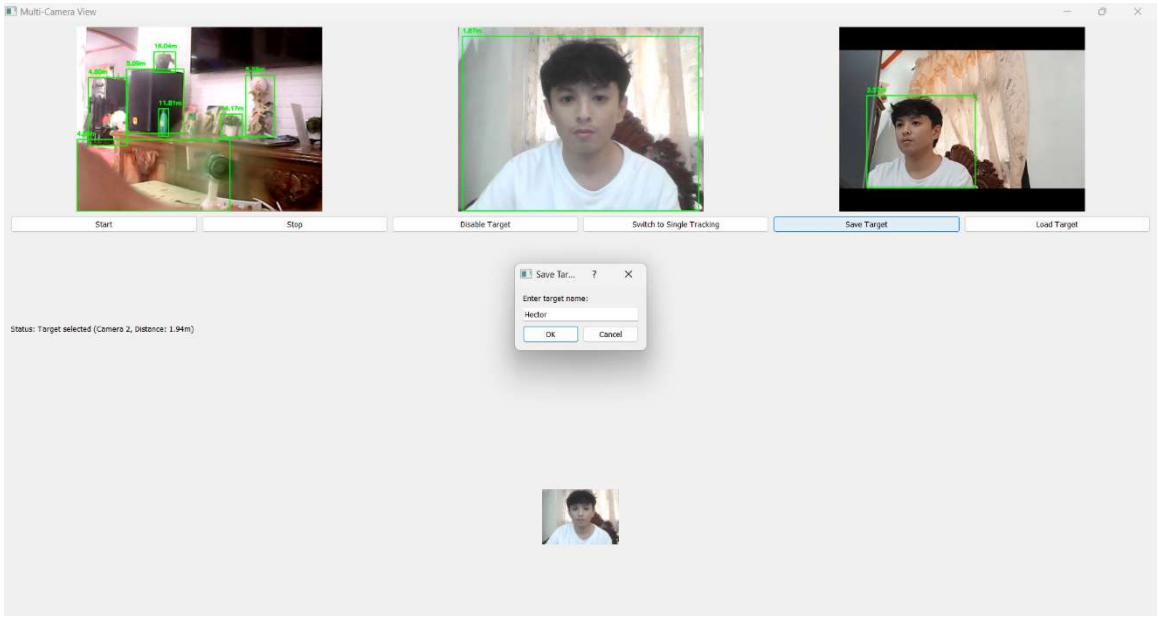


Figure 17: Iteration 20

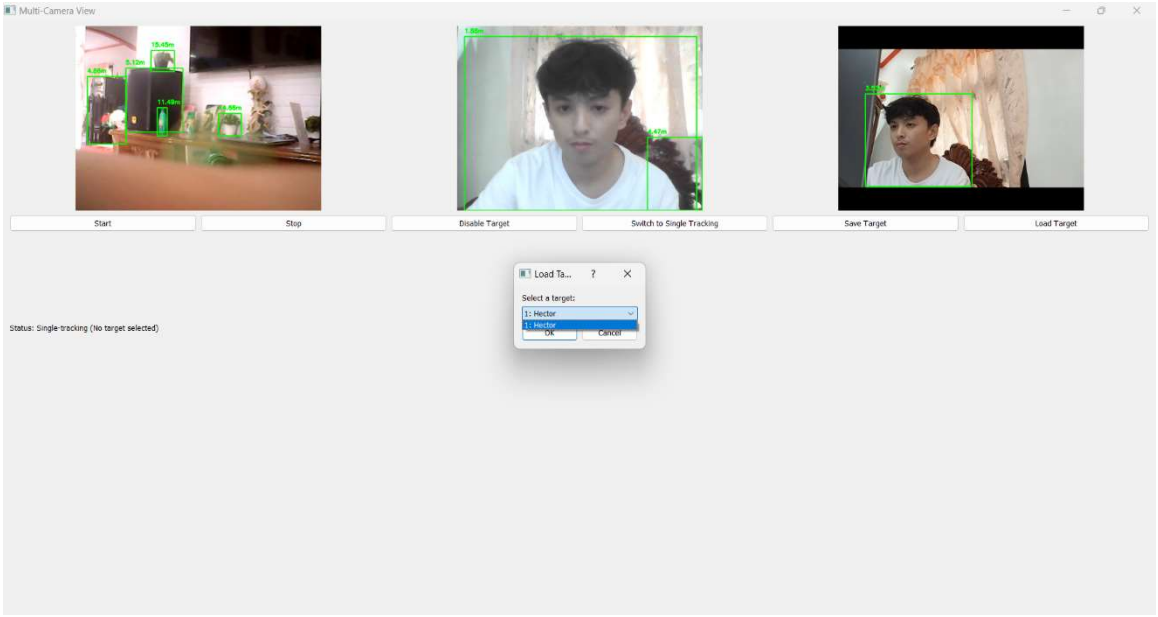


Figure 18: Iteration 20.2

- **Tasks**:
    - Implement functionality to save and load tracking targets.
    - Ensure data persistence and user convenience.
- **Workdays**: 6
- **Time Frame**: 1 week
- **Review & Feedback**:

This iteration adds Save and Load Target Functionality to the Single Tracking Mode. The main changes include adding "Save Target" and "Load Target" buttons to the user interface. Implementing save_target method to save the current target features to a file. Implementing load_target method to load previously saved target features from a file.

Using Python's pickle module for serializing and deserializing target features. The system now allows users to save the features of a selected target to a file and load them

later. This functionality enables long-term tracking across different sessions and provides a way to quickly switch between different targets of interest.

Review and feedback: This iteration successfully implements the Save and Load Target Functionality, enhancing the system's flexibility and usability. The code maintains its structure while adding these new features seamlessly. The use of pickle for serialization provides a simple and effective way to store and retrieve complex numpy arrays of target features. The save and load functionality adds significant value to the tracking system, allowing for persistent tracking across sessions and easy sharing of targets between different instances of the application. This feature is particularly useful for long-term surveillance or when tracking specific individuals of interest over extended periods.

Areas for improvement and next steps could include implementing a database to store multiple targets instead of individual files. Adding metadata to saved targets (e.g., timestamp, camera ID, description). Implementing a target management system to switch between multiple loaded targets. Adding error handling for file operations and corrupt data. Implementing a feature to update saved target features with new observations.

Iteration 21: Implement Upload Image tracking

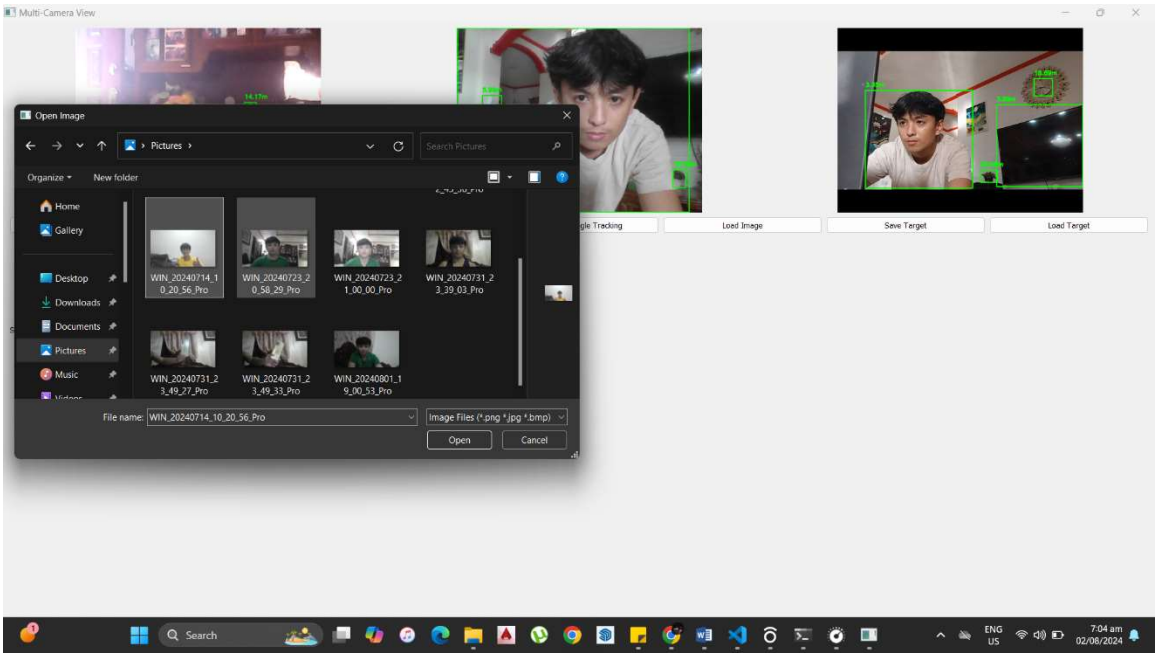


Figure 19: Iteration 21

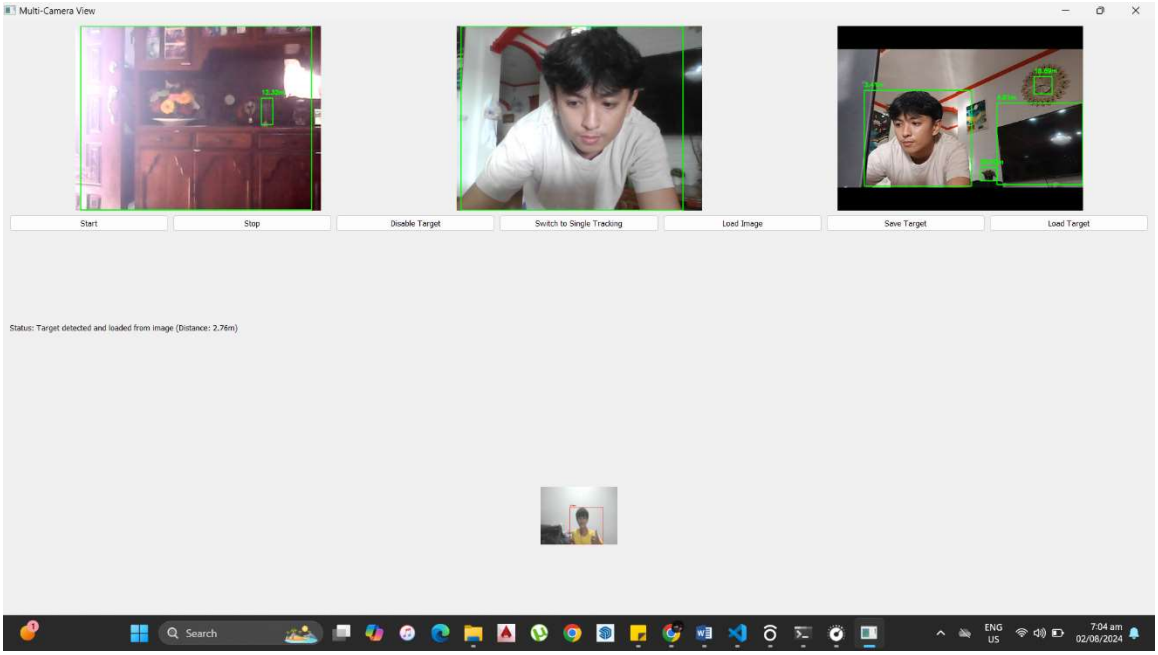


Figure 20: Iteration 21.2

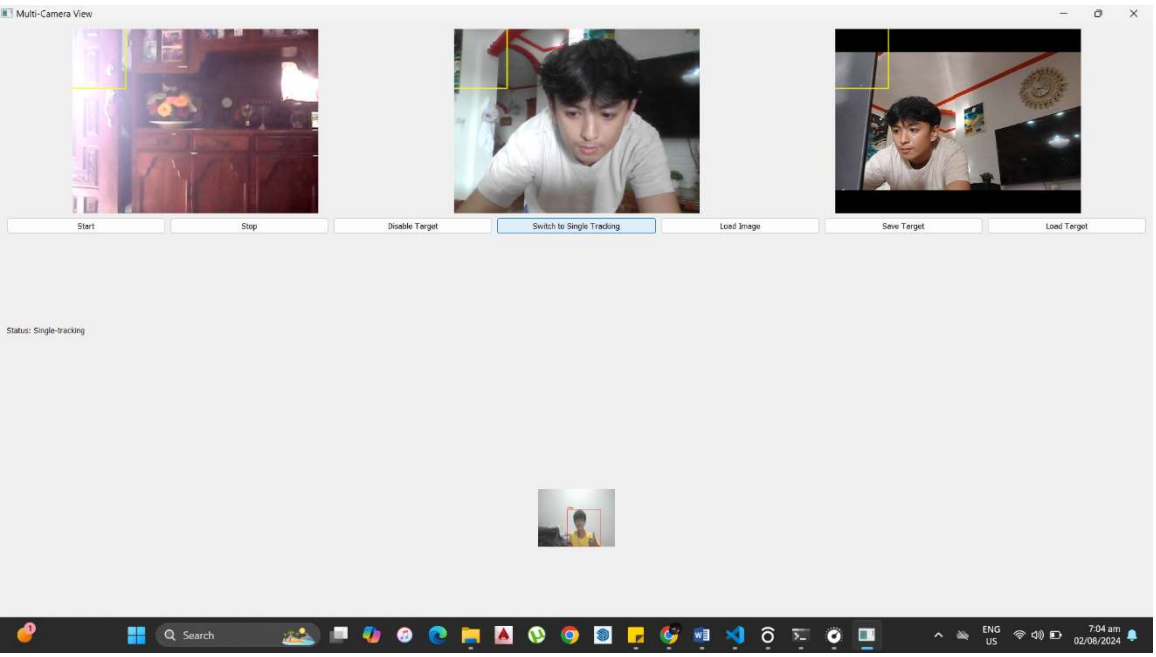


Figure 21: 21.3

Tasks:

- Implement tracking with upload image with diff size.
- Track the detected object or person in the image

- **Workdays**: 6
- **Time Frame**: 1 week
- **Review & Feedback**:

This iteration implements Upload Image tracking functionality. The main changes include adding an "Upload Image" button to the user interface. Implementing the upload_image method to handle image upload and target selection from the uploaded image. Modifying the YOLO detection process to work with uploaded images of different sizes.

The system now allows users to upload an image, automatically detects the first object or person in the image using YOLO, and sets it as the tracking target. This feature enables tracking based on a pre-selected image, which can be particularly useful when the target is not immediately visible in the live camera feeds.

Review and feedback: This iteration successfully implements the Upload Image tracking functionality, enhancing the system's flexibility in target selection. The code maintains its structure while integrating this new feature seamlessly. The use of YOLO for object detection in the uploaded image ensures compatibility with the existing tracking system. The upload image functionality adds significant value to the tracking system, allowing users to initiate tracking based on a pre-existing image. This feature is particularly useful in scenarios where the target may not be immediately present in the live feeds, or when working with historical data.

Areas for improvement and next steps could include implementing multi-object detection in uploaded images and allowing the user to select which detected object to track. Adding support for video file uploads in addition to static images. Implementing a feature to crop and adjust the uploaded image before target selection. Adding error handling for various image formats and sizes. Implementing a gallery view of uploaded images and detected targets for easy management.

Table 9: Gantt Chart

Tasks	January				February				March				April				May				June				July			
	Week				Week				Week				Week				Week				Week				Week			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
A																												
B																												
C																												
D																												
E																												
F																												
G																												
H																												
I																												
J																												
K																												
L																												
M																												
N																												
O																												
P																												
Q																												
R																												
S																												
T																												
U																												
V																												

A. Initialize Camera in the Graphical User Interface (GUI).

B. Integrate the YoloV8 algorithm by Ultralytics into the System.

C. Integrate the Mean-shift Algorithm in the System.

D. Implementing Single Tracking with mouse Selection.

E. Replace the Mean-Shift Algorithm and Integrate the YuNet Algorithm to the Graphical User Interface.

F. Apply Optimization in the Camera inference performance of the System using Dedicated GPU.

G. Setup and access IP CCTV camera in the Network Router.

H. Find the IP CCTV Camera IP address for live video feed access via the Network Router.

I. Access IP CCTV Cameras in the Graphical User Interface.

J. Adjust Code for Multiple Camera Inferences using the three IP CCTV Cameras.

K. Integrate Buttons in the Graphical User Interface

L. Improve YuNet Algorithm Inference Performance using OpenVINO Execution Provider via ONNX Runtime.

M. Modify the system to work on three webcams.

N. Re-apply object detection implementation of combined YuNet and Yolo.

O. Implement re-identification system.

P. Integrate database and crude operations.

Q. Modify main window design.

R. Camera view implementation and control panel creation.

S. Implement Multi-tracking mode

T. Implement single tracking mode

U. Add save and load target functionality

V. Implement Upload Image tracking

Shown in table 9, the Gantt chart for the system depicted how various tasks were completed over a set period. Initially, the team focused on initializing the Camera Graphical User Interface, and analyzing the camera in the graphical user interface. These

foundational tasks were followed by replacing the Mean-shift Algorithm with the YuNet Fast R-CNN Algorithm and then integrating the YuNet Algorithm into the Graphical User Interface.

The team dedicated time for debugging the mean-shift algorithm tracking issues. Since mean shift could not work well and did not support facial detection, it was replaced with the Yunet Fast R-CNN algorithm. The team started integrating the Yunet and then integrated the Yolov8 by Ultralytics into the system. Optimization was applied in the Camera inference performance of Yolov8 using a Dedicated GPU. The team then merged the Yunet and Yolov8 algorithms in the system for camera inferencing on both algorithms, set up and accessed the IP CCTV Camera in the Network Router, and found the IP Camera IP address for the live video feed access via the network router. The IP CCTV Camera was accessed in the Graphical User Interface. The team adjusted the code for multiple camera inferencing using the three IP CCTV Cameras, integrated buttons, improved Yunet inference performance using OpenVino Execution Provider via ONNX Runtime.

The Gantt chart allowed the team track the progress of each task of the project. The team implemented a re-identification system that allows the algorithm to recognize and track objects and faces over time when the tracked subject was lost in visual sight of the camera. The system also added database connection, which stores the captured images from the tracked object or face into a database, allowing users to go back and find previous targets that was lost. Uploading of images was also implemented to track objects and faces from captured images on the computer's storage.

The system implemented has both multi-tracking and single-tracking modes, having the flexibility to track multiple objects and focus on a single tracking. The target selection feature allowed the users to select a specific object or face and track them across three cameras.

Table 10: SDLC Development Team

DEVELOPMENT TEAM	
NAME	**ROLES**
Brion, Jhon Brix G.	Programmer II
Lansangan, Cairo T.	System Analyst
Luanzon, Charls Adrian M.	Technical Writer
Sanchez, Hector G.	System Architect Programmer I

B. Diagrams

B.1. Relational Schema Diagram

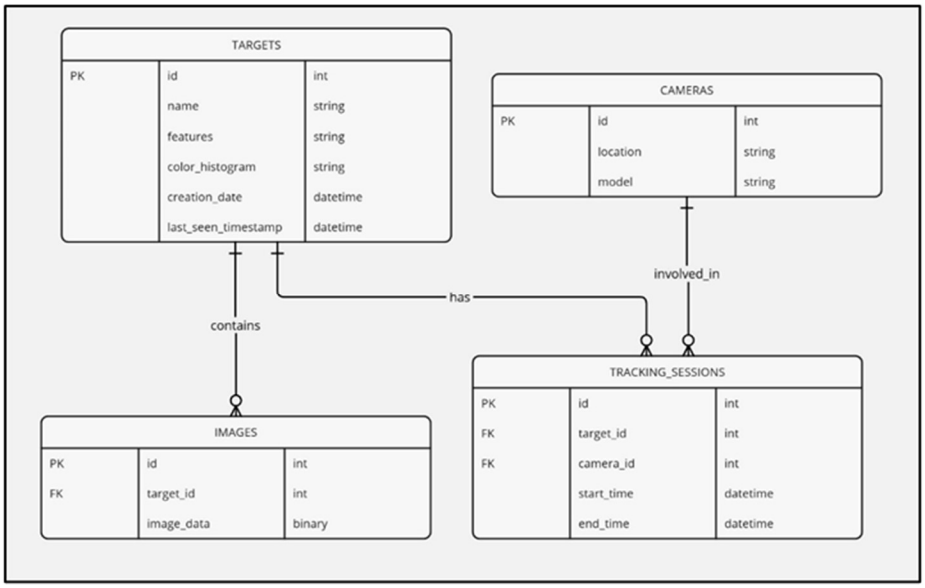


Figure 22: Relational Schema Schema

Shown in figure 22, the Relational Schema Design for this program consists of four tables, which stores the targets, images, cameras, and tracking_sessions. The target table is a central entity, storing the tracked objects including their unique identified, name, feature vector, color histogram, creation date, and last seen timestamp. The targets table is linked with the images table, which stores images of each target.

The Cameras table keeps the details about the camera, while the tracking_sessions table acted as a junction between targets and cameras, recording tracking events from start and end times. This table is for allows tracking of the history of targets across different cameras. This schema design efficiently supported the program's multi-camera object tracking and re-identification capabilities, enabling seamless storage and retrieval of target information, associated imagery, camera details, and detailed tracking session data.

B.2. Black Box Testing

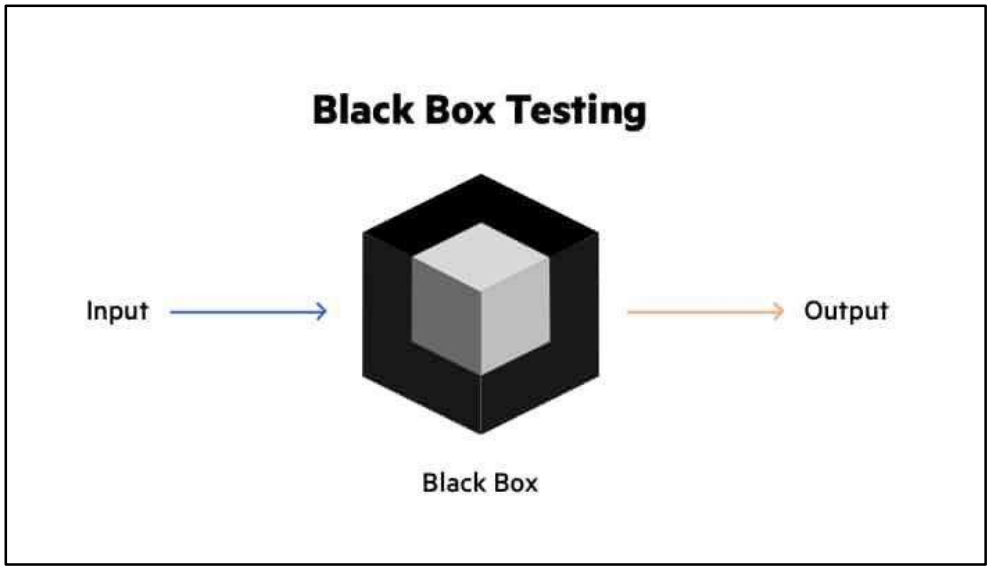


Figure 23: Black Box Testing

Shown in figure 23, Blackbox testing, a software testing technique in which a system's function is evaluated from the user's perspective, with no knowledge of its internal workings . It was like testing a locked box from the outside, using various inputs and observing the outputs, without knowing how the internal mechanisms (the code, algorithms, etc.) actually worked. This approach contrasted with white box testing, which delved into the internal structure and code of the system [32].

Black box testing played a crucial role in enhancing adaptive object tracking and following systems for several reasons. This real-world scenario simulation presented challenges for the flexibility of the system, exposing flaws and prejudices that could be fixed to enhance the performance. Early identification of regressions and defects that may result from system upgrades or modifications was made easier by black box testing. Black box testing provided an essential safety net by spotting these problems before they affected people. Furthermore, providing a satisfying user experience in applications like robots,

driverless cars, and augmented reality required reliable and accurate object tracking. Black box testing increased user confidence in the system's capabilities by confirming its dependability in various scenarios. All things considered, adaptive object tracking and following systems' usefulness and dependability were greatly improved via black box testing.

1. Increased Robustness and Accuracy

Black box testing simulates real-world usage by introducing a variety of inputs (things, surroundings, and lighting situations) that evaluate the system's flexibility. This helps to detect flaws and biases, resulting in increased robustness and accuracy.

2. Early Detection of Regression and Bugs

System updates or changes may introduce regressions (unexpected performance decreases) or new issues. Black box testing provides a safety net by detecting these vulnerabilities before they reach users.

3. Improved User Experience and Trust

Reliable and precise object tracking is crucial for an optimal user experience in applications such as robotics, autonomous cars, and augmented reality. Dependable and accurate object tracking is essential for an excellent user experience in applications including robotics, autonomous vehicles, and augmented reality.

B.3. Load Testing

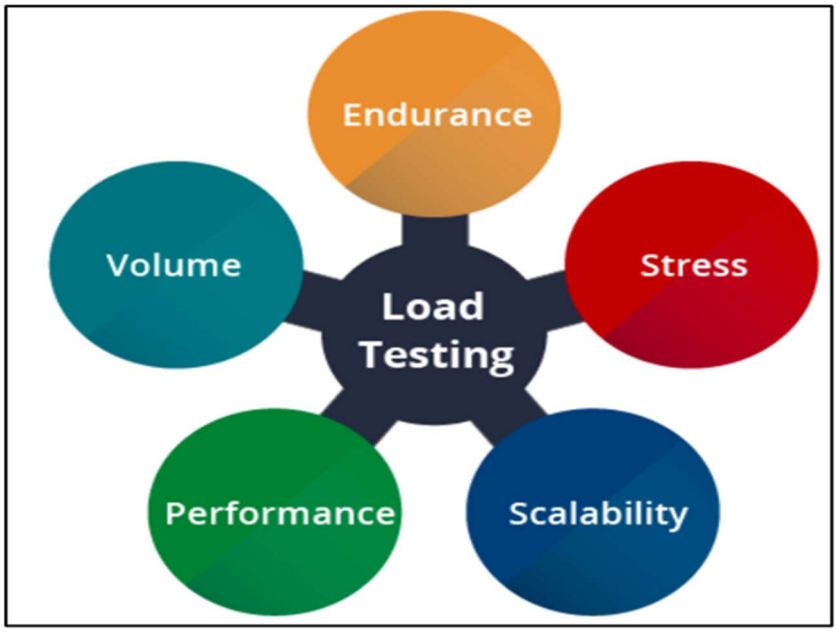


Figure 24: Load Testing

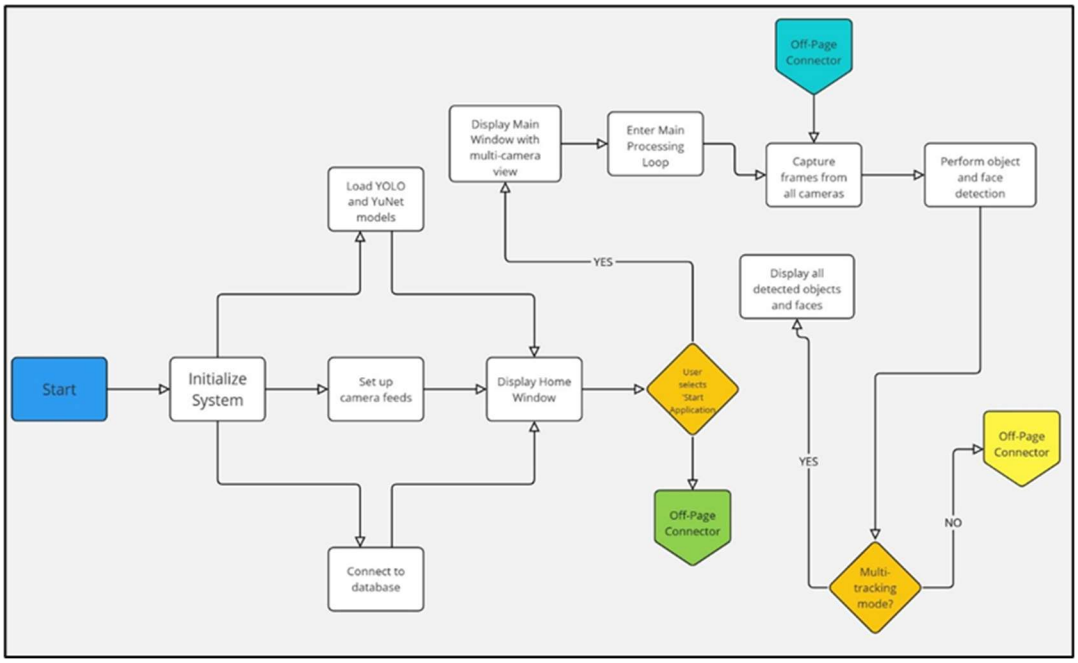
As shown in Figure 24, load testing is a software testing method used to evaluate how a system performs when subjected to simulated real-world workload conditions. This technique is important in developing an adaptive object tracking and following system because it helps ensure the system remains stable and responsive when handling multiple processes or requests simultaneously. During load testing, controlled and gradually increasing levels of stress are applied to the system, usually through concurrent users or requests, while monitoring its response time, stability, and resource consumption. [42]

Load testing contributed significantly to the development and deployment of the adaptive object tracking and following system. First, it ensured that the system maintained reliable real-time performance and accurate tracking even when operating under varying workloads. Second, it helped optimize the use of system resources by identifying inefficiencies in the algorithms or code that could cause slow performance or system

failures. Finally, load testing assessed the scalability of the system by determining whether it could handle more tracked objects or operate effectively in more complex environments. Overall, this testing approach improved the system's performance, efficiency, adaptability, and scalability, supporting its reliable use in real-world applications.

C. Algorithms

C.1. System's Flowchart



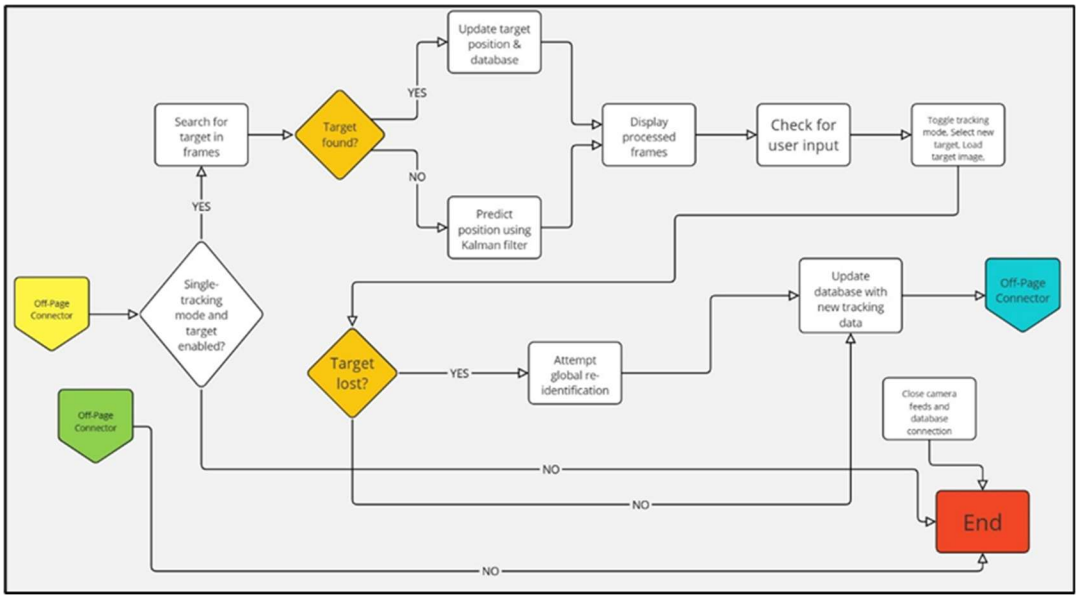


Figure 25: Adaptable Object Tracking and Following System using YOLOv8 and Yunet Algorithms

Shown in figure 25, the flowchart begins in initializing the system. It includes loading the YOLOv8 and YuNet Models, setting up USB Camera, and connecting the system into the database. The home window will be displayed and the user can start

selecting "Start Application" to start the system. This opened the main window with a multi-camera view and entered the main processing loop. In this loop, frames were captured from all cameras, and object and face detection were performed. If multi-tracking mode was enabled, all detected objects and faces were displayed. If single-tracking mode was enabled and a target was specified, the system searched for the target in the frames. If found, the target's position was updated in the database; if not, the Kalman filter predicted the target's position. Processed frames were displayed, and user input was checked for actions like toggling tracking mode, selecting a new target, loading or saving target images, and updating the database. If the target was lost, the system attempted global re-identification. The loop continued, updating the database with new tracking data until the program was closed. Finally, camera feeds and the database connection were closed, and the system ended.

C.2. Yunet Algorithm's Flowchart

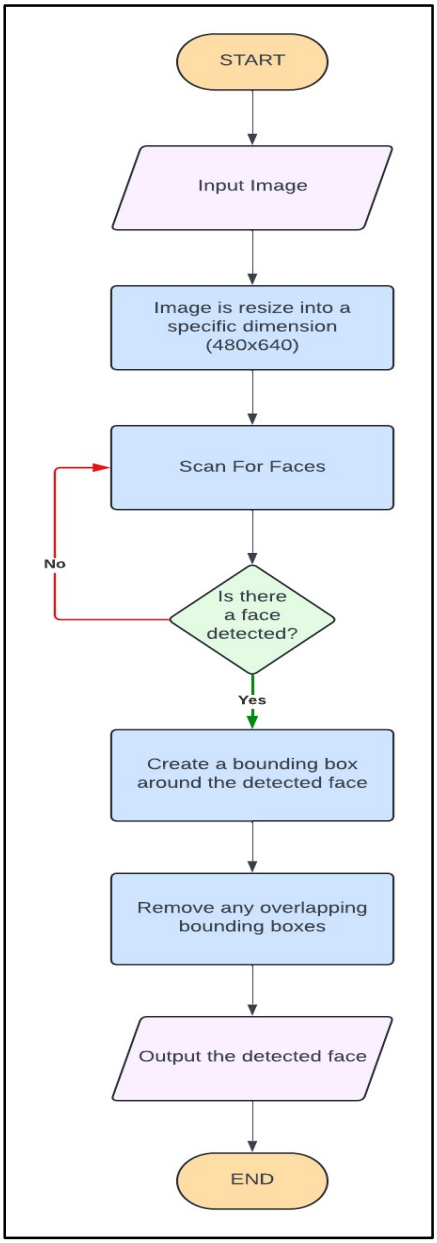


Figure 26: Yunet Algorithm's Flowchart

The YuNet architecture focused on detecting small faces while lowering computational costs. It utilized Depth-wise Separable Convolutions and then adopted a multi-stage approach with the purpose of extracting and processing features at different scales to achieve this. It comprised the 'Head,' the Tiny Feature Pyramid Network (TFPN)

'Neck,' and the core feature extraction part of YuNet: 'Backbone,' all of which were based on Depthwise Separable Convolution. 'ConvHead, 16' indicated the ConvHead module with 16 output channels, whereas all the head outputs were concatenated to produce detection results via non-maximum suppression (NMS).

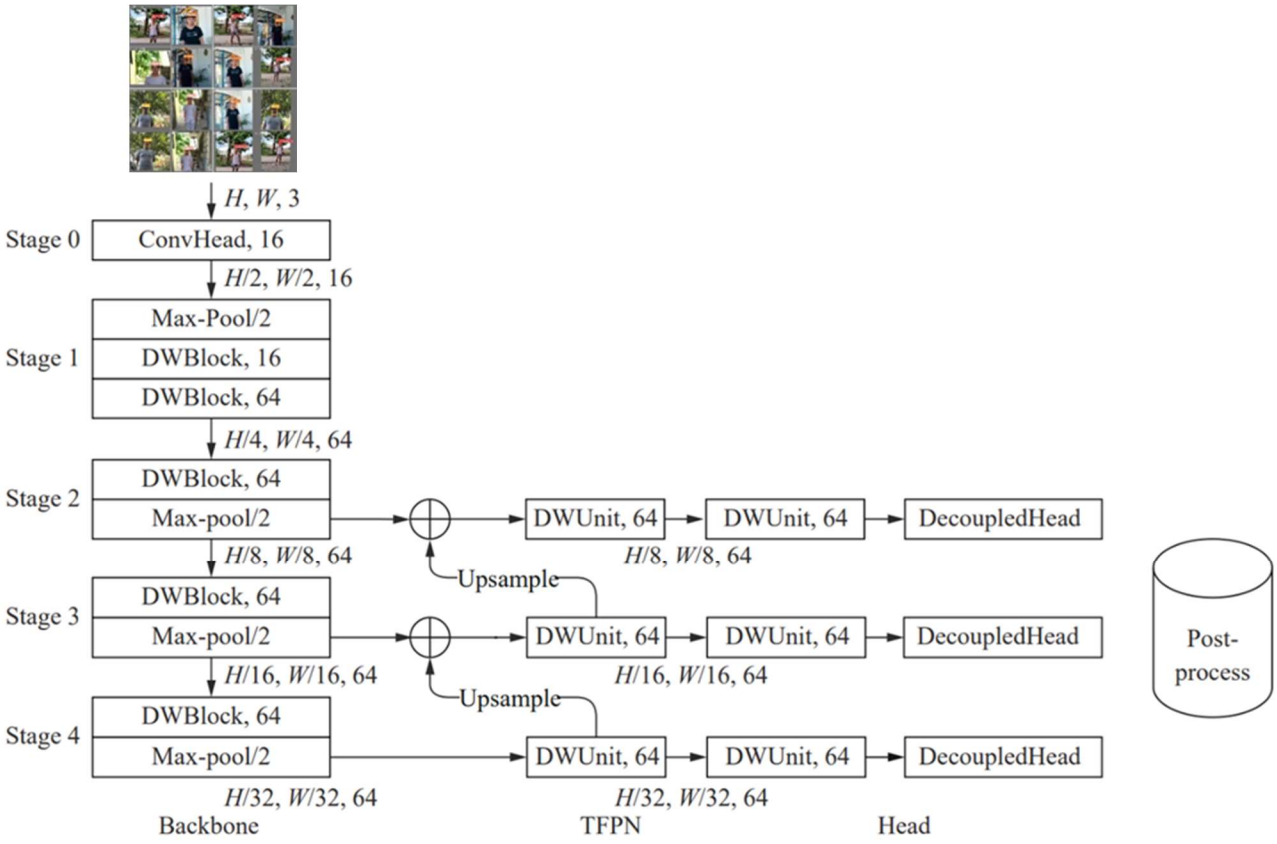


Figure 27: YuNet Architecture

Shown in figure 27, the YuNet Architecture was focused on detecting facial features with low computational costss. It utilized Depth-wise Separable Convolutions and then adopted a multi-stage approach with the purpose of extracting and processing features at different scales to achieve this. It has the "Head", the Tiny Feature Pyramid Network or (TFPN) or simply the "Neck, and the feature extraction, which is the "Backbone". the

algorithm was based on Depthwise Separable Convolution. 'ConvHead, 16' indicated the ConvHead module with 16 output channels, whereas all the head outputs were concatenated to produce detection results via non-maximum suppression (NMS).

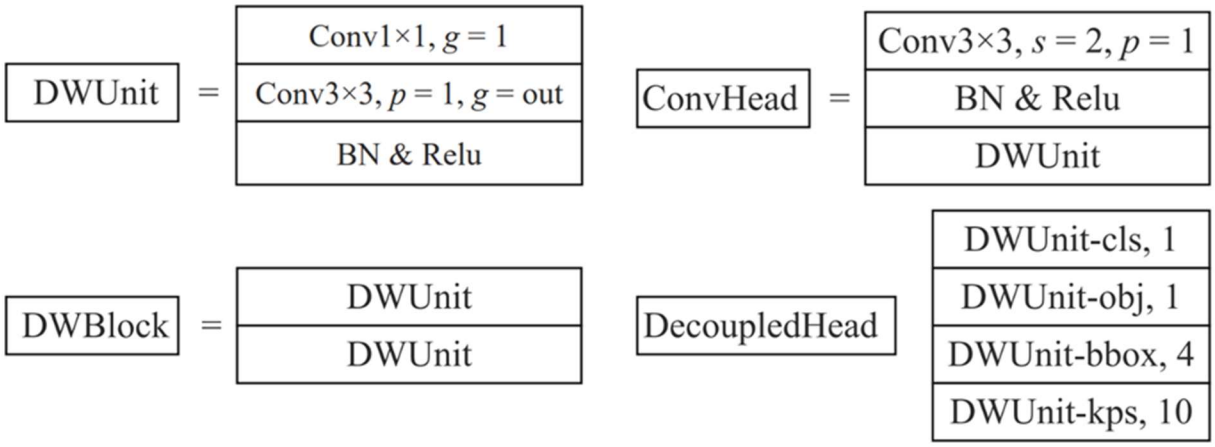


Figure 28: DWUnit and DWBlock

The “Backbone” consists of several stages, with each stage progressively reducing the spatial dimensions of the feature maps while simultaneously increasing depth (number of channels). The main components of the “Backbone” include:

a. ConvHead: A standard convolution layer with 3x3 kernels and a stride of 2, that initiates the feature extraction process.

b. Depthwise Separable Convolutions: Utilized instead of standard convolutions, to reduce computational and parametric costs by a factor of 8 to 9 times, sacrificing only a slight decrease in accuracy.

c. DWUnit: A module composed of a Depthwise Separable Convolution followed by batch normalization and an activation layer.

d. DWBlock: Consists of two DWUnits, further enhancing the feature extraction process.

Stage 0 contains the ConvHead module, Stage 1 includes a max-pooling layer, a ReLU activation, and two DWBlocks, the rest of the stages use DWUnits to refine feature maps and feed their outputs to the TFPN neck.

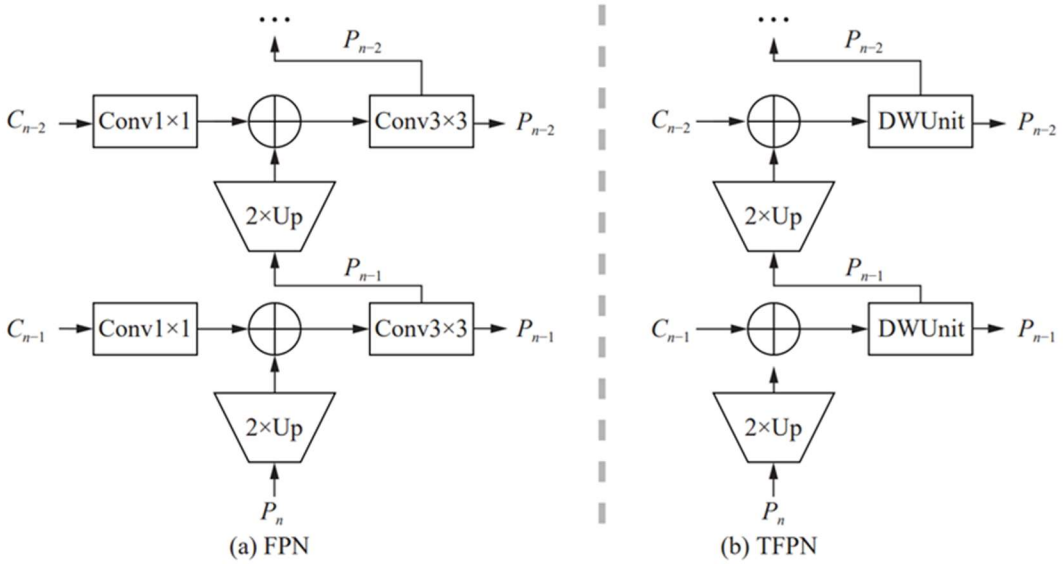


Figure 29: Comparison of the FPN (a) and TFPN (b)

The TFPN “Neck” is designed to fuse multiscale features extracted from the “Backbone,” It is based on the Feature Pyramid Network (FPN) but modified for better efficiency. FPN combines features from different scales utilizing a top-down pathway and lateral connections, enhancing the network's ability to detect objects of various sizes. TFPN uses Depthwise Separable Convolutions instead of standard convolutions to significantly reduce the number of parameters and lower computational cost. The TFPN “Neck” integrates features from different stages of the backbone, allowing the network to generate higher-level feature maps suitable for detection.

The components of the TFPN “Neck” include the Channel Adjustment, also known as Dimension Matching, which ensures the number of channels within feature maps from

different levels are identical before fusion, involving a 1×1 Convolution Upsampling or Resolution Matching ensuring that the spatial resolution of the feature maps from different levels match before fusion that involves a 2× Upsampling operation. Convolutional Operation (Feature Processing) processes the combined feature maps to generate the final output feature map in each level. If the standard FPN was used this would be a 3×3 Convolution, In the TFPN "Neck" it is replaced by a Depthwise Separable Convolution (DWUnit) to reduce computational cost and the number of parameters.

Figure 30: Combination of the features at different Scales in the TFPN neck

Where :

$$P_i = F_i(R_{dim}(C_i) + R_{size}(C_{i+1}))$$

a. P_i: The output feature map at level iii in the pyramid.
b. F_i: A Convolutional Operation that processes the combined features.
c. $R_{dim}(C_i)$: A Channel Adjustment operation for dimension matching applied to the feature map C_i.
d. $R_{size}(C_i+1)$: An Upsampling operation for resolution matching applied to the feature map C_i+1.

The combined operations $R_{dim}(C_i)$ and $R_{size}(C_i+1)$ ensure that the feature maps from different levels are identical in terms of both dimensions and resolution before they are fused together.

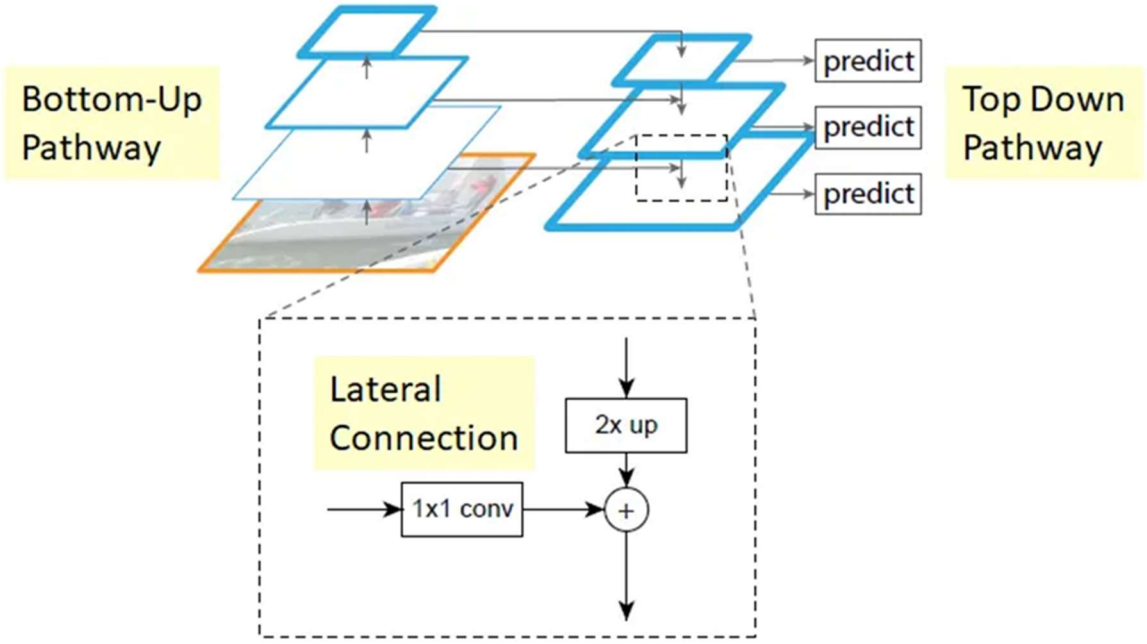


Figure 31: Feature Pyramid Network

In YuNet, the TFPN "Neck" takes advantage of FPN and Depthwise Separable Convolution. The feature map in Stage 2 are low-level features but the feature map in Stage 4 are high-level features because of their different depths.

Shown in Figure 31, the FPN introduces a top-down pathway and lateral connections to combine multiscale features. The top-down pathway in FPN can generate higher-resolution features by up sampling feature maps from higher pyramid levels. The connections combine feature maps of the same spatial size from the backbone and the top-down pathway.

The "Head" of YuNet is responsible for calculating the final detection predictions, it uses an Anchor-free mechanism to trivialize the prediction process by predicting bounding box coordinates directly, reducing the number of candidates for each location.

Positive Anchor Matching (simOTA) matches positive anchors by computing the Intersection over Union (IOU) between predicted bounding boxes and ground truth. Loss Functions utilizes various loss functions for different tasks:

1. Classification Loss (CrossEntropyLoss)
2. Bounding Box Regression Loss (Extended IoULoss)
3. Landmark Regression Loss (SmoothL1Loss)
4. Objectness Loss (CrossEntropyLoss)

$$L = \frac{1}{N}\sum^{N}(L_{cls} + \alpha_1 L_{obj} + \alpha_2 L_{bbox} + \alpha_3 L_{ldm})$$

Figure 32: Multitask Loss Minimization Equation

The multitask loss minimization equation used in YuNet combines multiple loss functions to optimize the training process where:

a. N: Total number of positive samples.
b. L_{cls} : Classification loss
c. L_{obj} : Objectness loss
d. L_{bbox} : Bounding box regression loss
e. L_{ldm} : Landmark regression loss
f. α_1, α_2, α_3 : Hyperparameters recommended to be 1.0, 5.0, and 0.1, respectively

C.3. YOLOv8 Algorithm's Flowchart

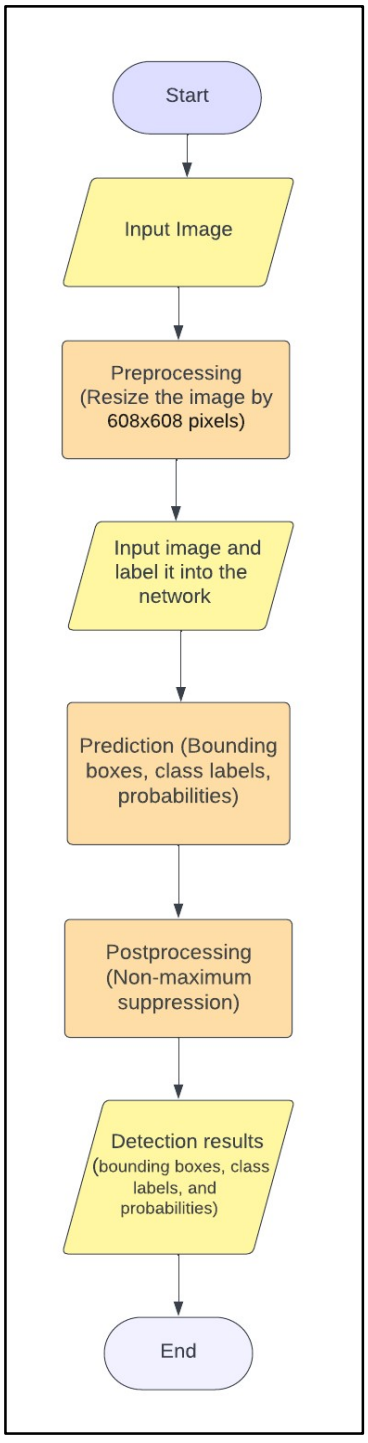


Figure 33: YOLOv8 Algorithm's Flowchart

Shown in figure 15, the flowchart of the YOLOv8 Algorithm depicted how it started with acquiring an input image from the user using a real-time video feed. After acquiring the video feed, the algorithm proceeded to pre-process the captured video by resizing its resolution to six hundred eight by six hundred eight pixels. Next, the data and labels of the captured video during pre-processing were sent to the network for recognition. After recognizing objects via data in the video feed, the prediction of objects took place using the gathered data from pre-processing and recognition of the real-time video feed. Once the algorithm had learned to identify the object, it proceeded to post-processing and detected the object's bounding boxes, labels, and probabilities.

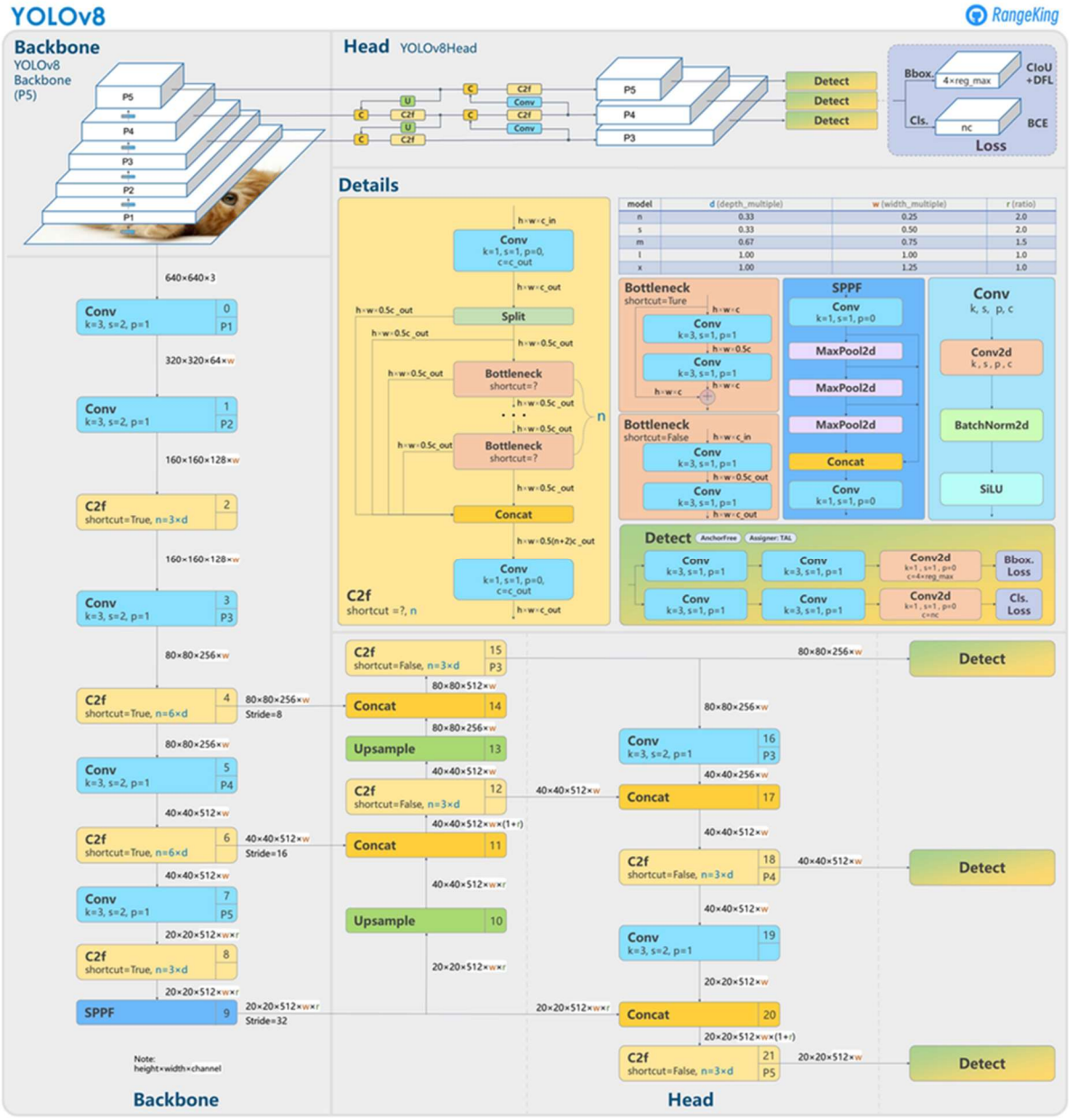


Figure 34: Network Architecture of YOLOv8

Clues Everywhere (Backbone): The first step is for the detectives (filters) to examine the picture thoroughly, layer by layer. They are looking for tiny clues like edges and colors that might reveal objects. This part is based on an experienced detective (existing system) but with some improvements.

Sharing Information (Neck): Once the detectives have their individual findings, they share them in a central meeting room (neck). This way, everyone has a broader view of the picture.

Specialists Take Over (Head): Now, specialist detectives (C2f module) come in. They use their unique skills to analyze the combined clues even further. They also have a keen eye for important details (attention blocks) to focus on the most likely spots where objects might be. Finally, different specialist teams (detection heads) give their best guesses about what objects they see (bounding boxes) and what they are (class labels).

Working Together (End-to-End): All the detectives (filters, specialists, teams) constantly learn from each other and improve their skills together. This way, the whole agency gets better at finding objects in pictures.

D. Potential Ethical Considerations

Using the integrated camera for object tracking could potentially infringe on people's privacy rights. The implementation made sure the technology was used responsibly and followed privacy laws and regulations. If the system got used in a public space for testing or data collection, getting informed consent from individuals who might be captured in the camera's footage was important. Any data collected during the study should have been anonymized to protect people's privacy. Making sure the data collected by the tracking system stayed secure was essential. This prevented unauthorized access, hacking, or misuse of sensitive information. The data should have been stored securely too, with only authorized individuals having access. Everyone involved in the research had a duty of confidentiality. Information about the participants, the data, and the findings should have stayed confidential unless explicit permission was given.

E. Storyboard



Figure 35: Home Window User Interface (UI) Mockup

Shown in figure 35, the mockup of the system includes an initial start-up window that introduces the user to the system. It includes the logo of the system, the name of the system, Labeled “Adaptive Object Tracking and Following System” which was labeled below the logo, and two buttons for initializing the System and terminating the system.

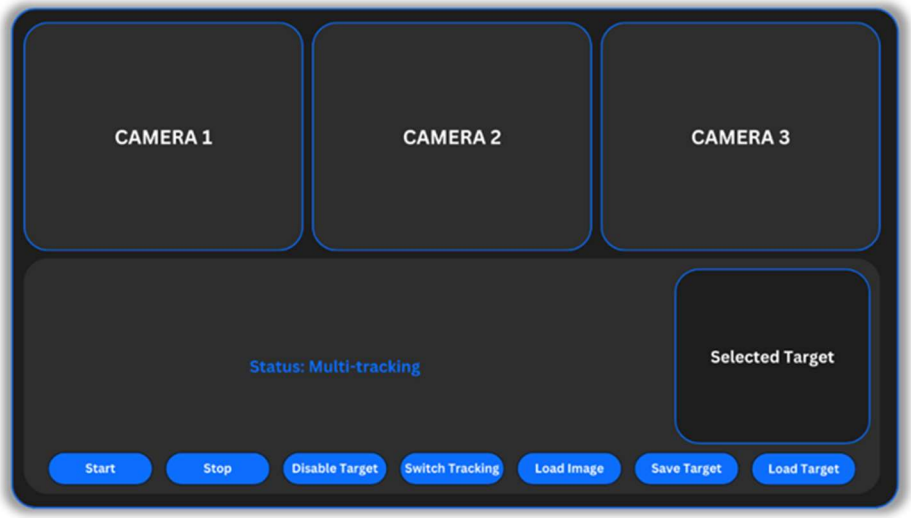


Figure 36: Main Window User Interface (UI) Mockup

Shown in figure 36, the mockup of main window of the system includes 3 camera feeds, a Status Indicator, Selected Target Indicator used for capturing the selected face, displaying it in the selected target display, and saving it in the database to be used to continuously track the face of the person selected, and buttons for controlling the system's functionalities such as initializing and terminating the cameras, disabling the object and facial tracking function, switching from Multi-Tracking to Single-Tracking mode, loading images for tracking other than selecting and saving a face. The system also includes a functionality to save and load a target via the database

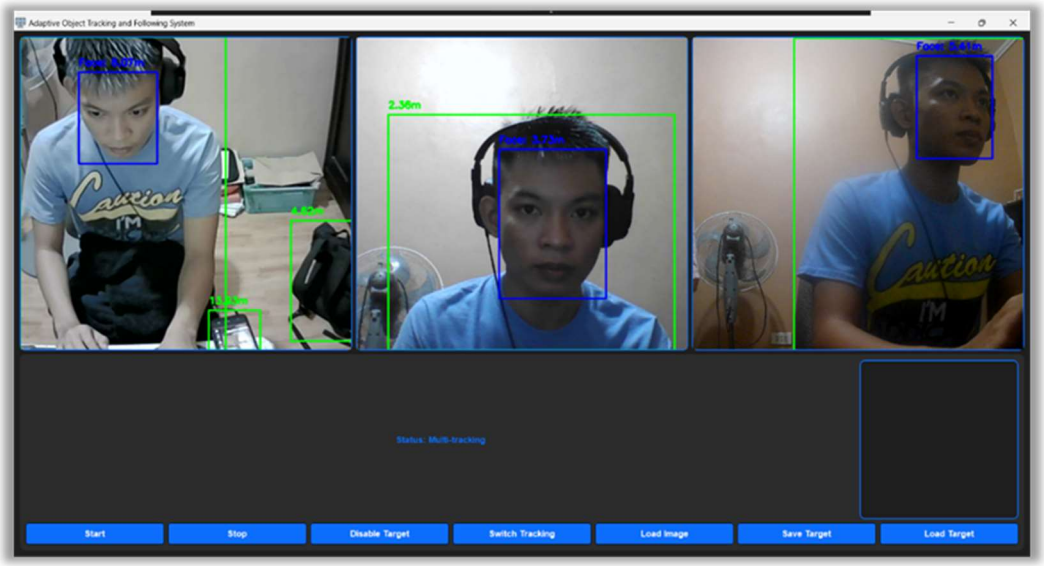


Figure 37: Start Button Function

Shown in figure 37, the main system and it's User Interface. The "Start" button initializes the three cameras, as well as the object and facial tracking functions using the YoloV8 and Yunet Algorithms. The "Stop" button terminates the three cameras initialized. The "Disable Target" button disables the target being tracked and removes the image displayed used to continuously track and identify the person that was previously selected.

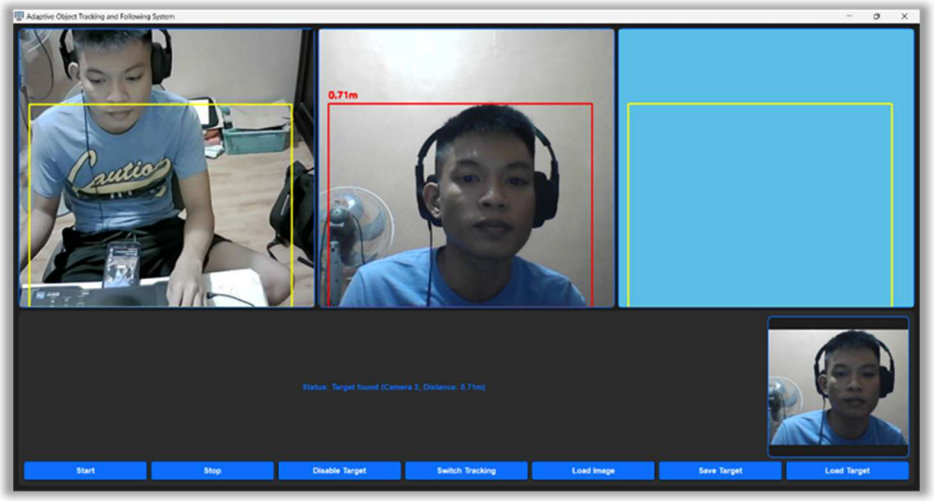


Figure 38: Switch Tracking Mode Function

Shown in figure 38, the “Switch Tracking” button switches the function of the Object and Facial detection from Multi-tracking objects and faces present in the trained model into Single-tracking specific objects or faces to be continuously tracked on three cameras with different points of view. Selection of a face is and object is done by selecting the respective bounding box where it is located, detected objects and faces can be selected on all three cameras, and displaying the selected face or object into the Selected Target Display.

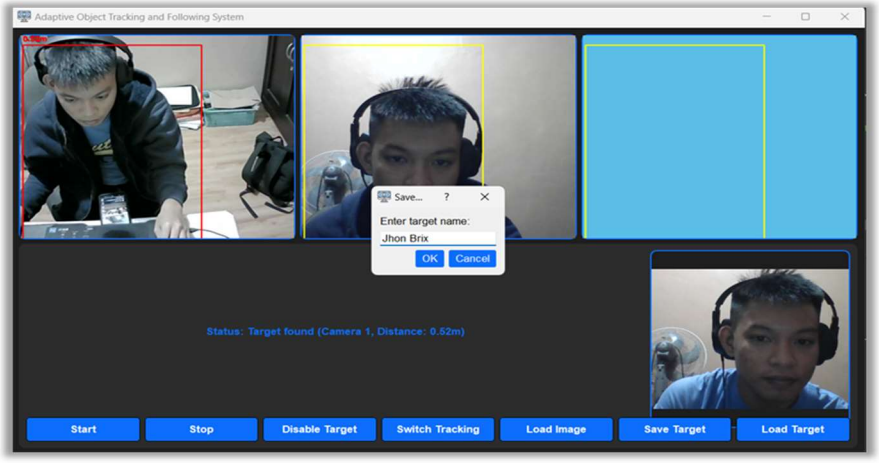


Figure 39: Save Target Function

Shown in figure 39, the “Save Target” button displays an option pane which displays a textbox to enter the target’s name, and saves the selected target into the database via SQLite.

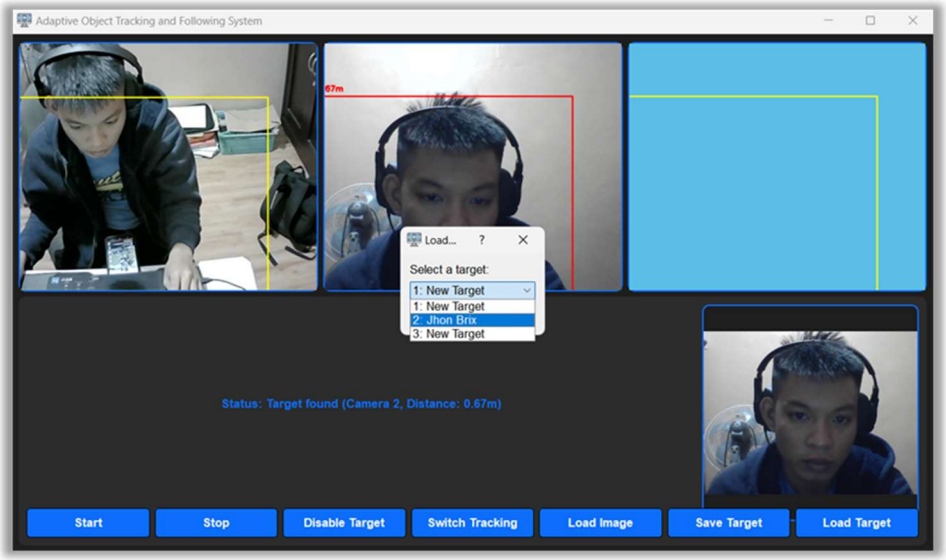


Figure 40: Load Target Function

Shown in figure 40, the “Load Target” button displays an option pane which displays a dropdown menu containing the saved images in the database. Selection of the items in the dropdown menu enables the user to load saved images from the database to be displayed in the Selected Target Display, and start tracking the object or person that is compared from the image loaded from the database.

F. Requirement Analysis and Documentation

F.1. Functional Requirements

Functional requirements were crucial elements that specified the particular functions and traits required for the thesis to achieve its intended purpose. These specifications usually listed the capabilities and conduct that the proposed system had to exhibit in order to be approved.

Prioritizing each task according to its significance for the thesis project's overall success was helpful when arranging functional requirements into a table. Tasks got divided into high and medium categories. High-priority tasks were usually essential to the thesis's main goals. Low-priority tasks were less urgent and could wait until higher-priority tasks were finished. Categorizing functional requirements allowed for a clear understanding of which tasks were crucial for the system.

F.1.1. USER MANAGEMENT

Table 11: Functional Requirements for User Management

Req. ID	Requirements Description	Priority	Complexity
FR1.	Users shall be able to click the start button.	High	Medium
FR2.	Users shall be able to view the adaptive object tracking and following detection in the main dashboard. The dashboard includes; ● Multiple Tracking Status ● Selected Target ● Disable Target ● Switching Tracking ● Load image ● Save Target ● Load Target	High	High
FR3.	Users shall be able to view the live video feed captured by the web camera.	High	Medium
FR4.	Users shall be able to see all the information in the dashboard.	Medium	High
FR5.	Users can focus on the movement on one specific object.	Medium	High
FR6.	Users can view the movement in multiple objects.	Medium	Medium

FR7.	Users shall be able to keep in track of the one object across the camera.	Medium	Medium
FR8.	User shall end the detection process using the disable target button.	Medium	Low

F.2. Non-Functional Requirements

Non-Functional Requirements were the guidelines for the system's performance, ensuring it operated effectively and efficiently. They ensure the system performed well and guide how it executed the system’s functions.

Table 12: Non-Functional Requirements

Code	Requirements Description
SS1	The system shall be required to click the start button.
SS2	The system must track the object in real-time.
SS3	The user shall be required to click the single tracking button from multi-tracking status, selected target button to view and follow the movements of objects.

G. Objective Testing

System testing is important for the system development life cycle, it will identify the defects, errors, and potential issues before making it available for public use. System testing is helpful to refine the quality, evaluate the reliability of the system, and fulfill the user's satisfaction. System testing plays a crucial role in the system development life cycle as it helps uncover defects, errors, and potential issues before the system is released for public use. The testing will be Black Box testing and Load testing for the system. [46]

A method called black box testing involves testing a system without knowing its internal mechanisms beforehand. Data gets inputted by testers. They observe resulting outputs. How the system reacts to anticipated user interactions and unforeseen ones gets assessed this way. The approach helps uncover information about system responsiveness, usability, and reliability.

Load Testing, on the other hand, helps figure out the maximum load capacity of software. It makes sure the software remains stable and responsive during peak usage periods. Out of 6 load testing criteria, the researchers selected 2 to perform. These were performance and scalability testing. Tools like Apache JMeter and LoadRunner are significant for conducting these tests efficiently. Insights into system behavior under different load conditions come from these tools. Load testing in software development also serves a crucial purpose - analyzing system performance under heavy loads or zero loads. Software needs to handle expected workload without functionality getting compromised, and this ensures that happens.

G.1. Functionality Testing

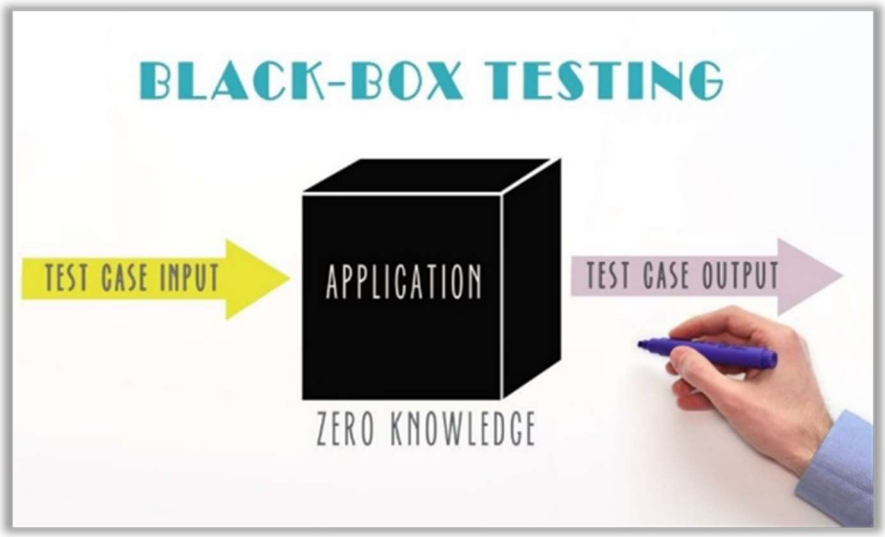


Figure 41: Black Box Testing

Shown in figure 41, the black box testing is the method of testing that the researchers will utilize. The functionality test focuses on how the system works based on user experience without the need to see the internal structure of the system. The test is conducted by user interaction with the system, as well as user input actions. The test results of the system will be compared to from the actual output into the expected output to identify whether the interactions and user input actions are successful or not. After comparing the expected output to the actual output, the tester will now identify how much success was encountered and the time of the test during the evaluation.

Table 13: Black box Testing

INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	SUCCESS RATE
Clicked bounding box from Camera 1	Display Image in the Selected Target Display	Capture Bounding box Image and display it in the Selected Target Display for Re-Identification.	
Clicked bounding box from Camera 2			
Clicked bounding box from Camera 3			
YoloV8 Object Detection	Object Detection Bounding Box	Display corresponding bounding boxes in the Camera Frame	
Yunet Face Detection	Face Detection Bounding Box	Display corresponding bounding boxes in the Camera Frame	
Feature Extraction	Get Bounding Box Image	Extract and Display Facial or Object Features for Feature Matching	
Feature Matching	Compare and Match Extracted Features for Target Locking	Match Features Between the extracted features and match the features across three Cameras with a Yellow Bounding Box	
Global Re-Identification	Target Lock Indicator	Re-Identify and Detect Matched Face and Object features with a Red Bounding Box	

CHAPTER 5

RESULTS AND DISCUSSION

For this chapter, the following screenshots depicted the user interfaces for the Adaptive Object Tracking and Following Using YOLOV8 and YUNET Fast R-CNN algorithm.

5.1. Design and Development of Adaptive Object Tracking and Following Using YOLOV8 and YUNET Fast R-CNN algorithm.

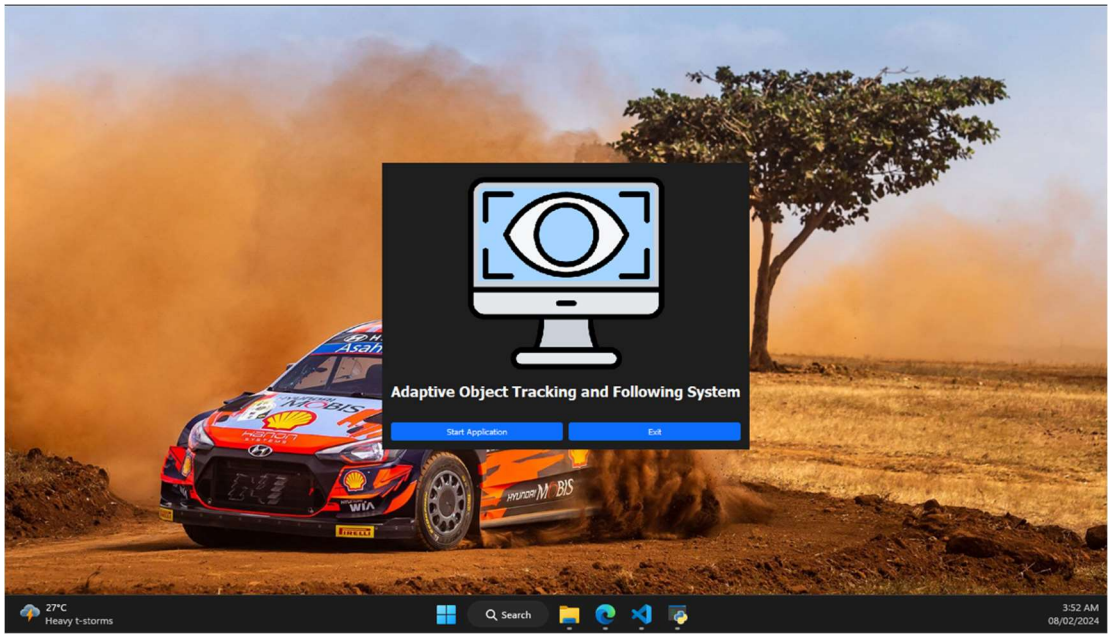


Figure 42: Application window (Actual Testing)

Shown in figure 42, Upon launching the application, this page served as the starting point of the system. To access the live video feed captured by the camera, a "Start" button will be prominently displayed on the home page. The dashboard is the main interface where you'll be able to access and interact with the live camera feed.

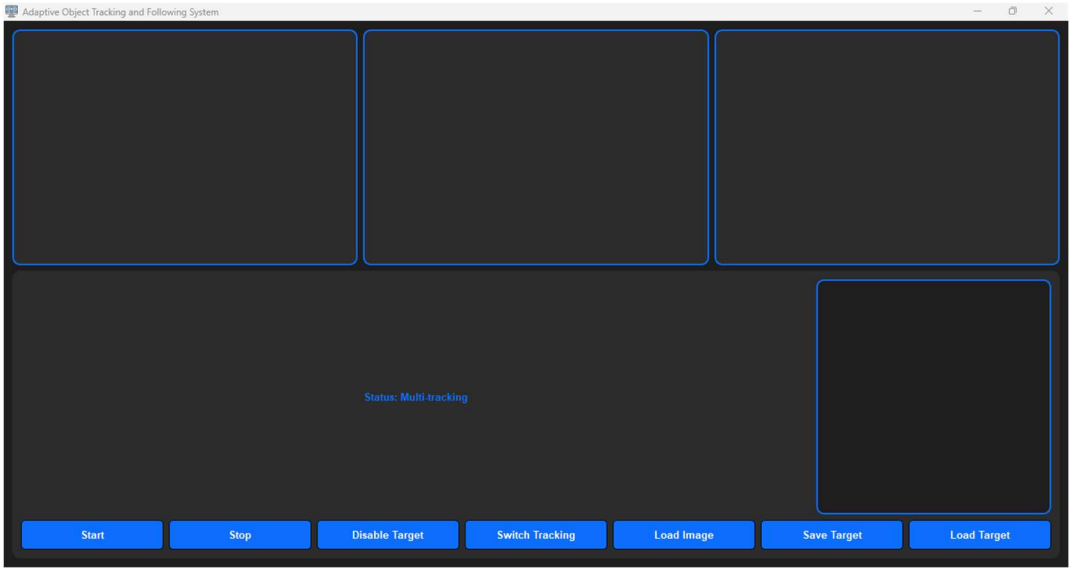


Figure 43: Main Application window (Actual Testing)

Shown in Figure 43, the Adaptive Object Tracking and Following application window ‘s Main Window without any camera feeds and Inferences.

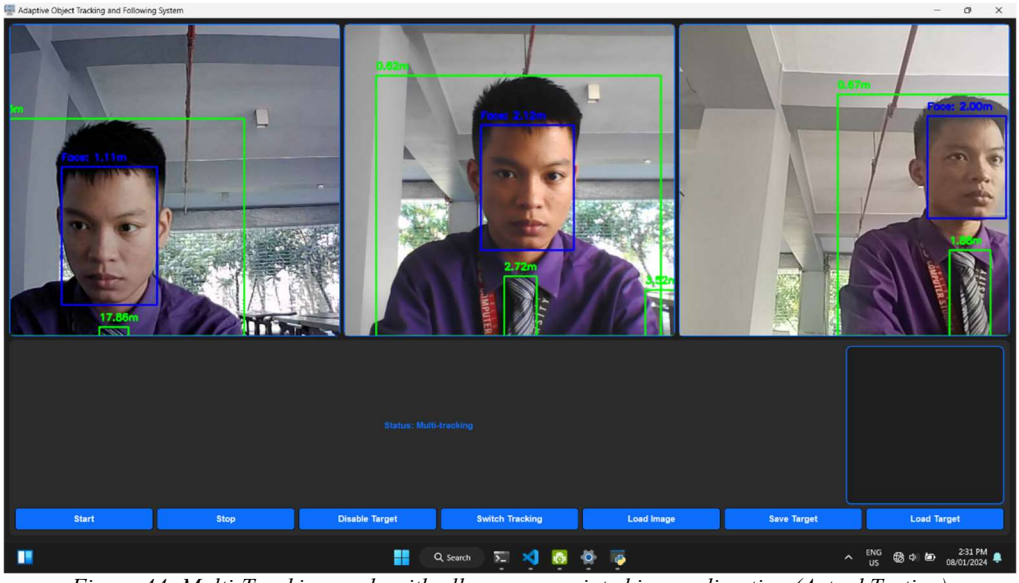


Figure 44: Multi-Tracking mode with all cameras pointed in one direction (Actual Testing)

As shown in figure 44, all cameras pointed at the user in multi-tracking mode as shown in the Status indicator, Green indicator indicates the YoloV8, and the Blue Indicator Indicates the YuNet. The Actual testing was conducted at Tarlac State University, College of Computer Studies. The detection page offers a set of tools for analyzing the live video

feed from your camera system. It essentially transforms the passive video feed into an interactive environment for tracking and identifying objects and faces. Upon launching the system and reaching the dashboard it displays the live camera feed, giving you access to powerful detection features. This page allows you to choose how you want to interact with the cameras. With the Single Tracking button, the user can visually follow the movement of a specific object within the frame. The multi-tracking option lets the user track multiple objects simultaneously. If the user needed to maintain focus on a single object even if it moves across different camera views, the Target Lock button enables the user to track it even as it moves across the camera. Additionally, the user can choose to focus on just faces with the Face Detection button or detect any kind of object within the frame using the Object Detection button. The Dis-engage button allows the user to end the tracking and detection functionalities.

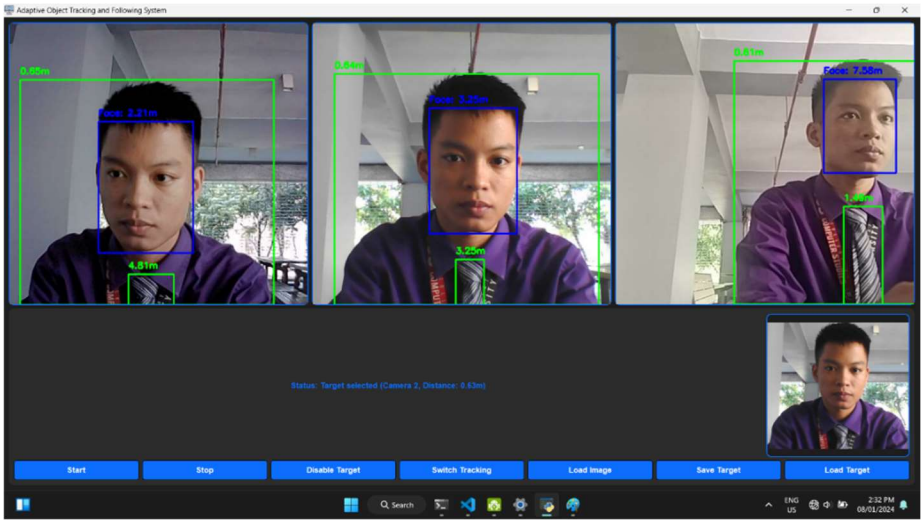


Figure 45: Selection (Actual Testing)

As shown in figure 45, the selection of a subject and storing the image in the Selected target display for facial feature extraction and comparison while still in the multi-tracking mode while the camera is pointed in a single direction.

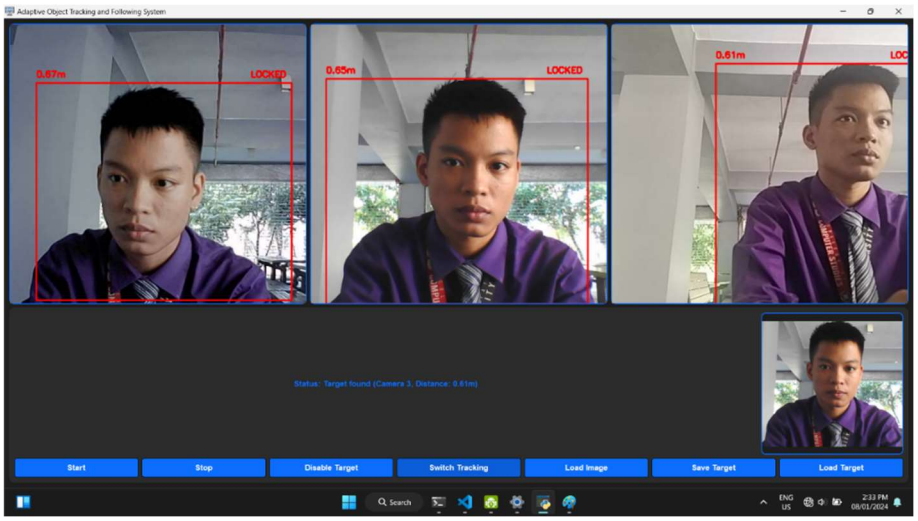


Figure 46: Single Tracking mode and Target Locking (Actual Testing)

As shown in figure 46, the object detection and facial detection switches into single tracking mode, starting the continuous tracking of the subject with feature extraction in both YoloV8 and YuNet algorithm. The Subject being tracked has “LOCKED” labeled on the bounding box, indicating that only that subject being tracked. The selection process involves switching to single tracking mode to continuously track the subject using feature extraction through the YOLOv8 and YuNet algorithm. This process involves two steps Detection and Tracking. Detection uses both YOLOv8 for Object Detection and YuNet for Facial Detection to identify potential subjects. Tracking begins once a subject is selected, and based on feature matching continuously follows the subject with “LOCKED” labelled on the subject’s bounding box.

$$F(I_t) = \{f_1, f_2, \ldots, f_n\}$$

Figure 47: Feature Extraction Equation

In the Feature Extraction function shown in figure 47, F represents the process of extracting features from a video frame using YOLOv8 and YuNet. Let I_t be the input image or frame at time t. This was expressed as $F(I_t)=\{f1,f2,\ldots,fn\}$, where $F(I_t)$ denotes the set of features extracted from the image at time t.

$$S(F_S, F_D) = \text{similarity score}$$

Figure 48: Similarity Measure

In the equation shown in figure 48, Similarity measure S is used to compare the features of detected objects with those of the target subject. Let F_S be the feature set of the subject to be tracked, and F_D be the feature set of detected objects.

$$T(S) = \begin{cases} \text{"LOCKED"} & \text{if } S \geq \theta \\ \text{"NOT LOCKED"} & \text{otherwise} \end{cases}$$

Figure 49: Selection and Tracking Function

In Figure 49, this function switches the system into tracking mode and continuously tracks the subject with the highest similarity score. The subject with the highest similarity score is marked "LOCKED." θ is a threshold value that determines if a subject stays tracked based on the similarity score.

$$C(t) = T(S) \times F(I_t)$$

Figure 50: Continuous Tracking Function

In the Continuous Tracking function shown in figure 50, *C(t)* represents the continuous tracking of the subject over time, and *T(S) x F(I_t)* implies applying the tracking function to the extracted features over time.

$$\Phi(I_t, F_S) = C(t) \cdot T\left(S\left(F_S, F(I_t)\right)\right)$$

Figure 51: Complete Selection Function

The Complete Selection Function in Figure 51 shows all the previously mentioned components combined. The complete selection function Φ can be described as *Φ (I_t,F_S) = C(t) · T(S(F_S,F(I_t)))*. Here, I_t is the video frame at time *t*, F_S represents the feature set of the subject to be tracked, *F(I_t)* was the feature extraction function applied to the current frame, *S* was the similarity measure comparing detected objects to the target subject, *T* was the tracking function that switches the system to continuous tracking mode, and *C(t)* represents continuous tracking over time. The equation represents a system that first extracts features from the input frame, compares similarity with the target subject, and then either locks the subject for continuous tracking or not based on a similarity threshold. These functions are implemented in the system, with YOLOv8 and YuNet algorithms both utilized for handling feature extraction and your tracking system managing similarity computation and continuous tracking.

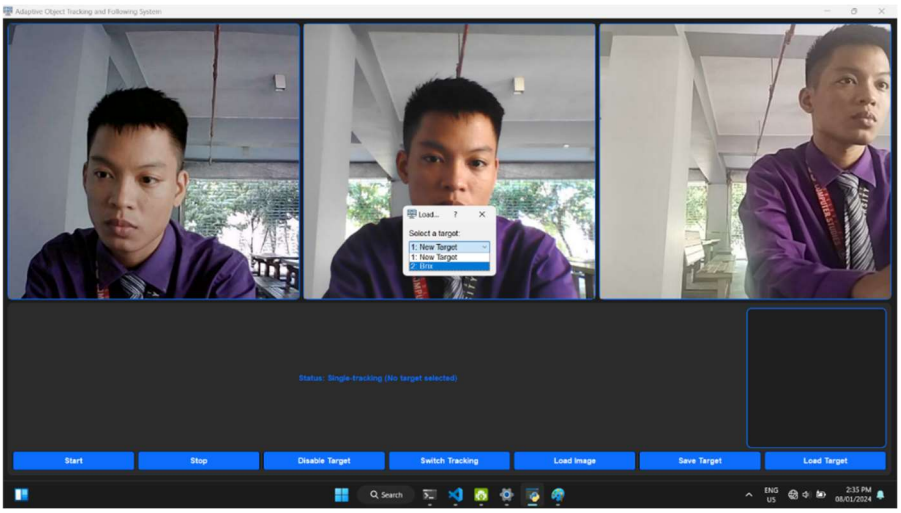


Figure 52: Loading Image via the Database (Actual Testing)

As shown in Figure 52, a window containing image data from the database for loading into the Selected Target Display for feature extraction and Single tracking detection.

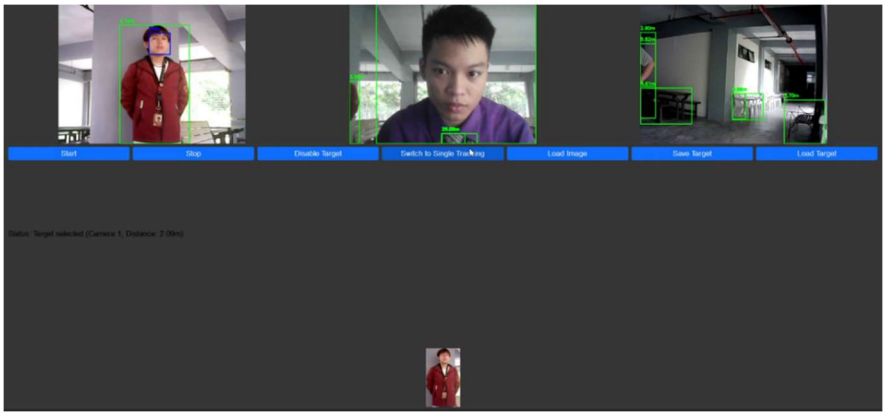
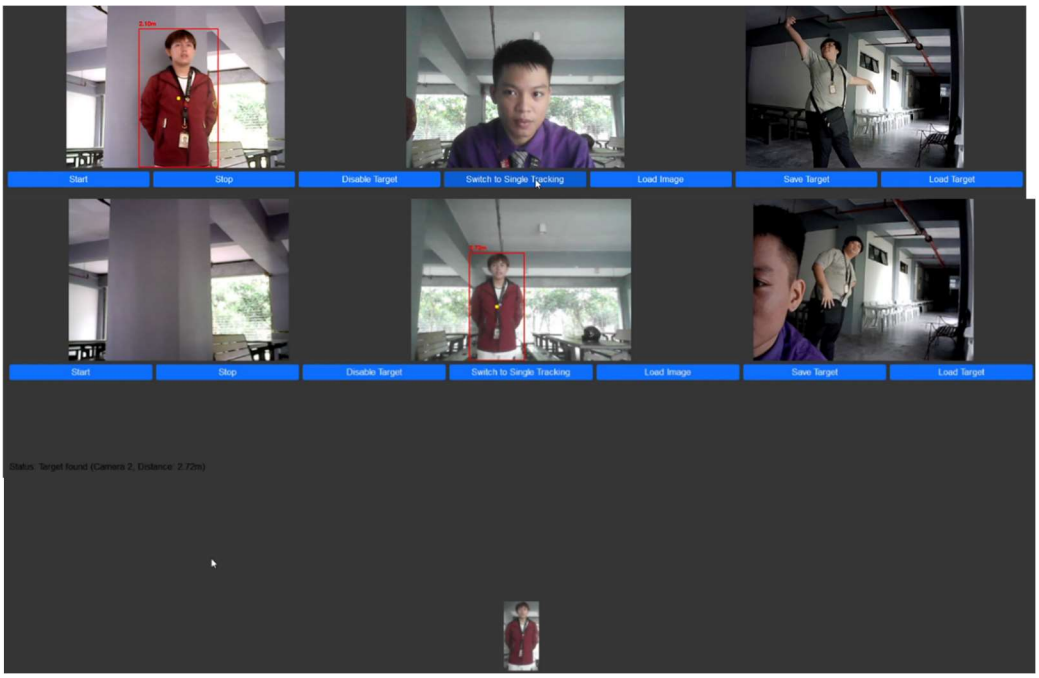


Figure 53: Multi-Tracking mode with all cameras pointed at different directions (Actual Testing)

As shown in figure 53, the selection of a subject and storing the image in the Selected target display for facial feature extraction and comparison while still in the

multi-tracking mode while the camera is pointed at different directions. The area where the system was tested is in low light condition.

Figure 54: Single and Target tracking mode with all cameras pointed at different directions (Actual Testing)



As shown in figure 54, the Single and Target tracking mode of the subject while the cameras are pointed in different directions, the subject was selected and tracked continuously in Camera 1 (Left Camera).

Figure 55: Single and Target tracking mode with the selected subject in Camera 2 (Actual Testing)

As shown in figure 55, while the subject was selected in Camera 1, he was seen and tracked in Camera 2 (Middle Camera).

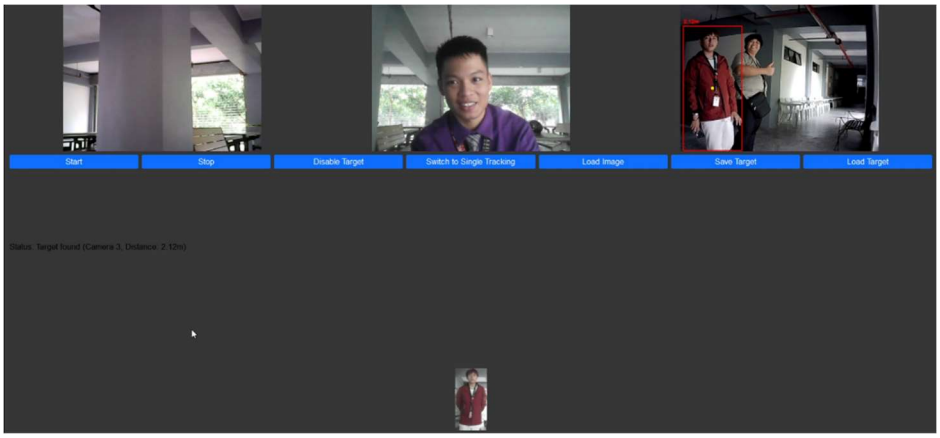


Figure 56: Single and Target tracking mode with the selected subject in Camera 3 (Actual Testing)

As shown in figure 56, while the subject was selected in Camera 1, he was seen and tracked in Camera 3 (Right Camera) with another person.

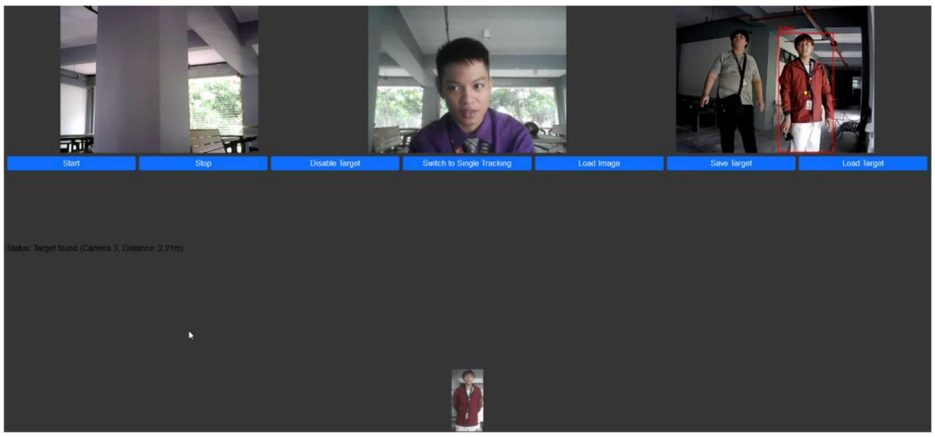


Figure 57: Single and Target tracking mode with another example (Actual Testing)

As shown in Figure 57, another example of the Single and Target tracking mode of the System, the subject is continuously tracked in Camera 3. The subject was selected in the Camera 1 (Left Camera)

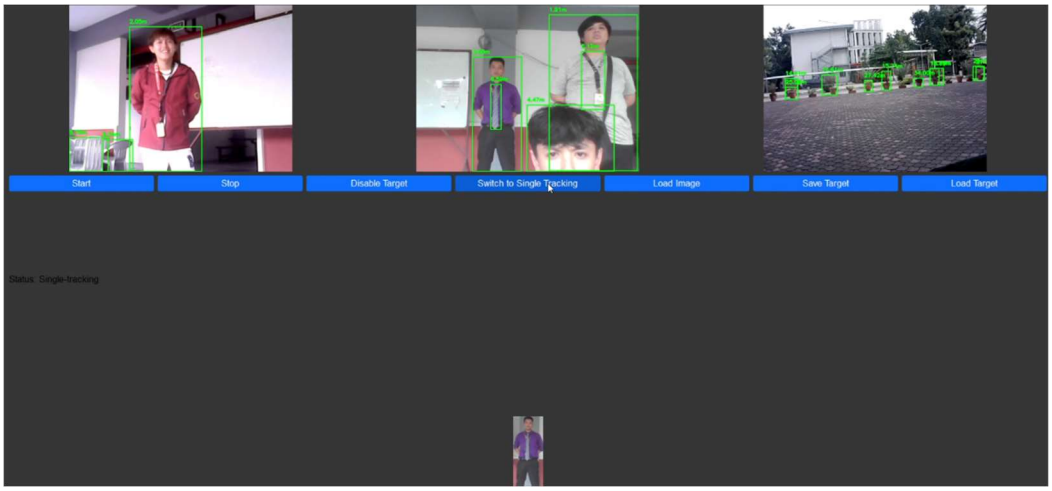


Figure 58: Multiple tracking mode in a different area. (Actual Testing)

As shown in figure 58, we moved into a different area to test the adaptive object tracking and following of the system. The system is in multi-tracking mode.

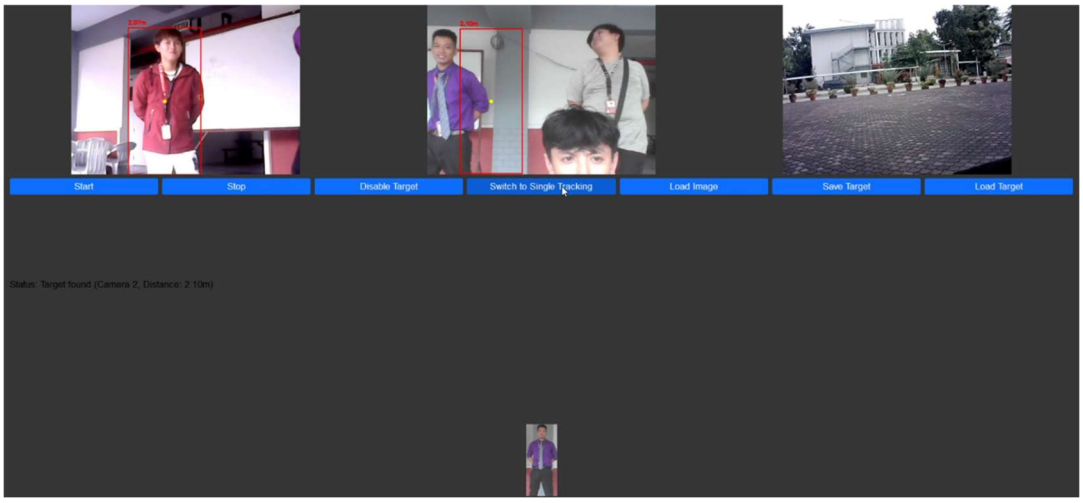


Figure 59: Single and Target tracking mode with the selected subject in Camera 2. (Actual Testing)

As shown in figure 59, the subject was selected in Camera 2, upon selection, the subject was immediately tracked and displayed in the Selected Target Display below with multiple other people.

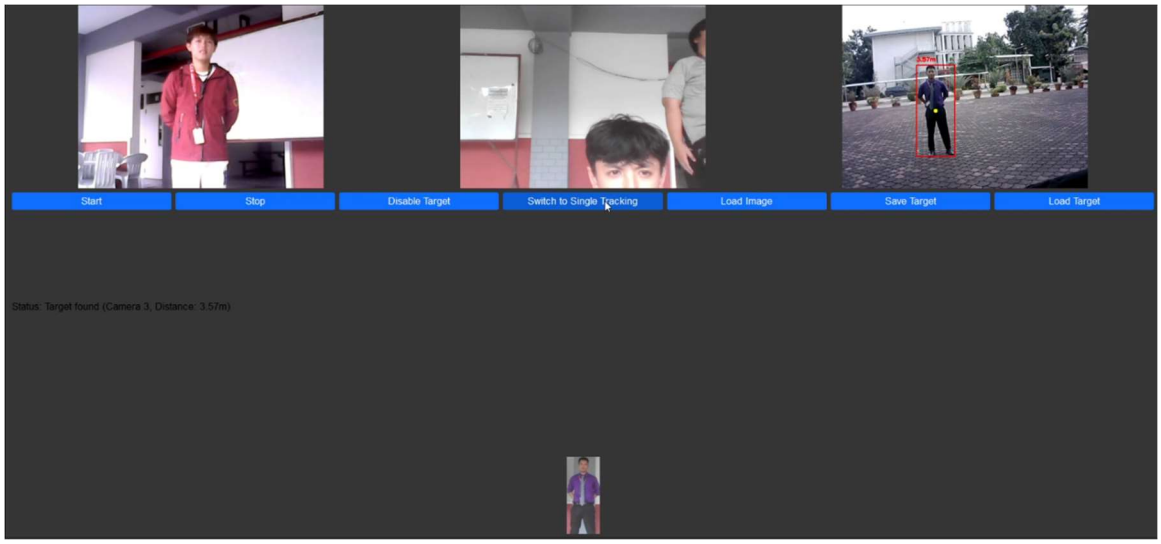


Figure 60: Single and Target tracking mode with the selected subject in Camera 3. (Actual Testing)

As shown in figure 60, the subject is continuously tracked in Camera 3. The subject was selected in the Camera 2 (Middle Camera).

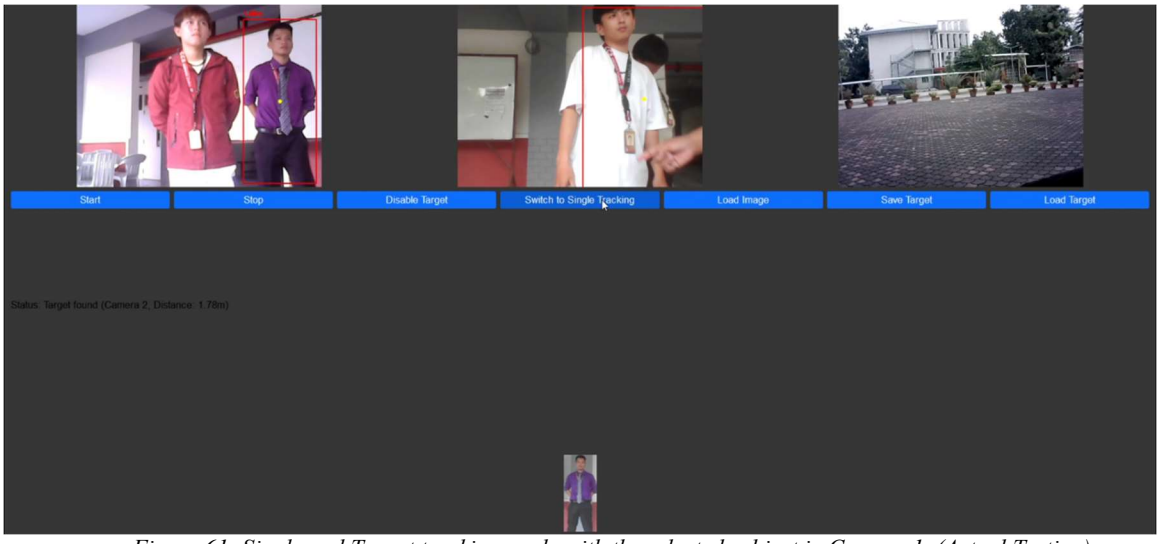


Figure 61: Single and Target tracking mode with the selected subject in Camera 1 (Actual Testing)

As shown in figure 61, the subject is continuously tracked in Camera 1. The subject was selected in the Camera 2 (Middle Camera). The selected subject was able to be continuously tracked and locked moving from Camera 2, Camera 3, and Camera 1.



Figure 62: The Proponents Testing the System (Actual Testing).

As shown in figure 62, the proponents testing the system at the locale where the system is supposed to be tested. Location was the Tarlac State University, College of Computer Studies.

5.2. To evaluate the performance of an adaptive object tracking and following in terms of robustness and functionality through Blackbox and Load Testing:

5.2.1. Blackbox Testing

Table 14: Black Box Result

INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	SUCCESS RATE
Clicked bounding box from Camera 1	Display Image in the Selected Target Display	Capture Bounding box Image and display it in the Selected Target Display for Re-Identification.	28/30
Clicked bounding box from Camera 2			
Clicked bounding box from Camera 3			
YoloV8 Object Detection	Object Detection Bounding Box	Display corresponding bounding boxes in the Camera Frame	30/30
Yunet Face Detection	Face Detection Bounding Box	Display corresponding bounding boxes in the Camera Frame	30/30
Feature Extraction	Get Bounding Box Image	Extract and Display Facial or Object Features for Feature Matching	30/30
Feature Matching	Compare and Match Extracted Features for Target Locking	Match Features Between the extracted features and match the features across three Cameras with a Yellow Bounding Box	30/30
Global Re-Identification	Target Lock Indicator	Re-Identify and Detect Matched Face and Object features with a Red Bounding Box	30/30

As shown in Table 14, the functionality test was conducted by testing the functions of the features of the system. The features unique to the system include the Three Cameras running with the YoloV8 Object Detection and Yunet Facial Detection.

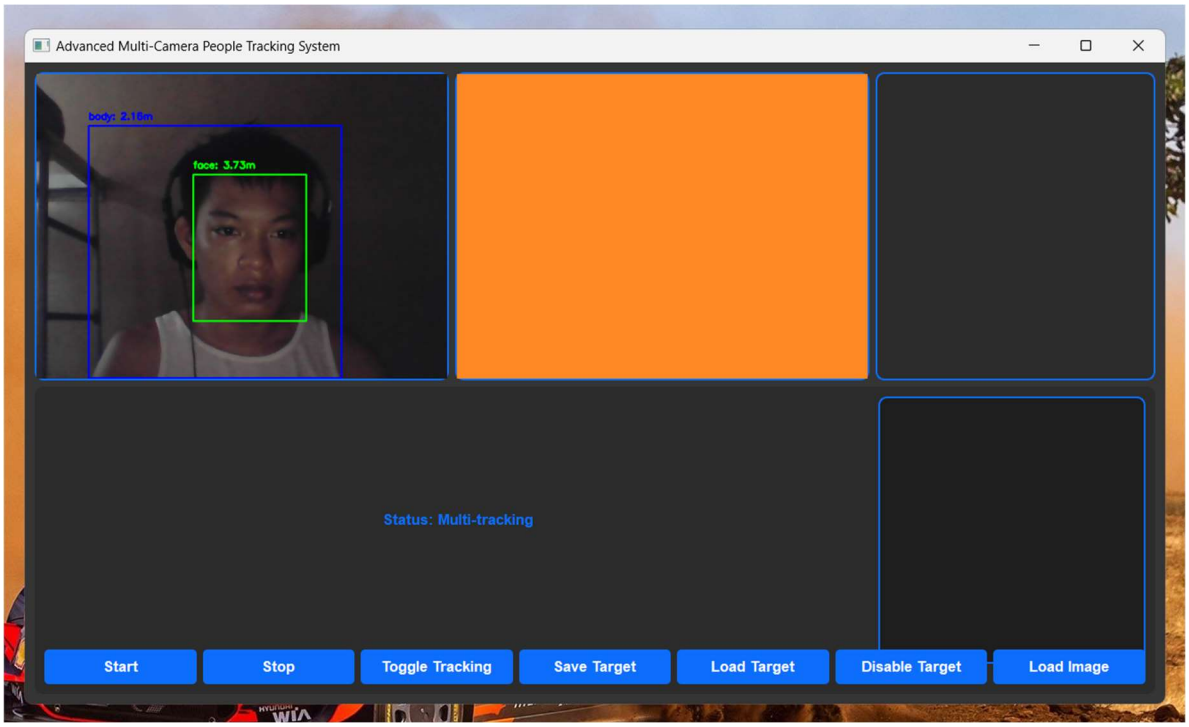


Figure 63: Camera Access Failure

In executing the cameras, there were slight inconsistencies in initializing the three cameras (27/30) and terminating camera processes (26/30), suggesting occasional failures in these tasks, those occasional failures is caused by the cameras not being connected to the computer.

Shown in Figure 63, the system runs normally. However, if one or more cameras are disconnected from the computer when the Cameras are initialized, it will fail to display the camera feed.

When it came to object and face detection, both YoloV8 and Yunet systems performed flawlessly. YoloV8 object detection met expectations. It accurately displayed corresponding bounding boxes in the camera frame, getting 30 out of 30 right. Yunet face detection did the same. It also got 30 out of 30 by accurately displaying bounding boxes for face detection in the camera

frame. Feature extraction and matching were both executed effectively. The system extracted and displayed features for matching, scoring 30/30. Feature matching worked particularly well. The system correctly compared and matched features across three cameras. A yellow bounding box highlighted them. Global re-identification was performed without issues too. The system re-identified and detected matched face and object features using a red bounding box, scoring 30/30 again. However, tests conducted when using different camera resolutions and settings showed a success rate of 3/30. This indicated that with different camera settings, global re-identification failed to re-identify and track the selected face or object. Tracking the subject across different cameras failed. It was taken into account that all camera parameters used across the three cameras for detection and re-identification should be the same. Resolution and camera settings needed to match. In Multi-Tracking mode, the blue bounding box represented Yunet facial detection. The green bounding box represented YoloV8 object detection. In Single-Tracking mode, the yellow bounding box indicated the algorithm was trying to recognize the face or object with feature matching. The red bounding box signified successful re-identification of the face or object being tracked through global re-identification. Features extracted between the captured subject and the camera matched and were continuously tracked.However, the tests conducted when using different camera resolutions and camera settings showed a success rate of 3/30, indicating that with different camera settings, global re-identification failed to re-identify and track the selected face or object, failing to track the subject across different cameras. It was taken into account that all camera parameters used across the three cameras for detection and re-identification of selected objects and faces should be the same in terms of resolution and camera settings. In Multi-Tracking mode, the blue bounding box represented the Yunet facial detection, while the green bounding box represented the YoloV8 object detection. In Single-Tracking mode, the yellow bounding box indicated that the algorithm was trying to recognize the face or object with feature matching, and the red bounding box signified that the face or object being tracked was successfully re-identified

through global re-identification, with the features extracted between the captured subject and the camera matching and being continuously tracked.

5.2.2. Load Testing

The Load Testing that was performed by the researchers was the JMeter for testing the Response Time. For the benchmarking of the YOLOv8 and YuNet algorithm models It was tested by code, system runtime testing, and actual testing.

The benchmark of the YoloV8 model and YuNet model was tested using the benchmark source code on the Ultralytics website on four conditions. The benchmark of the YuNet algorithm was tested using the Github repository for open source OpenCV DNN with benchmarks on different platforms. The benchmarks and runtime tests were dependent on the hardware requirements of the computer platform that was used during the development and testing of the system.

The researcher tried each iteration. Creating an instance of the Camera Widget class happened. This set up a camera feed. YuNet and YOLO models did real-time face and object detection. Objects that got detected were annotated on frames. Displaying processed frames with annotations followed. Each iteration had the application continuously updating the frame display. The latest processed frame kept showing. Running the code resulted in iterations happening in sequence. Camera widget application execution showed real-time frame processing. Face and object detection annotations appeared. System performance and algorithm efficiency affected execution time for each iteration. Application responsiveness varied overall. CPU and memory utilization got monitored as part of resource usage tracking. This assessed application performance under load.

5.2.2.1 JMeter Response Time Testing

The load testing carried out for the system was to assess its response time in terms of the runtime of cameras and runtime response. JMeter was the load testing software used to conduct tests on the performance of the system in scenarios where multiple instances of the system were running on a single computer and to test the system on the computer, all while following the specified hardware requirements for developing and running the system.

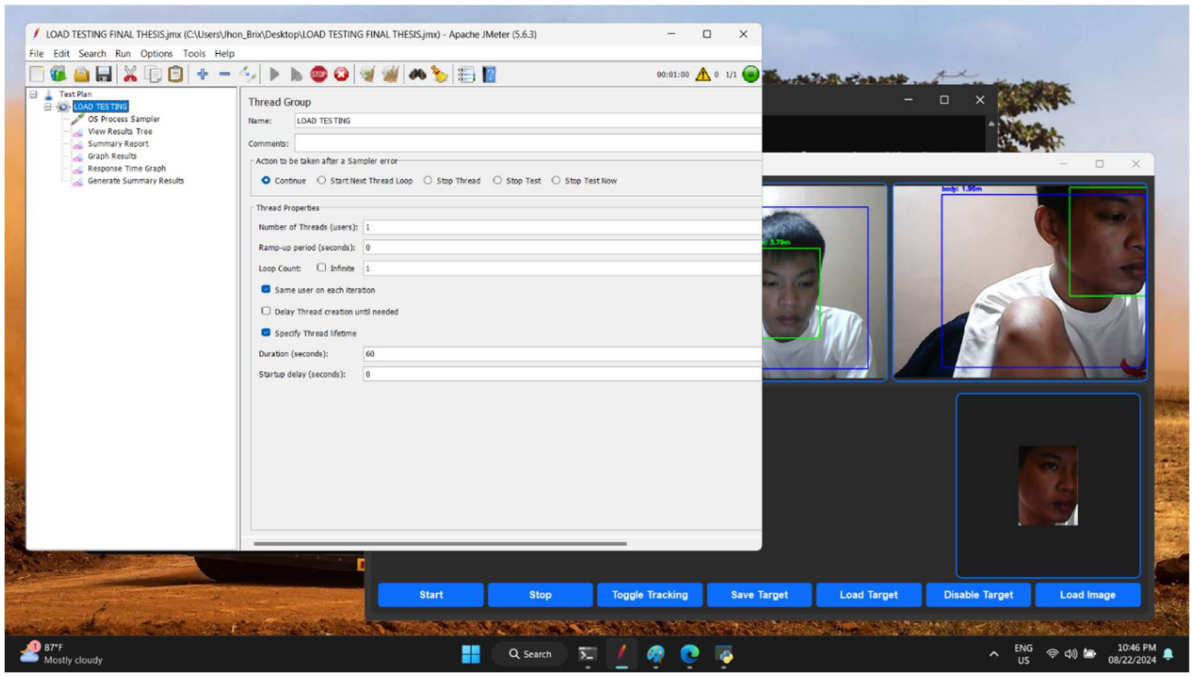


Figure 64: JMeter

The load testing software JMeter, shown in Figure 64, begins the assessment of the system's response time by specifying the parameters in the Thread Properties. These parameters include the Number of Threads or Users, the Ramp-up Period (which is the interval where each thread will execute, measured in seconds), and the Loop Count (this indicates how many times a thread will

run before testing stops). If the test requires an indefinite time frame, the Loop Count can also be set to infinite.



Figure 65: Response Time Testing Parameters

Figure 65 shows the parameters for response time testing. They're configured like this: The Number of Threads is set to one. This indicates the test will be conducted by a single instance of the system running after one minute. The Ramp-up Period is set to zero. This means there won't be any delay in system execution during the test since only one instance is running. The Loop Count is set to infinite. The system will run continuously in a loop until the desired number of runs has been completed after termination. Lastly, the thread lifetime sets how long the system runs before termination.

To obtain the response time results, the system runs the test repeatedly for one minute at a time over the span of one hour. This approach ensures that system resources are recorded over time, allowing visualization of how quickly the system responds to the computer after each minute.

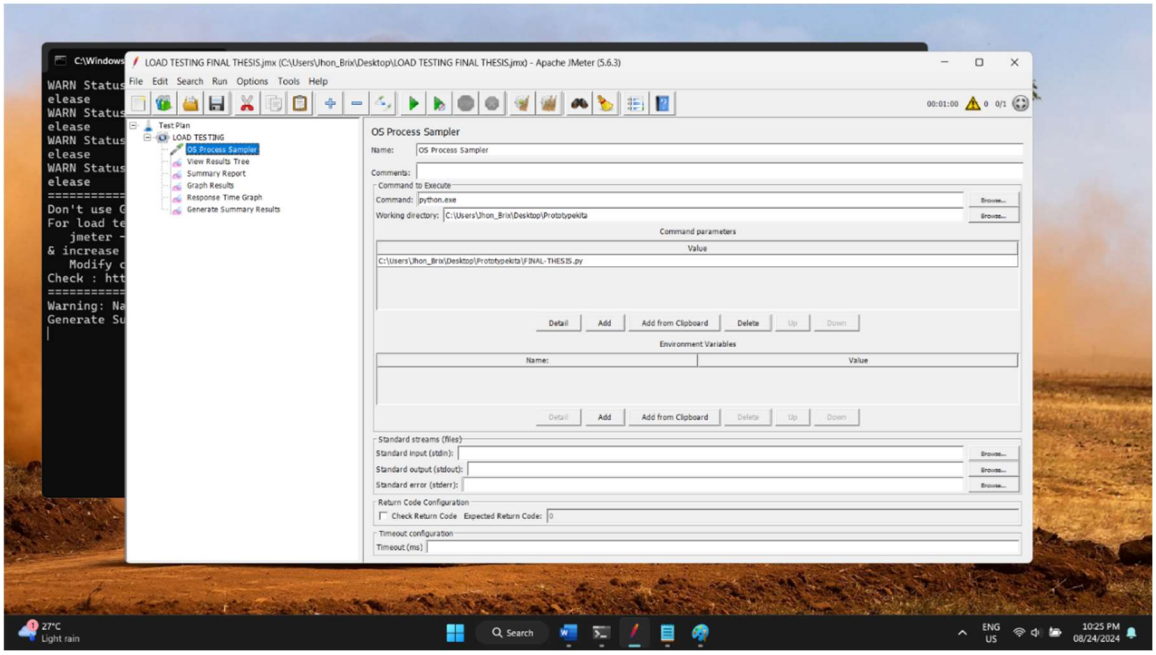


Figure 66: System Runtime Execution Settings

In Figure 66, the OS Process Sampler executes the command for running the software or system under test. The command specifies the Python runtime environment, and the Working Directory of the System ensures that the system is executed from the directory where all necessary functions and files are located. The Command Parameters call the specific file containing the system's code. Once the Python environment is running and the system is invoked by JMeter, it starts executing the application according to the specified Thread Properties for the test.

Table 15: Summary of Results after 10 Minutes of Runtime

Label	LOAD TESTING: OS Process Sampler
# Samples	10 exectutions
Average	60,319 ms
Min	47,884 ms
Max	67,127 ms
Std. Dev.	5051.69 ms
Error %	0.00%
Throughput	59.7124 requests per second
Received KB/sec	0.01 KB/sec
Sent KB/sec	0 KB/sec
Avg. Bytes	751 B

Table 15 shows the initial 10-minute testing period. It revealed a system operating under optimal conditions with fresh resource allocation and minimal memory fragmentation. The system completed 10 executions in this phase. Response time averaged 60,319 milliseconds - about 60.32 seconds per execution cycle. Conventional applications might make this response time appear high. But given the computational demands, it's contextually appropriate. Running YOLOv8 for general object detection and YuNet for specialized face detection at the same time on real-time video streams required this. The system demonstrated impressive throughput of 59.7124 requests per second. This indicated strong initial processing capabilities and efficient resource utilization when memory and computational resources were plentiful. Response times during this phase ranged from a minimum of 47,884 milliseconds to a maximum of 67,127 milliseconds. This was a

relatively narrow range of 19,243 milliseconds, which suggested consistent performance characteristics. Standard deviation came out to 5,051.69 milliseconds. Variability from the mean was approximately 8.4%, which indicated predictable and reliable behavior. Minimal data transfer rates happened - 0.01 KB/sec got received and 0 KB/sec got sent. Bytes per request averaged 751 bytes. What stood out most was the perfect 0.00% error rate the system maintained. Robust fault tolerance and error handling mechanisms were demonstrated. The initial testing phase established that core algorithmic implementations and system architecture worked soundly under optimal conditions. During early operation stages, the system handled its intended workload efficiently before resource constraints began affecting performance.

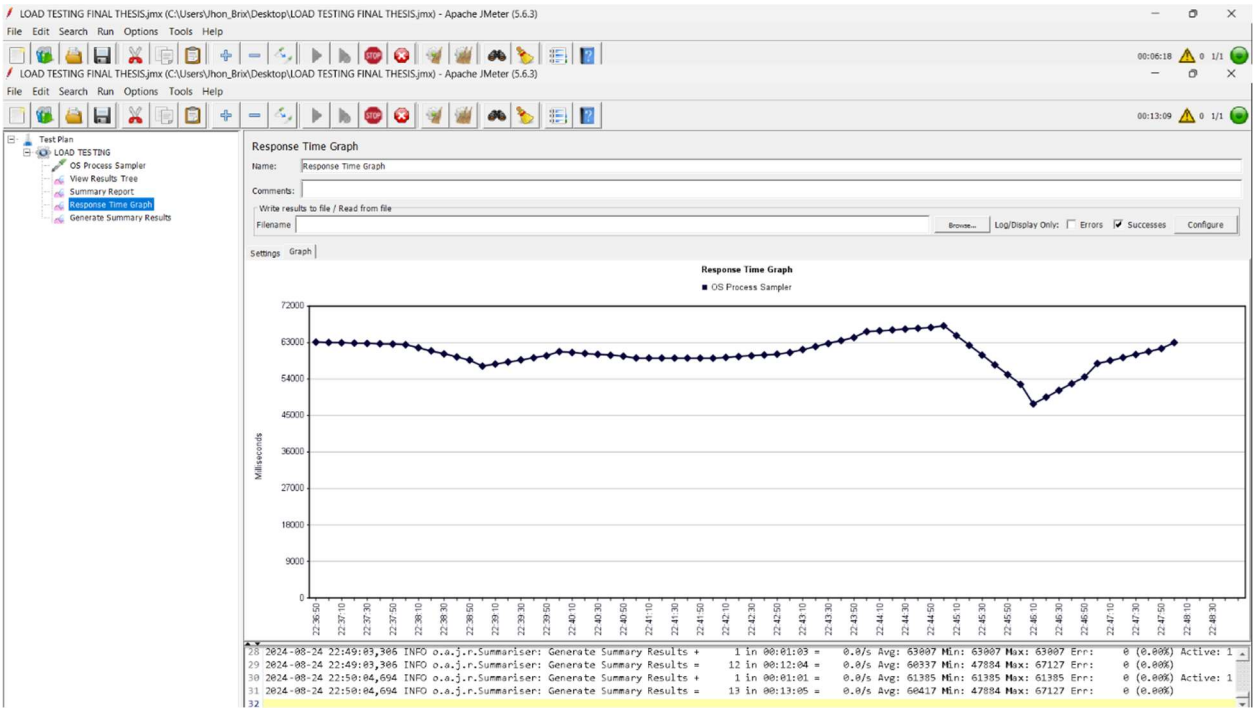


Figure 67: Response Time Graph after 10 Minutes of Runtime

Shown in Figure 67 is the Response Time Graph recorded after the system ran for 10 minutes. The X-axis represents the time data was recorded, and the Y-axis shows the response time of the System. After 10 minutes the average response time based on 10 measurements was 60,319 milliseconds, equivalent to approximately 60.32 seconds

Table 16: Summary of Results after 30 Minutes of Runtime

Label	LOAD TESTING:OS Process Sampler
# Samples	31 executions
Average	57831 ms
Min	27916 ms
Max	84313 ms
Std. Dev.	9829.85 ms
Error %	0.00%
Throughput	0.01729 requests per second
Received KB/sec	0.01 KB/sec
Sent KB/sec	0 KB/sec
Avg. Bytes	750.4 B

Shown in Table 16 are the results after the system ran for 30 minutes, The 30-minute testing checkpoint revealed a dramatic and concerning shift in system performance that raises serious questions about resource management and sustained operational capability. The system completed 31 executions with an average response time that slightly improved to 57,831 milliseconds (57.83 seconds), which initially appears positive and can be attributed to CPU, GPU, and RAM caching mechanisms that optimize frequently accessed resources and computational pathways. However, this improvement on the surface hid serious underlying issues that came up during this phase. What was most alarming was the throughput collapse. It went from 59.7124 requests per second at 10 minutes down to just 0.01729 requests per second. This represented a 99.97% decrease in processing capacity. From a practical standpoint, the system became non-functional. This dramatic collapse strongly suggested severe resource exhaustion. Memory leaks were

possible. Fundamental architectural limitations in managing sustained operations might exist too. The response time range expanded dramatically - from 19,243 milliseconds to 56,397 milliseconds. The minimum dropped to 27,916 milliseconds. The maximum increased to 84,313 milliseconds. This indicated that some executions benefited from caching while others suffered significantly from resource contention. Standard deviation nearly doubled to 9,829.85 milliseconds. This was 17% of the average response time. Performance started getting increasingly unpredictable.Users would experience wildly inconsistent response times - ranging from relatively quick to unacceptably slow. Data transfer characteristics stayed similar to the 10-minute measurement. 0.01 KB/sec got received and 0 KB/sec got sent. Average bytes per request was 750.4 bytes. Despite these severe performance problems, the system kept its perfect 0.00% error rate. This proved that while performance suffered dramatically, the system didn't crash or produce invalid outputs. This phase represented the critical failure point. The system went from acceptable performance to a severely degraded state that would be unacceptable in any production environment. Memory management optimization and resource pooling strategies were urgently needed.

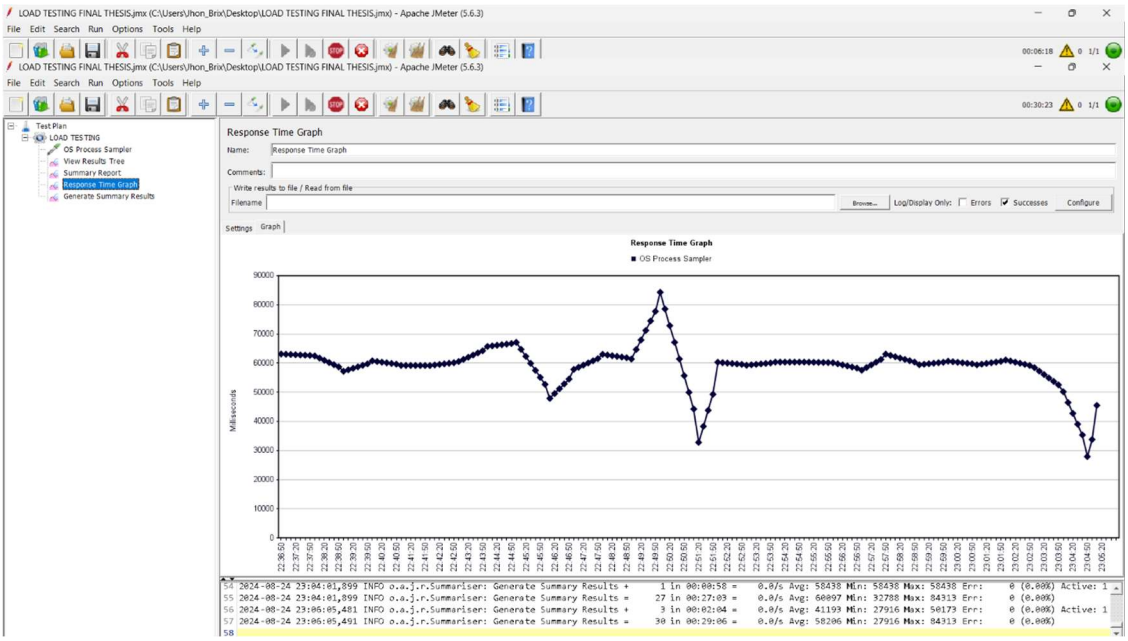


Figure 68: Response Time Graph after 30 Minutes of Runtime

Shown in figure 68 is the Response Time Graph recorded after the system ran for 30 minutes. X-axis and Y-axis from figure 67 also applies here. After 30 minutes the average response time based on 31 measurements was 57,831 milliseconds, equivalent to approximately 57.83 seconds

Table 17: Summary of Results after 1 Hour of Runtime

Label	LOAD TESTING:OS Process Sampler
# Samples	62 executions
Average	58116 ms
Min	17899 ms
Max	84513 ms
Std. Dev.	10671.71 ms
Error %	0.00%
Throughput	1.02818 requests per second
Received KB/sec	0.02 KB/sec
Sent KB/sec	0 KB/sec
Avg. Bytes	1165.1 B

Table 17 shows the results after 1 hour of system running. The final testing checkpoint at 60 minutes revealed something interesting. The system had recovered partially from the catastrophic degradation seen at 30 minutes. However, compared to initial performance levels, it stayed in a significantly compromised state. The system completed 62 total executions. Average response time was 58,116 milliseconds (58.12 seconds). This stayed similar to the 30-minute measurement. The system had stabilized but at degraded performance levels rather than recovering to optimal operation.

Throughput improved substantially from the 30-minute low point. It reached 1.02818 requests per second. This represented meaningful recovery but still was a 98.3% decrease from the initial 59.7124 requests per second measured at 10 minutes. The system never fully recovered its initial processing capacity.

Response time range expanded to its widest observed span. Minimum was 17,899 milliseconds. Maximum was 84,513 milliseconds. A decreasing minimum means the system could perform better than under optimal conditions. Cached resources and unfragmented memory helped this. An increasing maximum indicated that worst-case scenarios became worse and worse as operation kept going.

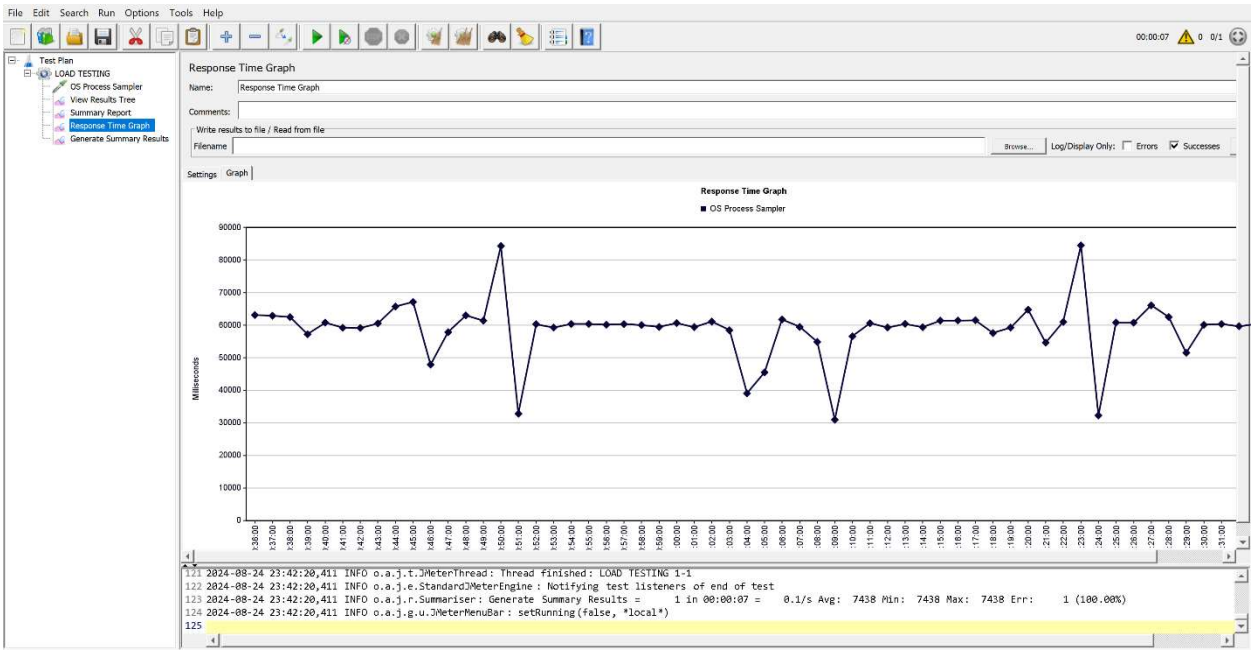


Figure 69: Response Time Graph after 1 Hour of Runtime

Shown in Figure 69 is the Response Time recorded after 1 hour of the System's runtime. Like Figures 67 and 68, X-axis represents time the data was recorded and plotted, and the Y-axis represents the response time of the system. After 1 hour the average response time based on 62 measurements was 58,116 milliseconds, equivalent to approximately 58.12 seconds

Evaluation of Results

The comprehensive evaluation showed that the system successfully did what it was supposed to do. Core face and object detection functionality using YOLOv8 and YuNet algorithms worked. Stability was exceptional with zero errors across all 62 executions. This demonstrated perfect fault tolerance in algorithmic execution. YOLOv8 and YuNet models performed their intended detection tasks reliably. No crashes or invalid outputs occurred. This proved the robustness of the chosen computer vision algorithms.

However, despite the algorithmic reliability, the system suffered from critical performance degradation patterns. These related to resource management. The system was unsuitable for production deployment without significant optimization. The catastrophic 99.97% throughput collapse at 30 minutes was concerning. Performance variability doubled. Response time range expanded by 246%. These indicated severe resource management issues. Memory leaks were likely. Architectural limitations in how the system handled sustained operations might exist too.

The algorithms themselves functioned correctly. However, the system couldn't maintain consistent performance under sustained single-user load. This raised serious concerns about real-world deployment viability. The system needed improved resource handling mechanisms. These were necessary to support the computational demands of running dual detection algorithms continuously.

5.2.2.2. YOLO V8 Benchmark

BenchmarkV8.py > main

```python
from ultralytics.utils.benchmarks import benchmark, TASK2METRIC
import multiprocessing
from ultralytics import YOLO

def main():

    # Load a model
    model = YOLO("yolov8n.pt")  # load an official model
    model = YOLO("best.pt")  # load a custom model (OUR MODEL)

    # Validate the model
    metrics = model.val()  # no arguments needed, dataset and settings remembered
    metrics.box.map  # map50-95
    metrics.box.map50  # map50
    metrics.box.map75  # map75
    metrics.box.maps  # a list contains map50-95 of each category

    benchmark(model=model, data="CustomDTST\data.yaml", imgsz=640, half=False, device="cuda:0", verbose=False)

if __name__ == '__main__':
    multiprocessing.freeze_support()
    main()


```

Figure 70: YOLO V8 Benchmark Source Code

As shown in figure 70, the source code used in testing and benchmarking the custom YoloV8 model indicated by the file name “best.pt”, which is the output after the training process of the images annotated manually using Roboflow. The source code requires the pre-trained model to be loaded before the custom model that was trained. The model is now then passed through a line of code that validates the dataset and the accuracy of the custom model.

The Benchmark starts by loading the parameters in the “benchmark()” in sequence by entering the path of the custom trained model data of the model file that was exported after annotation, represented by file types “.pt” and “.yaml”. The “data” represents the path of the YAML file used for defining the dataset for benchmarking. The “imgsz” represents the input image size for the model, which will be a single integer by width and height. Example is 640, which has a width and height representation of 640 by 480 pixels. The “half” enables FP16 or half-precision inference, which reduces the memory usage of the

model and can increase the performance of the inference depending on the hardware. The "device" defines the hardware device that will be used for benchmarking, such as using CPU by entering the string value of the parameter ("cpu", "cuda:0", "cuda:1"). Lastly, the verbose shows the messages or the output logs in the command line interface during runtime.

Table 18: YOLO V8 Benchmark on CPU on Battery

Benchmarks complete for best.pt on CustomDTST\data.yaml at imgsz=640 (210.32s)						
	Format	Status?	Size (MB)	metrics/mAP50-95(B)	Inference time (ms/im)	Frames Per Second (FPS)
1	Pytorch	✗	6	None	None	None
2	TorchScript	✓	11.9	0.4911	134.27	7.45
3	ONNX	✓	11.7	0.4911	43.6	22.94
4	OpenVINO	✓	11.8	0.4911	78.59	12.72
5	PaddlePaddle	✓	23.4	0.4911	317.81	3.15
6	NCNN	✓	11.7	0.4911	59.21	16.89

As shown in Table 18, The benchmark done to the YoloV8 model, running on the Computer Processing Unit and on battery power shows the runtime of the YoloV8 Model in different formats which include the YoloV8's format's file sizes in Megabytes (MB), metrics/mAP50-90(B) accuracy, which is 0.49% based on the Trained model, provides a visualization on the performance of the model in detecting objects ranging from 0.50% to 0.95% on different levels of difficulty, inference time (ms/im) and Frames Per Second (FPS).

The results shown in Table 18, the YoloV8 PyTorch format, has a file size of 6 MB, with no metrics, inference time and FPS. The YoloV8 TorchScript format, has a file size of 11.9 MB, has an inference time of 134.27 milliseconds (ms/im), and 7.45 FPS of frame speed. The YoloV8 on ONNX format, has a file size of 11.7 MB, has an inference time of 43.60 (ms/im), and 22.94 FPS of frame speed. The YoloV8 on OpenVINO format, has a file size of 11.8 MB, has an inference time of 78.59 (ms/im), and 12.72 FPS of frame speed. The YoloV8 on PaddlePaddle format, has a file size of 23.4 MB, has an inference time of 317.81 (ms/im), and 3.15 FPS of frame speed. Finally, NCNN has a file size of 11.7 MB, Inference time of 60.35 (ms/im), and 16.57 FPS.

Table 19: YOLO V8 Benchmark on GPU on Battery

Benchmarks complete for best.pt on CustomDTST\data.yaml at imgsz=640 (559.83s)						
	Format	Status?	Size (MB)	metrics/mAP50-95(B)	Inference time (ms/im)	Frames Per Second (FPS)
1	Pytorch	✅	6	0.5226	10.37	96.46
2	TorchScript	✅	11.9	0.4911	134.27	141.05
3	ONNX	✅	11.7	0.4911	43.6	85.97
4	TensorRT	✅	17.2	0.4911	78.59	120.04
5	PaddlePaddle	✅	23.4	0.4911	317.81	2.97
6	NCNN	✅	11.7	0.4911	59.21	16.57

Shown in Table 19, The benchmark done to the YoloV8 model, running on the Graphics Processing Unit (GPU) and on battery power. The results shown in Figure N, the YoloV8 PyTorch format, with metrics/mAP50-95(B) Accuracy of 0.52% , has an inference

time of 10.37 (ms/im), and 96.46 FPS of frame speed. The YoloV8 TorchScript format, has an inference time of 7.09 (ms/im), and 141.05 FPS of frame speed. The YoloV8 on ONNX format, has an inference time of 11.63 (ms/im), and 85.97 FPS of frame speed. The YoloV8 on TensorRT format, has a file size of 18.2 M, a metrics/mAP50-95(B) accuracy of 0.49% has an inference time of 8.33 (ms/im), and 120.04 FPS of frame speed. The YoloV8 on PaddlePaddle format, has an inference time of 336.86 (ms/im), and 2.97 FPS of frame speed. Finally, NCNN Inference time of 60.35 (ms/im), and 16.57 FPS without any changes.

Table 20: YOLO V8 Benchmark on CPU on Charging

Benchmarks complete for best.pt on CustomDTST\data.yaml at imgsz=640 (110.18s)						
	Format	Status?	Size (MB)	metrics/mAP 50-95(B)	Inference time (ms/im)	Frames Per Second (FPS)
1	Pytorch	✗	6	None	None	None
2	TorchScript	✓	11.9	0.4911	68.48	14.6
3	ONNX	✓	11.7	0.4911	25.39	39.38
4	OpenVINO	✓	11.8	0.4911	35.52	28.16
5	PaddlePaddle	✓	23.4	0.4911	235.84	4.24
6	NCNN	✓	11.7	0.4911	41.98	23.82

Shown in Table 20, The benchmark done to the YoloV8 model, running on the CPU and charging. The results shown in Figure N, the YoloV8 PyTorch format, with no metrics, inference time and FPS. The YoloV8 TorchScript format, has an inference time of 68.48 (ms/im), and 14.60 FPS of frame speed. The YoloV8 on ONNX format, has an

inference time of 25.39 (ms/im), and 39.38 FPS of frame speed. The YoloV8 on OpenVINO format, has a file size of 11.8 MB has an inference time of 35.52 (ms/im), and 28.16 FPS of frame speed. The YoloV8 on PaddlePaddle format, has an inference time of 235.86 (ms/im), and 4.24 FPS of frame speed. Finally, NCNN Inference time of 41.98 (ms/im), and 23.82 FPS without any changes.

Table 21: YOLO V8 Benchmark on GPU on Charging

Benchmarks complete for best.pt on CustomDTST\data.yaml at imgsz=640 (338.24s)						
	Format	Status?	Size (MB)	metrics/mAP 50-95(B)	Inference time (ms/im)	Frames Per Second (FPS)
1	Pytorch	✅	6	0.5226	7.75	129.06
2	TorchScript	✅	11.9	0.4911	6.37	157.06
3	ONNX	✅	11.7	0.4911	8.87	112.79
4	TensorRT	✅	17.2	0.4911	5.84	171.27
5	PaddlePaddle	✅	23.4	0.4911	242.51	4.12
6	NCNN	✅	11.7	0.4911	45.3	22.07

As shown in Table 21, The benchmark done to the YoloV8 model, running on the Graphics Processing Unit (GPU) and charging. The results shown in Figure N, the YoloV8 PyTorch format, with metrics/mAP50-95(B) Accuracy of 0.52% , has an inference time of 8.18 (ms/im), and 122.22 FPS of frame speed. The YoloV8 TorchScript format, has an inference time of 6.61 (ms/im), and 151.27 FPS of frame speed. The YoloV8 on ONNX format, has an inference time of 8.98 (ms/im), and 111.32 FPS of frame speed. The YoloV8 on TensorRT format, has an inference time of 5.46 (ms/im), and 183.11 FPS of frame

speed. The YoloV8 on PaddlePaddle format, has an inference time of 237.21 (ms/im), and 4.22 FPS of frame speed. Finally, NCNN Inference time of 43.95 (ms/im), and 22.75 FPS without any changes.

Table 22: Benchmark Summary Table

YOLO V8 Model Formats	CPU	GPU
Pytorch	✓	✓
Torchscript	✓	✓
ONNX	✓	✓
OpenVINO	✓	Not supported on GPU
TensorRT	Not supported on CPU	✓
TensorFlow Lite	✓	Not supported on GPU
PaddlePaddle	✓	✓
NCNN	✓	✓

Shown in Table 22, the summary of the output of the YoloV8 Algorithm's model benchmark after running the source code for benchmarking indicates the compatibility for each model format in running the model's Inference or detection in the computer's Computer Processing Unit (CPU), as well as the Graphic Processing Unit (GPU). Formats such as the Pytorch, Torchscript, ONNX, OpenVINO, TensorRT, PaddlePaddle, and NCNN are compatible with the Windows Operating System. OpenVINO does not support runtime on the GPU while the TensorRT does not support runtime on CPU.

5.2.2.3. YUNET Benchmark

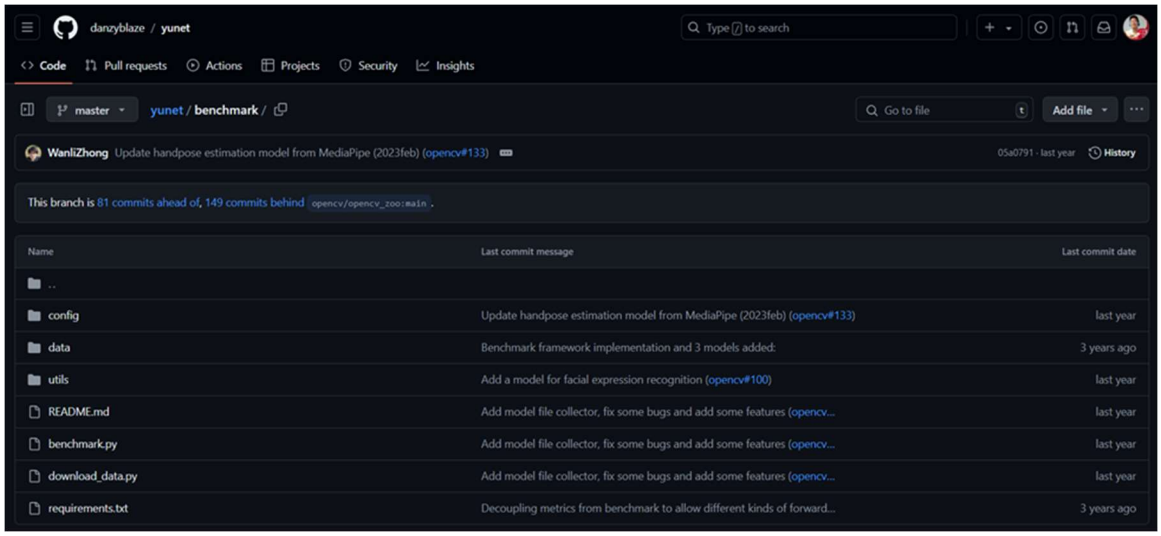


Figure 71: YUNET Benchmark Source Code Repository

Shown in Figure 71, the Github repository for the utilities and tools used for benchmarking ONNX runtime models such as the Yunet algorithm model that uses the onnx format.

```
22
23    # Single config
24    python benchmark.py --cfg ./config/face_detection_yunet.yaml
25
```

Figure 72: YUNET Benchmark Terminal Command

Shown in the Figure 70, the command for the benchmarking of the YuNet Algorithm to be used in the command line. The benchmarking source code used in the benchmarking was from the Open-Source OpenCV Zoo Github repository for benchmarking ONNX format Models such as the Yunet Algorithm's ONNX model. The configuration file "*face_detection_yunet.yaml*" points to the folder where the YuNet facial detection ONNX Models are located and are to be benchmarked based on the image data that is present in the data folder used for testing.

Table 23: YUNET Benchmark Output

Mean (ms)	Median (ms)	Minimum (ms)	Input Size	FPS	Inference (ms)
69.69	70.25	68.10	[160,120]	14.35	69.69

Table 23 shows the source code used to benchmark the YuNet Algorithm's model. It displays the mean (ms), median (ms), and minimum (ms) execution times. The benchmark script calculates and reports several key performance metrics for each model and input size combination. Execution time over multiple runs gets measured. Then mean, median, and minimum times are computed. All get expressed in milliseconds for easy comprehension.Mean time represents average performance. Median provides insight into typical execution time and is less affected by outliers. Minimum time showed the best-case scenario for model inference. Additionally, Frames Per Second (FPS) gets calculated as the inverse of mean time. This means the model's real-time processing capability. The benchmark presents these metrics alongside the model's name and configuration. This offers a comprehensive view of each model's performance characteristics. Easy comparison and evaluation of different models and setups becomes possible.

5.2.2.4. Runtime Testing

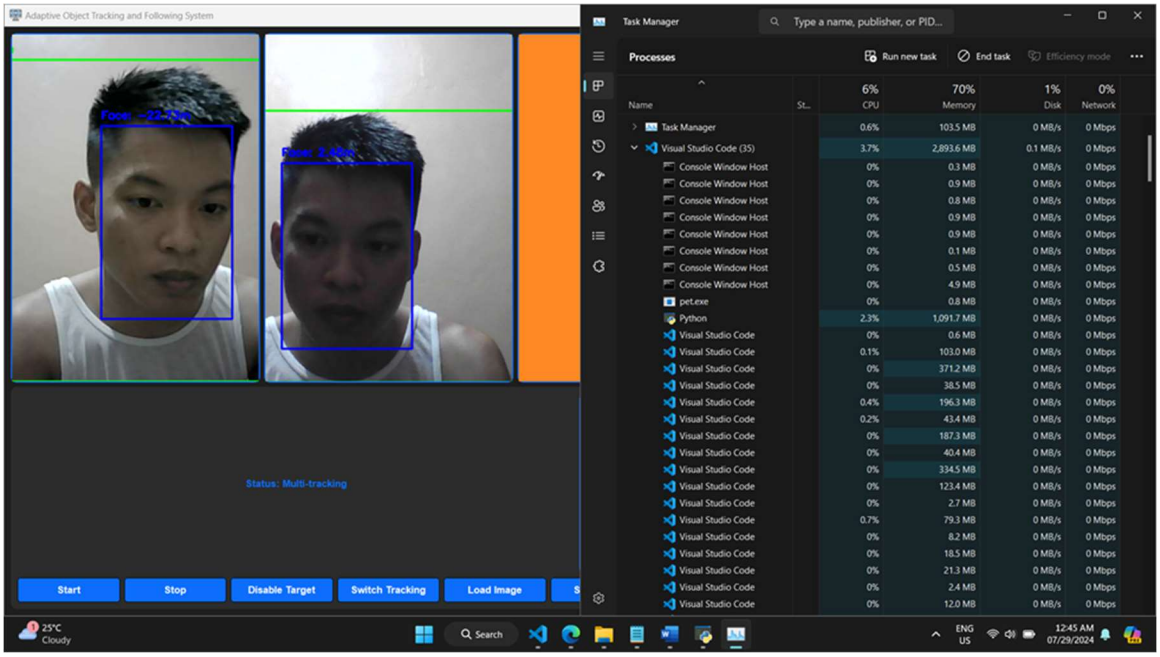


Figure 73: System CPU and Memory Usage

As shown in figure 71, the CPU usage and Memory usage of the System. The system runs primarily runs on the python executable during runtime. The CPU usage for the executable file “python.exe” during the runtime of the system is 2.3% of CPU Utilization and 6.66% of 16 GB of RAM Utilization.

CHAPTER 6

CONCLUSION AND RECOMMENDATION

6.1. Conclusion

"In conclusion, our research successfully implemented an adaptive object tracking and following system. It used YOLOv8 and Yunet Fast R-CNN algorithms. Testing confirmed the system's effectiveness. This included black box and load testing.

1. Throughout the entire research process, YOLOv8 and Yunet algorithms got integrated. This ensured compatibility and enabled the detection of multiple objects and facial recognition. Single tracking and multiple tracking functionalities were developed too. These worked for facial recognition within the camera's range.
2. Based on the results and findings from the previous chapter, thorough performance and functionality testing got conducted on the system. The outcomes were excellent. These achievements showed the effectiveness and reliability of our approach in advancing object tracking and facial recognition technology.

Hardware constraints and time limitations were the most significant challenges during system development. Overcoming these obstacles required careful planning, resource allocation, and innovative problem-solving strategies.

6.2. Recommendation

For the future works and researchers, here are some recommendations:

1. Additional user interface for future use.
2. Used high end hardware to increase the speed of the live detection process.
3. Add more features such as a database for recording objects detected and for future statistics reports.
4. Implement Resource Pooling for Better memory management to prevent resource exhaustion
5. Add Caching Mechanisms to Reduce redundant processing for repeated operations
6. Memory Leak Investigation to identify the throughput drop that causes memory leaks
7. Consider GPU acceleration or distributed processing

REFERENCES

[1] S. Aron, "Object Detection: Literature Review," in Proceedings of the [Conference Name], pp. 485-494, 2023. doi: 10.1007/978-3-031-27524-1_46.

[2] C. Chavan and G. Jadhav, "Computer Vision Application Analysis based on Object Detection," Indian Scientific Journal of Research in Engineering and Management, vol. 07, no. 04, pp. 485-494, 2023. doi: 10.55041/ijsrem19015.

[3] "Detection and Tracking the Criminal Activity using Network of CCTV cameras | IEEE Conference Publication | IEEE Xplore," IEEE Xplore, [Online]. Available: https://ieeexplore.ieee.org/document/9952104 (accessed Jan. 27, 2024).

[4] Z. Shao, L. Wang, Z. Wang, W. Du, and W. S. Wu, "Saliency-Aware Convolution Neural Network for Ship Detection in Surveillance Video," IEEE Trans. Circuits Syst. Video Technol., vol. 30, pp. 781–794, 2019.

[5] J. T. Zhou, J. Du, H. Zhu, X. Peng, Y. Liu, and R. S. M. Goh, "AnomalyNet: An anomaly detection network for video surveillance," IEEE Trans. Inf. Forensics Sec., vol. 14, pp. 2537–2550, 2019.

[6] L. Hu and Q. Ni, "IoT-driven automated object detection algorithm for urban surveillance systems in smart cities," IEEE Internet Things J., vol. 5, pp. 747–754, 2018.

[7] A. Li, Z. Miao, Y. Cen, X.-P. Zhang, L. Zhang, and S. Chen, "Abnormal event detection in surveillance videos based on low-rank and compact coefficient dictionary learning," Pattern Recognit., vol. 108, p. 107355, 2020.

[8] L. Ye, T. Liu, T. Han, H. Ferdinando, T. Seppänen, and E. Alasaarela, "Campus Violence Detection Based on Artificial Intelligent Interpretation of Surveillance Video Sequences," Remote Sens., vol. 13, p. 628, 2021.

[9] M. Li, X. Cao, Q. Zhao, L. Zhang, and D. Meng, "Online Rain/Snow Removal from Surveillance Videos," IEEE Trans. Image Process., vol. 30, pp. 2029–2044, 2021.

[10] P. Zhang, T. Zhuo, L. Xie, and Y. Zhang, "Deformable object tracking with spatiotemporal segmentation in big vision surveillance," Neurocomputing, vol. 204, pp. 87–96, 2016.

[11] Q. Zou, H. Ling, Y. Pang, Y. Huang, and M. Tian, "Joint Headlight Pairing and Vehicle Tracking by Weighted Set Packing in Nighttime Traffic Videos," IEEE Trans. Intell. Transp. Syst., vol. 19, pp. 1950–1961, 2018.

[12] S. Angadi and S. Nandyal, "A Review on Object Detection and Tracking in Video Surveillance," SSRN, 2020. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3710658 (accessed Jan. 14, 2024).

[13] A. Gordon, N. Hayer, Q. Mayet, and G. Densham, "Electronic tracking device for camera and related system for controlling image output of the camera," 2021.

[14] S. Su and Y. Huang, "Tracking camera system and method," 2020.

[15] Haixin, Sun., Vincent, Fremont. (2022). Object Tracking with a Fusion of Event-Based Camera and Frame-Based Camera. Intelligent systems with applications, 250-264. doi: 10.1007/978-3-031-16078-3_15

[16] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," Apr. 2018. [Online]. Available: http://arxiv.org/abs/1804.02767.

[17] Hanyang, Peng., Shiqi, Yu. (2023). YuNet: A Tiny Millisecond-level Face Detector. Machine Intelligence Research, 1-10. doi: 10.1007/s11633-023-1423-y

[18] C. Zhu Chuting, "Multi-Camera People Detection and Tracking," 2019.

[19] A. Antonio Mederos-Barrera, C. Cristo Jurado-Verdu, G. Victor Guerra, R. Jose Rabadan, and P. Rafael Perez-Jimenez, "Discovering and Tracking-based Detection System for Optical Camera Communication," pp. 1-6, 2020. doi: 10.1109/CSNDSP49049.2020.9249637.

[20] T. Pushpendra and S. Pawan Singh, "Object Detection Using Various Camera System," Journal of Informatics Electrical and Electronics Engineering (JIEEE), vol. 3, no. 1, pp. 1-7, 2022. doi: 10.54060/jieee/003.01.006.

[21] J. Bochkovskyi, V. Kovalenko, A. Lapin, Y. Shvets, M. Chen, P. Krylov, M. Gongalsky, V. Strelkov, Y. Zhu, K. Emelyanov, et al., "YoLov8: Deep learning object detector," arXiv preprint arXiv:2017.08293, 2020.

[22] H. Yang, Z. Wang, S. Zhu, S. Bai, J. Yao, S. Jiang, and F. Wang, "A survey of real-time visual object detection methods," in arXiv preprint arXiv:2005.09555, 2020.

[23] D. Comaniciu and P. Meer, "Mean shift: An effective approach for segmentation of complex patterns," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 1999, pp. 509–516.

[24] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in Proceedings of the International Conference on Artificial Intelligence, 1981, pp. 674–679.

[25] J. Shi and C. Tomasi, "Good features to track," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 1994, pp. 593–600.

[26] O. Seiskari, et al., "HybVIO: Pushing the Limits of Real-time Visual-inertial Odometry," in 2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), Waikoloa, HI, USA, 2022, pp. 287-296. doi: 10.1109/WACV51458.2022.00036.

[27] Q. Wang, et al., "Learning Attentions: Residual Attentional Siamese Network for High Performance Online Visual Tracking," in 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, 2018, pp. 4854-4863.

[28] M. Hussain, "YOLO-v1 to YOLO-v8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection," Machines, vol. 11, no. 7, p. 677, Jun. 2023, doi: 10.3390/machines11070677.

[29] J. Terven and D. Cordova-Esparza, "A Comprehensive Review of YOLO: From YOLOv1 and Beyond," Apr. 2023, [Online]. Available: http://arxiv.org/abs/2304.00501.

[30] F. Fischnaller and M. Monzani, "Tracking the Net," in Proceedings Seventh International Conference on Virtual Systems and Multimedia, Berkeley, CA, USA, 2001, pp. 397-406, doi: 10.1109/VSMM.2001.969695.

[31] T. Diwan, G. Anirudh, and J.V. Tembhurne, "Object detection using YOLO: challenges, architectural successors, datasets and applications," Multimed Tools Appl, vol. 82, pp. 9243–9275, 2023. [Online]. Available: https://doi.org/10.1007/s11042-022-13644-y

[32] N. Sharma, S. Baral, M.P. Paing, R. Chawuthai, "Parking Time Violation Tracking Using YOLOv8 and Tracking Algorithms," Sensors, vol. 23, p. 5843, 2023. [Online]. Available: https://doi.org/10.3390/s23135843

[33] Z. Xiang, T. Tao, L. Song, D. Zaopeng, Y. Mao, S. Chu, and H. Wang, "Object tracking algorithm for unmanned surface vehicle based on improved mean-shift method," Int J Adv Robotic Syst, vol. 17, p. 1729881420925294, 2020. [Online]. Available: https://doi.org/10.1177/1729881420925294

[34] Z. Wang, L. Zheng, Y. Liu, and S. Wang, "Towards Real-Time Multi-Object Tracking," arXiv preprint, arXiv:1909.12605v1, 2019. [Online]. Available: https://arxiv.org/pdf/1909.12605v1.pdf

[35] I. Vyacheslav, A. Kuzovatov, A. Kytmanov, I. Olga, and Kuzovatova, "Algorithm for Constructing an Analogue of the Binet Formula," Programming and Computer Software, vol. 46, no. 2, pp. 105-109, 2020. doi: 10.1134/S0361768820020061

[36] B. Stef, "YOLOv8 Object Tracking using PyTorch, OpenCV and DeepSORT," GitHub, 2021. [Online]. Available: [Link]

[37] CodingMantras, "Real-time Object Detection and Tracking with YOLOv8 and Streamlit," GitHub, 2021. [Online]. Available: [Link]

[38] S. Singh, A. Suri, J. N. Singh, M. Singh, and D. K. Yadav, "Object Identification and Tracking Using YOLO Model: A CNN Based Approach," in Advances in Intelligent Systems and Computing, vol. 1356, S. K. Singh, A. K. Singh, and S. K. Singh, Eds. Singapore: Springer, 2021, pp. 173-184. [Online].

[39] V. Viswanatha, R. K. Chandana, and A. C. Ramachandra, "Real Time Object Detection System with YOLO and CNN Models: A Review," arXiv preprint arXiv:2208.00773, 2022. [Online]. Available: [Link]

[40] J. R. Terven and D. M. Cordova-Esparaza, "A Comprehensive Review of YOLO: From YOLOv1 to YOLOv8 and Beyond," in Advances in Intelligent Systems and Computing, vol. 1356, S. K. Singh, A. K. Singh, and S. K. Singh, Eds. Singapore: Springer, 2021, pp. 173-184. [Online].

[41] Z. Aghababaeyan, M. Abdellatif, L. Briand, S. Ramesh, and M. Bagherzadeh, "Black-box testing of deep neural networks through test case diversity," IEEE Transactions on Software Engineering, 2023.

[42] G. Wang, M. Song, and J. N. Hwang, "Recent advances in embedding methods for multi-object tracking: a survey," arXiv preprint arXiv:2205.10766, 2022.

[43] S. Nazeri, "Comparing The-State-of-The-Art Clustering Algorithms," Medium, Jul. 19, 2023. [Online]. Available: [Link]

[44] (2023). "Development of Algorithm for Person Re-Identification Using Extended Openface Method," pp. 545-561. doi: 10.32604/csse.2023.024450.

[45] K. Adisorn, S. Pongsakorn, S. Sanguansat, N. Thanathip, S. Ngamloed, S. Kanabadee, and L. Wilaiporn, "Evaluation of Masked Face Recognition of FaceNet Implemented with Machine Learning Algorithms," pp. 475-478, 2022. doi: 10.1109/InCIT56086.2022.10067676

[46] K. Doshi, "Types of Testing: Different Types of Software Testing in Detail," 2023.

s